Induction principle for an impredicatively encoded general W-type

Steven Bronsveld, Herman Geuvers, Niels van der Weide

arxiv:2505.13495v1; CC BY4.0

Editorial note. The fixed source is the author Master thesis version, not a doctoral thesis. arXiv lists three authors; native cover lists Bronsveld as author and Geuvers/van der Weide as supervisors. Both original facts retained in SOURCE_NOTE.

Editorial note. All native mathematical chapters, complete author proofs and TikZ diagrams retained. Native front-matter logo and index/TOC output are presentation only and omitted from wrapper; original native source retained. No thesis mathematics rewritten.

Editorial note. W and M general principles are the counted results. Stream special case, supporting quotient construction, categorical lemmas and beta/eta statements are uncounted.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- Impredicative dependent product universe, sigma types, intensional identity types and functional extensionality. Exact author system chapter and full inference-rule appendix retained in author-body.tex; no assumptions added.

1 Introduction

Polymorphic lambda calculus ($\lambda 2$), also known as System F, was first introduced by Jean-Yves Girard in 1971 [Gir71] and John C. Reynolds in 1974 [Rey74]. It is a very expressive system in the sense that it is possible to create both inductive data types, such as natural numbers and lists as well as coinductive data types, such as streams and infinite trees [GTL93; Geu14a]. Under the Curry-Howard correspondence, it is isomorphic to second-order propositional logic.
One of the defining features of System F is that it is an impredicative system. This means that it allows quantification over all types in the universe. This impredicativity enables recursive definitions that are not based on well-foundedness. This allows for the creation of self-referential inductive types and infinite coinductive types. In mathematics, impredicativity is bountiful, for example, the closure of a set A can be defined as $\overline{A} := \bigcap\{C \mid A \subseteq C \wedge C \text{ is closed}\}$. Here, we quantify over all closed sets C, including $\overline{A}$ itself.
The (co)inductive data types definable in System F behave well in the sense that they adhere to the expected formation, introduction, elimination, and computation rules (β-rules). Unfortunately, they do not satisfy the expected uniqueness rules (the η-rules). These uniqueness rules enable us to prove the appropriate (co)induction principles. A concrete example is that it is not possible to derive the induction principle for the natural numbers [Geu01].
We zoom out to place this thesis and System F in a broader context. The field of type theory finds its origins in mathematical logic and the theory of computation. A pioneer in the field is Alonzo Church, who introduced lambda-calculus as a model of computation [Chu40]. He found a way to encode natural numbers by using repeated application of a function. For example, the number zero is defined as $\lambda f.\lambda x.x$ and the number three is defined as $\lambda f.\lambda x.f\ f\ x$. These numbers are currently known as the "Church Numerals". This encoding can be generalized to many other inductive data types with the important characteristic that they can be typed using System F. He also created a system of types to solve some paradoxes in his (untyped) lambda calculus. This system became known as simply typed lambda calculus.
Around the same time as Girard worked on System F, Martin-Löf worked on different and more powerful systems involving dependent types. Contrary to System F, he used a predicative flavor of type theory. He introduced many different systems and many new concepts such as identity types and W-types [MS84]. His work greatly influenced the later field of Homotopy Type Theory (HoTT). This project combines the field of homotopy theory with the field of type theory to create a new foundational mathematical system built on the univalence axiom [HOT13].
In 1984, Thierry Coquand and Gérard Huet introduced The Calculus of Constructions [CH88]. This is a very powerful system that allows for the creation of both data structures and mathematical proofs. It is an extension of System F and uses the same impredicative data type definitions. This system was used as the basis of the proof assistant Coq, named after Coquand. In the ordering of different type systems known as the lambda-cube (or Barendregt cube, after its creator Henk Barendregt), the Calculus of Constructions (λC) is put at the top and simply typed lambda calculus ($\lambda\!\rightarrow$) at put at the bottom [Bar91].
The Coq system evolved away from the use of impredicative data structures in favor of adding inductive types directly [PP89; Pau93]. The main motive was

the lack of induction principles in the System F style inductive types.
In this thesis, we extend the 2018 paper "*Impredicative Encodings of (Higher) Inductive Types*" by Steve Awodey, Jonas Frey, and Sam Speight [AFS18]. In this paper, the authors use a modification of the HoTT system with one impredicative bottom universe to encode System F-style data types in such a way that the uniqueness rules (or η-rules) are satisfied. They take inspiration from the way inductive structures are defined in category theory: as the initial algebra of some functor. They show that these improved types satisfy their respective induction principles.
We affirmatively answer two open questions conjectured in the preceding master's thesis of Sam Speight [Spe17]. The first question is whether the technique can be extended to general inductive types by encoding W-types. In addition, we encode a quotient type and show that it satisfies the induction principle. The second question is whether the dual notion of the technique can be extended to general coinductive types by encoding M-types. We affirmatively answer this question too. We first encode a coinductive stream type and show the bisimulation principle. We then finish by encoding an M-type that satisfies the η-rule and showing the general coinduction principle for M-types.
We give a brief overview of this thesis.
In chapter 2, we discuss the preliminaries of this thesis, such as a basic understanding of category theory, algebras, coalgebras, and some type theory.
In chapter 3, we introduce the type system that we shall use throughout this thesis. We elaborate on System F and the link with the system of Awodey, Frey, and Speight. We introduce Π-types, Σ-types, intensional identity types, and the axioms of function extensionality and uniqueness of identity proofs (UIP).
In chapter 4, we introduce an impredicative list type. We then follow the technique of impredicativity encoding the type to obtain a list type that satisfies the η-rule. We proceed to prove the induction principle for this encoded list type.
In chapter 5, we introduce an impredicative quotient type. We again encode this type such that it satisfies the required η-rule in order to prove the induction principle for quotients.
In chapter 6, we define an coinductive stream type. We use our newly defined quotient types to create an impredicative encoding. We show that this encoding satisfies the η-rule and prove the coinduction principle (also known as bisimulation) for this stream type.
In chapter 7, we take a step back and look at the general pattern of the encodings of the previous chapters in order to apply the technique to general inductive and coinductive types.
In chapter 8, we define impredicative W-types. We make an impredicative encoding of these types and show that this satisfies the η-rule. We prove the induction principle for W-types, showing that we can create general inductive types.
In chapter 9, we define impredicative M-types. We create an impredicative encoding of these types using a quotient. We show that this encoding satisfies the η-rule. We finish by proving that the coinduction principle holds for these new M-types.

2 Preliminaries

2.1 Category theory

In this thesis, we assume some basic knowledge of category theory. We make use of initial and final objects and of products and coproducts. We also assume some basic diagram-reading abilities. We denote a morphism f between objects a and b by $a \xrightarrow{f} b$.
For products, we denote the projection functions as $(\pi_1 : A \times B \to A)$ and $(\pi_2 : A \times B \to B)$. We denote the pairing function as $\langle _ , _ \rangle$. For coproducts, we denote the injection functions as $(\text{inl} : A \to A + B)$ and $(\text{inr} : B \to A + B)$. We denote the case function as $([_ , _] : A + B \to C)$.
We assume some very basic knowledge about equalizers and coequalizers and how they are related to limits and colimits.

2.2 Algebra

A categorical way to define algebraic or inductive data structures is using F-algebra [JR12].

Definition 2.2.1. An **F-algebra** of an endofunctor $(F : \mathcal{C} \to \mathcal{C})$ is a tuple $\langle X, \alpha \rangle$ where X, also called the **carrier**, is an object of $\mathcal{C}$ and $(\alpha : F(X) \to X)$ is a $\mathcal{C}$-morphism.

Definition 2.2.2. Given F-algebras $\langle X, \alpha \rangle$ and $\langle Y, \beta \rangle$, we say that a $\mathcal{C}$-morphism $(f : X \to Y)$ is an **F-algebra morphism**, or F-morphism if it makes the diagram on the right commute. In formulas, we require that $f \circ \alpha = \beta \circ F(f)$. If f is an F-algebra morphism, we denote this as $(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle)$, implicitly stating that this commutativity requirement is satisfied.

$$\begin{array}{ccc} F(X) & \xrightarrow{\alpha} & X \\ {\scriptstyle F(f)}\downarrow & = & \downarrow{\scriptstyle f} \\ F(Y) & \xrightarrow[\beta]{} & Y \end{array}$$

Lemma 2.2.3. For any F-algebra $\langle X, \alpha \rangle$, the identity function id_X is an F-morphism.

Proof. We have to show that $\text{id}_X \circ \alpha = \alpha \circ F(\text{id}_X)$. This follows directly from the unfolding the definitions: $\text{id}_X \circ \alpha = \alpha = \alpha \circ \text{id}_{F(X)} = \alpha \circ F(\text{id}_X)$ □

Lemma 2.2.4. The composition of two F-morphisms is again an F-morphism.

Proof. Let $\langle X, \alpha \rangle$, $\langle Y, \beta \rangle$ and $\langle Z, \gamma \rangle$ be F-algebra. Assume we have two morphisms $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$ and $\big(g : \langle Y, \beta \rangle \to \langle Z, \gamma \rangle\big)$. We thus have:

$$f \circ \alpha = \beta \circ F(f) \tag{2.1}$$
$$g \circ \beta = \gamma \circ F(g) \tag{2.2}$$

We thus need to show that $\big(g \circ f : \langle X, \alpha \rangle \to \langle Z, \gamma \rangle\big)$. This means we need to show that $(g \circ f) \circ \alpha = \gamma \circ F(g \circ f)$.

$$
\begin{aligned}
(g \circ f) \circ \alpha &= g \circ (f \circ \alpha) \\
&\overset{2.1}{=} g \circ (\beta \circ F(f)) \\
&= (g \circ \beta) \circ F(f) \\
&\overset{2.2}{=} (\gamma \circ F(g)) \circ F(f) \\
&= \gamma \circ (F(g) \circ F(f)) \\
&= \gamma \circ F(g \circ f)
\end{aligned}
$$
□

Using these two lemmas, we can now show that the F-algebras together with the morphisms form a category.

Theorem 2.2.5. Given a category $\mathcal{C}$ and an endofunctor $F : \mathcal{C} \to \mathcal{C}$, the F-algebras together with the F-algebra morphisms form a category F-**Alg**.

Proof. It suffices to check the following.

- Each object $\langle X, \alpha \rangle$ has an identity morphism given by id_X. By 2.2.3 this is indeed an F-morphism.
- For any two morphisms $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$ and $\big(g : \langle Y, \beta \rangle \to \langle Z, \gamma \rangle\big)$, the composition is again a morphism by 2.2.4. □

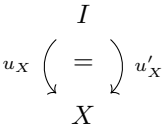
As we now have the category F-**Alg**, we can speak of the *initial* object of this category.

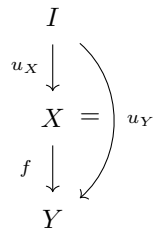
Definition 2.2.6. An F-algebra $\langle I, \gamma \rangle$ is called an **initial** F-algebra if for any F-algebra $\langle X, \alpha \rangle$ there exists an unique F-morphism $\big(u_X : \langle I, \gamma \rangle \to \langle X, \alpha \rangle\big)$. We speak of **weakly initial** algebra if the uniqueness requirement is dropped.

If it exists, this initial object is unique up to isomorphism. Therefore, we can speak of *the* initial object [JR12]. The uniqueness of the morphisms u_X of 2.2.6 can be stated in two equivalent ways as we show in 2.2.7.

Proposition 2.2.7. Let $\langle I, \gamma \rangle$ be an F-algebra. The following are equivalent.

1. For all F-algebras $\langle X, \alpha \rangle$, if $\big(u_X, u'_X : \langle I, \gamma \rangle \to \langle X, \alpha \rangle\big)$ then $u_X = u'_X$.
2. For all F-algebras $\langle X, \alpha \rangle$ and $\langle Y, \beta \rangle$, if $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$, $\big(u_X : \langle I, \gamma \rangle \to \langle X, \alpha \rangle\big)$ and $\big(u_Y : \langle I, \gamma \rangle \to \langle Y, \alpha \rangle\big)$, then $f \circ u_X = u_Y$.





Proof. (1 $\implies$ 2) Assume we have a morphism $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$. Then we also have that $\big(f \circ u_X : \langle I, \gamma \rangle \to \langle Y, \beta \rangle\big)$ is an F-morphism by 2.2.4. We also have that u_Y is such a morphism. Thus by 1) we have $f \circ u_X = u_Y$.
(2 $\implies$ 1) Take for $Y := X$. Let $\big(u_X, u'_X : \langle I, \gamma \rangle \to \langle X, \alpha \rangle\big)$. Notice that $\big(\mathrm{id}_X : \langle X, \alpha \rangle \to \langle X, \alpha \rangle\big)$ is an F-morphism by 2.2.3. Thus by 2) we have $\mathrm{id}_X \circ u_X = u'_X$. We conclude that $u_X = u'_X$. □

2.3 Coalgebra

The dual notion of an algebra is a coalgebra. They can be used to categorically define coinductive data types such as streams and non-wellfounded trees [JR12].

Definition 2.3.1. An **F-coalgebra** of an endofunctor $(F : \mathcal{C} \to \mathcal{C})$ is a tuple $\langle X, \alpha \rangle$ where X, also called the **carrier**, is an object of $\mathcal{C}$ and $(\alpha : X \to F(X))$ is a $\mathcal{C}$-morphism.

Definition 2.3.2. Given F-coalgebras $\langle X, \alpha \rangle$ and $\langle Y, \beta \rangle$, we say that a $\mathcal{C}$-morphism $(f : X \to Y)$ is an **F-coalgebra morphism**, or F-morphism, if it makes the diagram on the right commute. In formulas, we require that $\beta \circ f = F(f) \circ \alpha$. If f is an F-coalgebra morphism, we denote this as $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$, implicitly stating that this commutativity requirement is satisfied.

$$\begin{array}{ccc} X & \xrightarrow{\alpha} & F(X) \\ {\scriptstyle f}\downarrow & = & \downarrow{\scriptstyle F(f)} \\ Y & \xrightarrow[\beta]{} & F(Y) \end{array}$$

We have some similar results to the ones we saw for algebras. The proofs have been omitted as they are nearly identical to the ones for algebras.

Lemma 2.3.3. For any F-coalgebra $\langle X, \alpha \rangle$, the identity morphism $(\mathrm{id}_X : X \to X)$ is an F-coalgebra morphism.

Lemma 2.3.4. The composition of two F-coalgebra morphisms is an F-coalgebra morphism.

These two lemmas allow us to define the category of F-coalgebras.

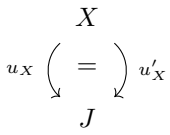
Theorem 2.3.5. Given a category $\mathcal{C}$ and an endofunctor $(F : \mathcal{C} \to \mathcal{C})$, the F-coalgebras together with F-coalgebra morphisms form a category, denoted as F-**CoAlg**.

Definition 2.3.6. An F-coalgebra $\langle J, \gamma \rangle$ is said to be **final** if for any F-coalgebra $\langle X, \alpha \rangle$, there exists a unique F-coalgebra morphism $\big(u_X : \langle X, \alpha \rangle \to \langle J, \gamma \rangle\big)$. We speak of **weakly final** coalgebra if the uniqueness requirement is dropped.

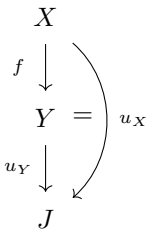
If it exists, this final object is unique up to isomorphism. Therefore, we can speak of *the* final object [JR12] The uniqueness of the morphisms u_X of 2.3.6 can be stated in two equivalent ways as we show in 2.3.7.

Proposition 2.3.7. Let $\langle J, \gamma \rangle$ be an F-coalgebra. The following are equivalent.

1. For all F-algebras $\langle X, \alpha \rangle$, if $\big(u_X, u'_X : \langle X, \alpha \rangle \to \langle J, \gamma \rangle\big)$ then $u_X = u'_X$.



2. For all F-algebras $\langle X, \alpha \rangle$ and $\langle Y, \beta \rangle$, if $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$, $\big(u_X : \langle X, \alpha \rangle \to \langle I, \gamma \rangle\big)$ and $\big(u_Y : \langle Y, \alpha \rangle \to \langle J, \gamma \rangle\big)$,
then $u_X = u_Y \circ f$.



Proof. ($1 \implies 2$) Assume we have a morphism $\big(f : \langle X, \alpha \rangle \to \langle Y, \beta \rangle\big)$. Then we also have that $\big(u_Y \circ f : \langle X, \gamma \rangle \to \langle J, \beta \rangle\big)$ is an F-morphism by 2.3.4. We also have that u_X is such a morphism. Thus by 1) we have $u_X = u_Y \circ f$.
($2 \implies 1$) Take for $Y := X$. Let $\big(u_X, u'_X : \langle X, \alpha \rangle \to \langle J, \gamma \rangle\big)$. Notice that $\big(\mathrm{id}_X : \langle X, \alpha \rangle \to \langle X, \alpha \rangle\big)$ is an F-morphism by 2.3.3. Thus by 2) we have $u_X = u'_X \circ \mathrm{id}_X$, thus $u_X = u'_X$. □

2.4 Type theory

We assume some knowledge of type theory. We assume the reader is familiar with simply typed lambda calculus, the Curry-Howard correspondence, and the λ-cube (mainly $\lambda 2$ and λC) [GTL93; Bar91; NG14]. We assume familiarity with typing judgments such as $(\Gamma \vdash a : A)$ and β-reduction. Whenever we have that $(\Gamma \vdash a : A)$, we say that a is an inhabitant, element, or member of the type A. We often drop the context Γ and write $(a : A)$. We furthermore expect some familiarity with Π-types, Σ-types, and equality types.

In this thesis, we make use of ideas from category theory to create inductive and coinductive data types. We keep these categorical statements separate from statements within the type system. We denote statements within the type system using `monospace font`. We believe that an explicit distinction is more accessible to the reader who is not deeply familiar with category theory.

Under the Curry-Howard correspondence, we can interpret types as propositions and terms as proofs. This means that we can use the system to reason about mathematical statements, and even use these proofs within our definitions. If a proposition holds, we can construct an inhabitant of the proposition, and vice versa. Whenever we prove a lemma and need the proof term, we name the proof term (the witness) next to the number of the lemma (see for example 4.3.3, 4.3.4 and 4.3.8). We furthermore have that the arrow $\to$ and the implication symbol $\implies$ are actually one and the same. We still make use of both for emphasis.

We make use of three notions of equality: syntactical equality, β-equality, and propositional equality (sometimes called definitional equality). Within the type system, we can define terms and types. We say that two terms/types are syntactically equal if they are the same by definition, where we of course allow bounded variable renaming. We write $\mathtt{a} := \mathtt{b}$ both when defining a term/type and to express that two terms/types are syntactically equal. We say that two terms are β-equal if they are equal up to β-reduction. We write $\mathtt{a} \overset{\beta}{=} \mathtt{b}$. We say that two terms are propositionally equal if there is a proof that the two terms are equal. We write $\mathtt{a} = \mathtt{b}$. This notation is also used to denote the corresponding *equality type*. By the Curry-Howard correspondence, we thus have that two terms/types are propositionally equal if we have a proof term $\big(\Gamma \vdash \mathtt{p} : (\mathtt{a} = \mathtt{b})\big)$. Often, we explicitly give the proof term, when showing that two terms are propositionally equal by writing $\mathtt{a} \overset{\mathtt{p}}{=} \mathtt{b}$.

Notation 2.4.1. We denote **function composition** as $f \circ g := \lambda x.f\ (g\ x)$.

3 The system

In this chapter, we outline the system in which we shall work. It was used by Awodey, Frey, and Speight in [AFS18] and detailed in the master thesis of Sam Speight [Spe17]. This system shall be used throughout this thesis. The synopsis is that it is (intensional) Martin-Löf type theory, with one impredicative bottom universe $\mathcal{U}$, sigma-types (Σ-types), intensional identity types (equality types, or $=$-types), impredicative product types (Π-types) and function extensionality. Its inference rules are laid out in chapter C.

There are two ways to look at the system: as an extension of Girard's System F (also known as $\lambda 2$) [GTL93], or as a subset of homotopy type theory (HoTT) with one impredicative bottom universe [HOT13]. The relation with System F is often emphasized by various authors [AFS18; Spe17; Rip23]. This relation boils down to the impredicativity that is often associated with System F and the style of creating data types. System F is very expressive in that it is possible to create both inductive and coinductive data types. Even though our system has humble beginnings in $\lambda 2$, the various extensions make that we actually deal with a system similar in strength to that of the calculus of constructions λC.

We explain our system using a constructive approach, starting with System F and working our way up. This means that we will not delve into the details of the syntax and the inference rules of the system here. Instead, we focus on the most important concepts. For a more formal definition we refer to chapter C, the HoTT book [HOT13] or the master thesis of Sam Speight [Spe17].

We want to shed some light on a foundational principle that underpins this thesis. That is the desire of *defining* instead of *postulating*. In the tradition of foundational mathematics and type theory specifically, there are many different systems with variable degrees of expressivity. These systems vary in the degree to which you can define certain structures or prove certain properties. We strive to create simple, yet powerful systems. There is thus a big difference between a system in which many concepts are postulated and a system in which many concepts can be defined. We shall make this distinction as clear as possible since it is the guiding principle that makes our current system so interesting.

In the next section, we start by showing the basic concepts of System F. After that, we extend these basic principles with the concepts of product-types, sigma-types, equality-types, and the axioms of function extensionality, uniqueness of identity proofs and existential identity.

3.1 System F

System F, or polymorphic λ-calculus ($\lambda 2$), was orignally introduced by Jean-Yves Girard in 1972 [Gir71] and John Reynolds in 1974 [Rey74]. Its typing rules are simple, yet powerful and given in chapter A. We illustrate the power of System F by defining the natural numbers also known as the Church Numerals.

Example 3.1.1. We define the **natural numbers** in System F.

$$\mathbb{N}^* := \prod(X : \mathcal{U}).X \to (X \to X) \to X \qquad \text{(formation)}$$
$$\mathtt{0}^* := \lambda(X : \mathcal{U}).\lambda(z : X).\lambda(s : X \to X).\ z \qquad \text{(introduction)}$$
$$\mathtt{S}^* := \lambda(n : \mathbb{N}^*)\lambda(X : \mathcal{U}).\lambda(z : X).\lambda(s : X \to X).\ s\ (n\ X\ z\ s) \qquad \text{(introduction)}$$

Naturally, we want to check that these definitions are well-typed. We check that the type of $\mathtt{S}^*$ is indeed $\mathbb{N}^* \to \mathbb{N}^*$, so that the judgement $(\vdash \mathtt{S}^* : \mathbb{N}^* \to \mathbb{N}^*)$ holds.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{n : \mathbb{N}^* \vdash n : \mathbb{N}^*}\ \text{id} \quad \dfrac{}{X : \mathcal{U} \vdash X : \mathcal{U}}\ \text{id}}{n : \mathbb{N}^*, X : \mathcal{U} \vdash n\ X : X \to (X \to X) \to X}\ \Pi\text{-elim} \quad \dfrac{}{z : X \vdash z : X}\ \text{id}}{n : \mathbb{N}^*, X : \mathcal{U}, z : X, s : X \to X \vdash n\ X\ z : (X \to X) \to X}\ \to\text{-elim} \quad \dfrac{}{s : X \to X \vdash s : X \to X}\ \text{id}}{\vdots}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\vdots}{n : \mathbb{N}^*, X : \mathcal{U}, z : X, s : X \to X \vdash n\ X\ z\ s : X}\ \to\text{-elim} \quad \dfrac{}{s : X \to X \vdash s : X \to X}\ \text{id}}{n : \mathbb{N}^*, X : \mathcal{U}, z : X, s : X \to X \vdash s\ (n\ X\ z\ s) : X}\ \to\text{-elim}}{n : \mathbb{N}^*, X : \mathcal{U}, z : X \vdash \lambda s.\ s\ (n\ X\ z\ s) : (X \to X) \to X}\ \to\text{-intro}}{n : \mathbb{N}^*, X : \mathcal{U} \vdash \lambda z.\lambda s.\ s\ (n\ X\ z\ s) : X \to (X \to X) \to X}\ \to\text{-intro}}{n : \mathbb{N}^* \vdash \lambda X.\lambda z.\lambda s.\ s\ (n\ X\ z\ s) : \Pi(X : \mathcal{U}).X \to (X \to X) \to X}\ \Pi\text{-intro}}{n : \mathbb{N}^* \vdash \lambda X.\lambda z.\lambda s.\ s\ (n\ X\ z\ s) : \mathbb{N}^*}\ \text{def}}{\vdash \lambda n.\lambda X.\lambda z.\lambda s.\ s\ (n\ X\ z\ s) : \mathbb{N}^* \to \mathbb{N}^*}\ \to\text{-intro}}{\vdash \mathtt{S}^* : \mathbb{N}^* \to \mathbb{N}^*}\ \text{def}$$

This example serves as an illustration of how typing judgments are made. This work is best left to a computer, but it is important to understand the principles behind it.
We still need a way to define functions on the natural numbers. For this purpose, we need a recursor:

$$\mathtt{rec}^*_{\mathbb{N}}\ (X : \mathcal{U})\ (z : X)\ (s : X \to X \to X)\ (n : \mathbb{N}^*) := n\ X\ z\ s \qquad \text{(elimination)}$$

Let's take a moment to consider one of the defining characteristics of System F: it is impredicative. Impredicativity refers to a style of definitions in which one quantifies over the whole universe. This allows for certain self-referential features that are not based on well-foundedness. In the case of the natural numbers, we see that we use $\prod(X : \mathcal{U})$ to define them. This means that X can be any type in the universe. Specifically, we can have $X := \mathbb{N}^*$. We thus

define the natural numbers by quantifying over the whole universe, including the natural numbers that we just defined. A philosophical argument in favor of impredicativity could be to say that our definition is actually a description or schema to describe certain features. In this light, terms of the type $\mathbb{N}^*$ are those that, regardless of the choice of X, satisfy the description $X \to (X \to X) \to X$. The fact that we can apply $\mathbb{N}^*$ to itself is not just a weird consequence, but a necessary feature. For example, if we want to define a function on the natural numbers, that doubles the result, we use the recursor that takes $\mathbb{N}^*$ as an argument:

$$\begin{aligned} &\mathtt{doubleN} : \mathbb{N}^* \to \mathbb{N}^* \\ &\mathtt{doubleN} := \mathtt{rec}^*_{\mathbb{N}}\ \mathbb{N}^*\ \mathtt{0}^*\ (\lambda(n : \mathbb{N}^*).\ \mathtt{S}^*\ (\mathtt{S}^*\ n)) \end{aligned}$$

In addition to the formation, introduction, and elimination rules, we also have that the expected computation rules, or β-rules, hold for the natural numbers. These rules show that the introduction rules and elimination rules interact in the expected way.

Lemma 3.1.2. Let $(X : \mathcal{U})$ be a type. Let $(z : X)$ be an element of X and let $(s : X \to X)$ be some function on X. Furthermore, let $(n : \mathbb{N}^*)$ be a natural number. The following β-rules hold for $\mathbb{N}^*$.

$$\begin{aligned} \mathtt{rec}^*_{\mathbb{N}}\ X\ z\ s\ \mathtt{0}^* &= z && \text{(computation)} \\ \mathtt{rec}^*_{\mathbb{N}}\ X\ z\ s\ (\mathtt{S}^*\ n) &= s\ (\mathtt{rec}^*_{\mathbb{N}}\ X\ z\ s\ n) && \text{(computation)} \end{aligned}$$

Proof. Let $(X : \mathcal{U})$, $(x : X)$, $(s : X \to X)$ and $(n : \mathbb{N}^*)$ be as described. The β-rules hold by unfolding the definitions and applying β-reduction.

$$\begin{aligned} \mathtt{rec}^*_{\mathbb{N}}\ X\ z\ s\ \mathtt{0}^* &:= \mathtt{0}^*\ X\ z\ s \\ &:= \Big(\lambda(X : \mathcal{U}).\lambda(z : X).\lambda(s : X \to X \to X).\ z\Big)\ X\ z\ s \\ &\overset{\beta}{=} z \end{aligned}$$

$$\begin{aligned} \mathtt{rec}^*_{\mathbb{N}}\ X\ z\ s\ (\mathtt{S}^*\ n) &:= (\mathtt{S}^*\ n)\ X\ z\ s \\ &:= \Big(\lambda(n : \mathbb{N}^*).\lambda(X : \mathcal{U}).\lambda(z : X).\lambda(s : X \to X \to X).\ s\ (n\ X\ z\ s)\Big)\ n\ X\ z\ s \\ &\overset{\beta}{=} s\ (n\ X\ z\ s) \\ &=: s\ (\mathtt{rec}^*_{\mathbb{N}}\ X\ z\ s\ n) \end{aligned}$$

□

To highlight the expressivity of System F, we give a few examples of types that you can define within the system. There are many more examples that we shall see throughout this thesis.

Example 3.1.3. We use System F to define a unit, empty, product, and coproduct The details are worked out in chapter B.

$$\begin{aligned}
\mathbf{1}^* &:= \prod(X : \mathcal{U}).X \to X \\
\mathbf{0}^* &:= \prod(X : \mathcal{U}).X \\
A +^* B &:= \prod(X : \mathcal{U}).(A \to X) \to (B \to X) \to X \\
A \times^* B &:= \prod(X : \mathcal{U}).(A \to B \to X) \to X
\end{aligned}$$

3.2 Extending System F

In the previous section, we have seen that System F, or $\lambda 2$, is a very expressive system. It allows us to define many structures. In this section, we extend System F with the concepts of Π-types, Σ-types, =-types, the function extensionality axiom, the uniqueness of identity proofs axiom and a new existential identity axiom. For the complete set of inference rules, we refer to chapter C.
We postulate a cumulative hierarchy of universes $\mathcal{U} : \mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \ldots$, where the bottom universe $\mathcal{U}$ is impredicative[1]. The cumulativity means that types present in the impredicative universe $\mathcal{U}$ are also present in $\mathcal{U}_0$ and types present in U_i are also present in U_{i+1}.
We first extend System F with product types (Π-types). These product types are a generalization of the arrow types of System F that allow us to define dependent functions, where the *type* of the output depends on the *value* of the input. They are defined to allow for impredicative quantification over the universe $\mathcal{U}$.

Definition 3.2.1. We define (impredicative) **product types**. We only give the formation and introduction rules. For the other rules, we refer to chapter C.

$$\frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, a : A \vdash B : \mathcal{U}}{\Gamma \vdash \prod(a : A).B : \mathcal{U}}\ \Pi\text{-form}_1 \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, a : A \vdash B : \mathcal{U}_i}{\Gamma \vdash \prod(a : A).B : \mathcal{U}_i}\ \Pi\text{-form}_2$$

$$\frac{\Gamma, a : A \vdash b : B}{\Gamma \vdash \lambda(a : A).b : \prod(a : A).B}\ \Pi\text{-intro}$$

The first formation rule allows for impredicative quantification since A can be any type (since each type exists in some universe U_i). The second formation rule is the standard predicative formation rule for product types. Note that System F style arrow types can be defined in terms of product types. We shall use the notation $A \to B$ as shorthand for $\prod(a : A).B$. We finally assume the usual elimination and computation and uniqueness rules for Π-types as detailed in [HOT13; Spe17].

Next, we define sigma types (Σ-types). These types are a generalization of the product types definable in System F. They allow us to define dependent pairs, where the *type* of the second component depends on the *value* of the first component.

[1] Note that in [HOT13], $\mathcal{U}$ is used to denote $\mathcal{U}_i$ for some unspecified i.

Definition 3.2.2. We define strong **sigma types**. We only give the formation, introduction, and uniqueness rules. For the other rules, we refer to chapter C.

$$\frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma, a : A \vdash B : \mathcal{U}}{\Gamma \vdash \sum(a : A).B : \mathcal{U}}\ \Sigma\text{-form}_1 \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, a : A \vdash B : \mathcal{U}_i}{\Gamma \vdash \sum(a : A).B : \mathcal{U}_i}\ \Sigma\text{-form}_1$$

$$\frac{\Gamma, x : A \vdash B[x] : \mathcal{U}_i \quad \Gamma \vdash a : A \quad \Gamma \vdash b : B[x := a]}{\Gamma \vdash \langle a, b\rangle : \sum(x : A).B[x]}\ \Sigma\text{-intro}$$

We shall use the notation $A \times B$ as shorthand for $\sum(x : A).B$. We furthermore have the two projection functions ($\texttt{pr1} : \sum(x : A).B \to A$) and ($\texttt{pr2} : \sum(x : A).B \to B$). We allow tuple destructuring and write $\lambda\langle a, b\rangle.M[a, b]$ as shorthand for $\lambda x.M[a := (\texttt{pr1}\ x),\ b := (\texttt{pr2}\ x)]$. Furthermore, the notation $\langle a, b, c\rangle$ is shorthand for $\langle a, \langle b, c\rangle\rangle$, and we syntactically define `pr3`. We finally assume the usual elimination and computation and uniqueness rules for Σ-types as detailed in [HOT13; Spe17] and chapter C.

Embeddings

We can use Σ-types to create embeddings or subtypes. Embeddings are the type theoretic analog of subsets in set theory. Given a predicate ($\texttt{IsEven} : \mathbb{N} \to \mathcal{U}$), we can create a subtype $\texttt{even} := \sum(n : \mathbb{N}).\texttt{IsEven}\ n$. Elements of this type are the even natural numbers. More precisely, they are tuples $\langle n, p\rangle$ where n is a natural number, and p is a proof of the `IsEven` predicate that states that n is indeed even. In summary, we call a Σ-type an embedding if the second component is a proposition (is of type $\mathcal{U}$).
As a consequence of the η-rule for Σ-types, we get the sigma injection principle ($\hookrightarrow$). This principle states that, if the sigma type is an embedding, then the equality of elements can be shown by proving the equality of the first component. Compare this to the concept of subsets: if we take a set X and some subset $Y := \{x \in X \mid P(x)\} \subseteq X$, the subset Y inherits the equality relation of X. In other words, if $x = x'$ in X, then $x = x'$ obviously still holds for $(x, x' \in Y)$.
This principle is formalized below in the following lemma.

Lemma 3.2.3. We have the following sigma injection principle.

$$(\hookrightarrow) : \prod(X :\ \mathcal{U}).\prod(P : X \to \mathcal{U}).\prod\big(y, y' : \sum(x : X).P\ x\big).y = y' \iff \texttt{pr1}\ y = \texttt{pr1}\ y'$$

Proof. We give the proof in chapter C. □

Next, we define intentional equality types (=-types), originally called identity types. These types allow us to define the notion of propositional equality.

Definition 3.2.4. We define **equality types**. We give the formation rules and the introduction rule. For the other rules, we refer to chapter C.

$$\frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash (a =_A b) : \mathcal{U}} =\text{-form}_1 \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash (a =_A b) : \mathcal{U}_i} =\text{-form}_1$$

$$\frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma \vdash a : A}{\Gamma \vdash \texttt{refl} : a =_A a} =\text{-intro}$$

In the future, we drop the subscript A and write $a = b$. When the proof term $(p : a = b)$ is present, we often make its use explicit by writing $a \overset{p}{=} b$.

Next, we add three axioms to the system. The first axiom is the axiom of function extensionality. This axiom states that any two functions that are equal for all values are equal. This axiom is essential in proving many theorems and indispensable when defining W-types.

Axiom 3.2.5. Let X, Y be types in some universe. The **function extensionality** axiom states that any two functions $\left(f, g : \prod(x : X).Y\right)$ are equal if they are equal for all values $(x : X)$. We formally write this as follows:

$$\texttt{FunExt} : \left(\prod(x : X).f\ x = g\ x\right) \implies f = g$$

We add the axiom of uniqueness of identity proofs (UIP). This axiom states that if we have two proofs of a statement, then these proofs are propositionally equal. Strictly speaking, we could do without this axiom by using the notion of 0-types by defining `Set` as those types for which UIP holds. This approach was taken in [AFS18, Introduction] and opens the door for higher inductive types such as $\mathcal{S}_1$. An interesting avenue for future work is to remove this axiom and adapt the relevant defintions.

Axiom 3.2.6. The **uniqueness of identity proofs** axiom states that any two proofs of equality between two elements are equal. We formally write this as follows:

$$\texttt{UIP} : \prod(X : \mathcal{U}) \prod(x, y : X) \prod(p, q : x = y).p = q$$

In section 6.1, we add another axiom to the system (6.1.4). This axiom is best explained in the context of that section. It states that we have a certain identity on existential types.
Now that we have defined our system, we can properly begin our journey. In the next chapter, we introduce the technique of [AFS18] by creating an impredicative encoding of a list type.

4 Lists

In this chapter, we apply the encoding technique of [AFS18] to create a finite list data type that has an induction principle. Our goal is to introduce the reader to the principle idea of encoding the uniqueness principle of an inductive type within its definition.
We begin by giving the common impredicative definition of impredicative lists and show that the β-rules, or computation rules, are satisfied for this definition. Next, we take a look at the categorical definition of inductive lists using a suitable initial algebra. We then combine these definitions into an impredicative encoding of a list type that also satisfies the η-rule. We finish by proving the induction principle for finite lists.
These lists shall range over some element type $(E : \mathcal{U})$, so we speak of lists *over* E. We globally assume that $\big(\Gamma \vdash E : \mathcal{U}\big)$. The following definitions can be straightforwardly generalized to any element type.

4.1 Impredicative list type

Let us begin by giving an impredicative finite list type [GTL93, 11.5.2 Lists].

Definition 4.1.1. We define an **impredicative list** data type $\texttt{List}^*$ over some element type $(E : \mathcal{U})$. We also define two constructors: $\texttt{nil}^*$ and $\texttt{cons}^*$.

$$\texttt{List}^* := \prod(X : \mathcal{U}).\ X \to (E \to X \to X) \to X \qquad \text{(formation)}$$

$$\texttt{nil}^* := \lambda(X : \mathcal{U}).(x : X).(f : E \to X \to X).x \qquad \text{(introduction)}$$

$$\texttt{cons}^*\ (e : E)\ (l : \texttt{List}^*) := \lambda(X : \mathcal{U}).(x : X).(f : E \to X \to X).f\ e\ (l\ X\ x\ f) \qquad \text{(introduction)}$$

Example 4.1.2. Using the element type $E := \mathbb{N}$, we can define the list $[1, 2, 3]$ or $1 : 2 : 3 : []$ as follows:

$$[\ 1,\ 2,\ 3\ ] := \quad \texttt{cons}^*\ 1\ \big(\texttt{cons}^*\ 2\ (\texttt{cons}^*\ 3\ \texttt{nil}^*)\big) : \texttt{List}^*_{\mathbb{N}}$$

It is clear that $\texttt{nil}^*$ and $\texttt{cons}^*$ are of the appropriate type. For an example of a full derivation, we refer to 3.1.1.

$$\begin{aligned} \Gamma \vdash \texttt{nil}^* &: \texttt{List}^* \\ \Gamma, (e : E), (l : \texttt{List}^*) \vdash \texttt{cons}^*\ e\ l &: \texttt{List}^* \end{aligned}$$

Note that we can also create lists of lists. For example $(\Gamma \vdash \texttt{cons}^*\ \texttt{nil}^*\ \texttt{nil}^* : \texttt{List}^*_{\texttt{List}^*_{\mathbb{N}}})$, which is a list with one element: an empty list.

Datatypes are not of much use if we can only construct data: we also want to *do* things with the data. Suppose we want to perform some operation on these lists, for example, mapping the list, or taking the `head` or `tail`. To this end, we need an elimination principle or *recursor*.

This recursor takes four arguments. The first argument $(X : \mathcal{U})$ specifies the output type. If we want to create a function from lists to natural numbers, we would set $X := \mathbb{N}$. If we output a new list, we would set $X := \texttt{List}^*$. The second argument $(x : X)$ specifies what we map the empty list to. The third argument $(g : E \to X \to X)$ specifies the recursive call. The fourth argument $(l : \texttt{List}^*)$ is the list to operate on.

Definition 4.1.3. We define the **recursor** for the $\texttt{List}^*$ type.

$$\texttt{rec}_1^* : \prod(X : \mathcal{U}).X \to (E \to X \to X) \to \texttt{List}^* \to X \quad \text{(elimination)}$$
$$\texttt{rec}_1^*\ X\ x\ g\ l := l\ X\ x\ g$$

Example 4.1.4. We use the list recursor to define a function that sums all elements in a list. We set $X := \mathbb{N}$, $x := 0$ and $g := \lambda e.\lambda n.e + n$.

$$\texttt{sum} : \texttt{List}_{\mathbb{N}}^* \to \mathbb{N}$$
$$\texttt{sum} := \texttt{rec}_1^*\ \mathbb{N}\ 0\ (\lambda e.\lambda n.e + n)$$

Example 4.1.5. We use the list recursor to define a function that maps all elements in a list to their square. We set $X := \texttt{List}_{\mathbb{N}}^*$, $x := \texttt{nil}^*$ and $g := \lambda e.\lambda l.\texttt{cons}^*\ (e * e)\ l$.

$$\texttt{square} : \texttt{List}_{\mathbb{N}}^* \to \texttt{List}_{\mathbb{N}}^*$$
$$\texttt{square} := \texttt{rec}_1^*\ \texttt{List}_{\mathbb{N}}^*\ \texttt{nil}^*\ (\lambda e.\lambda l.\texttt{cons}^*\ (e * e)\ l)$$

This recursor then needs to β-reduce in an expected way. We call these expected reductions the β-rules or computation rules. We show that this recursor indeed satisfies the expected β-rules.

Proposition 4.1.6. The list recursor $\texttt{rec}_1^*$ satisfies the following β-rules. Let $(X : \mathcal{U})$, $(x : X)$ and $(g : E \to X \to X)$.

$$\texttt{rec}_1^*\ X\ x\ g\ \texttt{nil}^* \overset{\beta}{=} x \quad \text{(computation)}$$
$$\texttt{rec}_1^*\ X\ x\ g\ (\texttt{cons}^*\ e\ l) \overset{\beta}{=} g\ e\ (\texttt{rec}_1^*\ X\ x\ g\ l) \quad \text{(computation)}$$

Proof. Let $(X : \mathcal{U})$, $(x : X)$ and $(g : E \to X \to X)$. We derrive the following:

$$\begin{aligned}
\texttt{rec}_1^*\ X\ x\ g\ \texttt{nil}^* &:= \texttt{nil}^*\ X\ x\ g \\
&\overset{\beta}{=} x \\
\texttt{rec}_1^*\ X\ x\ g\ (\texttt{cons}^*\ e\ l) &:= (\texttt{cons}^*\ e\ l)\ X\ x\ g \\
&\overset{\beta}{=} g\ e\ (l\ X\ x\ g) \\
&=: g\ e\ (\texttt{rec}_1^*\ X\ x\ g\ l)
\end{aligned}$$

□

To summarize, we have defined a finite list type $\texttt{List}^*$. We have two constructors or introduction rules: $\texttt{nil}^*$ and $\texttt{cons}^*$. We have a recursion principle $\texttt{rec}_1^*$ that satisfies the expected β-rules. Even though this definition seems to behave quite well out of the box we are unable to show the induction principle [Geu01]. This is because, even though the recursor satisfies the β-rules, it fails to satisfy the η-rule. This rule states that the recursor $\texttt{rec}_1^*\ X\ x\ g$ is unique in an appropriate sense. We would expect, for example that $\texttt{rec}_1^*\ \texttt{List}^*\ \texttt{nil}^*\ \texttt{cons}^* = \texttt{id}_{\texttt{List}^*}$. Using this η-rule, we can prove the induction principle.
In the next section, we jump to the categorical definition of lists using initial algebras. We then combine these definitions to create an impredicative encoding of a list type that satisfies the η-rule. We finish by proving the list induction principle.

4.2 Categorical lists

In this section, we use the theory of algebras of section 2.2 using a suitable functor $\mathcal{L}$. From the theory of algebra, we know that the initial algebra of $\mathcal{L}$-**Alg** is a good candidate for the definition of an inductive list type. We shall take inspiration from the uniqueness property of this initial algebra to define an impredicative list encoding that satisfies the η-rule.

Definition 4.2.1. We define the endofunctor $(\mathcal{L} : \mathcal{C} \to C)$. We leave the category $\mathcal{C}$ unspecified, assuming it is a category with finite products and coproducts and a unit object $\mathbf{1}$, for example the category **Set**. We also assume that $\mathcal{C}$ has some object E. We define $\mathcal{L}$ as follows:

$$\begin{aligned} \mathcal{L}(X) &:= \mathbf{1} + (E \times X) \\ \mathcal{L}(f : X \to Y) &:= [\text{inl}, \langle e, x\rangle \mapsto \text{inr}\langle e, f(x)\rangle] \end{aligned}$$

An $\mathcal{L}$-algebra is a tuple $\langle X, \alpha\rangle$, where $\mathcal{L}(X) \xrightarrow{\alpha} X$. If we unfold the definition of $\mathcal{L}$, we get the following: $\mathbf{1} + (E \times X) \xrightarrow{\alpha} X$. We thus see that α consists of two parts: a function $\mathbf{1} \to X$ and a function $(E \times X) \to X$. We write $\alpha := [x, g]$, where $(x : \star \to X)$ and $(g : (E \times X) \to X)$.
We instantiate the definition of $\mathcal{L}$-morphism given in 2.2.2, where we have that $(y : \star \to Y)$ and $(h : (E \times Y) \to Y)$.

$$\begin{array}{ccc} \mathbf{1} + (E \times X) & \xrightarrow{[x,g]} & X \\ {\scriptstyle [\text{inl},\langle e,x\rangle \mapsto \text{inr}\langle e, f(x)\rangle]}\downarrow & = & \downarrow {\scriptstyle f} \\ \mathbf{1} + (E \times Y) & \xrightarrow[[y,h]]{} & Y \end{array}$$

Figure 1: Commutative diagram of a $\mathcal{L}$-morphism.

If we write out this diagram in formulas we derive the following commutativity requirement:

$$
\begin{aligned}
f \circ [x, g] &= [y, h] \circ [\text{inl}, \langle e, x\rangle \mapsto \text{inr}\langle e, f(x)\rangle] && \iff \\
[f \circ x, f \circ g] &= [y, h \circ (\langle e, x\rangle \mapsto \langle e, f(x)\rangle)] && \iff \\
f \circ x = y \wedge f \circ g &= h \circ (\langle e, x\rangle \mapsto \langle e, f(x)\rangle) && (4.1)
\end{aligned}
$$

Recall that the initial object $\langle I, \gamma\rangle$ of the category of $\mathcal{L}$-algebras $\mathcal{L}$-**Alg** has exactly one morphism $\big(u_X : \langle I, \gamma\rangle \to \langle X, \alpha\rangle\big)$ for each object $\langle X, \alpha\rangle$ in the category L-**Alg**:

$$
\begin{array}{ccc}
1 + (E \times I) & \xrightarrow{\gamma} & I \\
{\scriptstyle \mathcal{L}(u_X)}\downarrow & = & \downarrow{\scriptstyle \exists! u_X} \\
\mathbf{1} + (E \times X) & \xrightarrow[\alpha]{} & X
\end{array}
$$

Figure 2: Commutative diagram of the initial $\mathcal{L}$-algebra.

These $\mathcal{L}$-morphisms u_X should of course satisfy the commutativity requirement of Equation (4.1). Writing down the uniqueness requirement is a bit more tricky. We use requirement 2 from 2.2.7. This states that, given a morphism $\big(f : \langle X, \alpha\rangle \to \langle Y, \beta\rangle\big)$, and morphisms $(u_X : I \to X)$ and $(u_Y : I \to Y)$ (which all satisfy Equation (4.1)), we have that $f \circ u_X = u_Y$. This construction is shown Figure 3. Compare the right-hand side of this diagram with requirement 2 from 2.2.7.

$$
\begin{array}{ccc}
1 + (E \times I) & \xrightarrow{\gamma} & I \\
{\scriptstyle \mathcal{L}(u_X)}\downarrow & = {\scriptstyle [x,g]} & \downarrow{\scriptstyle u_X} \\
\mathbf{1} + (E \times X) & \xrightarrow[\alpha]{} & X \\
{\scriptstyle \mathcal{L}(f)}\downarrow & = {\scriptstyle [y,h]} & \downarrow{\scriptstyle f} \\
\mathbf{1} + (E \times Y) & \xrightarrow[\beta]{} & Y
\end{array}
\qquad = \quad u_Y
$$

Figure 3: Diagram of the uniqueness requirement of an initial $\mathcal{L}$-algebra.

4.3 Impredicative encoding of lists

We are now ready to translate the categorical requirement of the initial $\mathcal{L}$-algebra back to type theory. We first need to create an analog of $\mathcal{L}$-algebras within type theory. We use triples $\langle X, x, g\rangle$ where $(X : \mathcal{U})$, $(x : X)$ and $(g : E \to X \to X)$. These triples can be thought of as the translation of categorical $\mathcal{L}$-algebras $\langle X, [x, g]\rangle$. We start by giving a type theoretical predicate that states that a morphism $(f : X \to Y)$ is an $\mathcal{L}$-algebra morphism.

Definition 4.3.1. Let $(X, Y : \mathcal{U}), (x : X), (y : Y), (g : E \to X \to X)$ and $(h : E \to Y \to Y)$. We define a predicate `MorphList` that states that $(f : X \to Y)$ forms an $\mathcal{L}$-algebra morphism. Compare this predicate with Equation (4.1).

$$\texttt{MorphList}\ X\ x\ g \quad Y\ y\ h \quad (f : X \to Y) := (f\ x) = y \wedge \prod (e :\ E).f \circ (g\ e) = (h\ e) \circ f$$

We can now define a subtype of $\texttt{List}^*$ that encodes the uniqueness requirement of the initial $\mathcal{L}$-algebra. We do this by essentially mimicking the uniqueness rule outlined in Figure 3 (where $u_X := \texttt{rec}_1^*\ X\ x\ g$ and $u_Y := \texttt{rec}_1^*\ Y\ y\ h$). We require that $f\ (\texttt{rec}_1^*\ X\ x\ g) = \texttt{rec}_1^*\ Y\ y\ h$ for all $\mathcal{L}$-morphisms $\big(f : \langle X, [x, g]\rangle \to \langle Y, [y, h]\rangle\big)$.

Definition 4.3.2. We define an **inductive list** type.

$$\texttt{List} := \sum (l : \texttt{List}^*).\ \texttt{LimList}\ l \qquad \text{(formation)}$$

$$\texttt{LimList}\ (l : \texttt{List}^*) := \prod_{X,Y:\mathcal{U}} \prod_{x:X,y:Y} \prod_{f:X\to Y} \prod_{\substack{g:E\to X\to X \\ h:E\to Y\to Y}} (\texttt{MorphList}\ X\ x\ g\quad Y\ y\ h\quad f) \implies f(\texttt{rec}_1^*\ X\ x\ g\ l) = \texttt{rec}_1^*\ Y\ y\ h\ l$$

Let's take a moment to understand this definition. The `List` type is defined using a sigma-type. In other words, its inhabitants are pairs $\langle l, p\rangle$, where $(l : \texttt{List}^*)$ is the previously defined impredicative list (4.1.1). The second component p is a proof term that $(\texttt{LimList}\ l)$ holds. This proof states that for all $\mathcal{L}$-algebras $\langle X, x, g\rangle$ and $\langle Y, y, h\rangle$, and all $\mathcal{L}$-algebra morphisms f, we have that $f\ (\texttt{rec}_1^*\ X\ x\ g\ l) = \texttt{rec}_1^*\ Y\ y\ h\ l$.
This new `List` type could very well be the empty type. We could have that no lists comply with the `LimList` predicate. Fortunately, we both have that $\texttt{nil}^*$ and $\texttt{cons}^*$ satisfy the `LimList` predicate. We can thus define $(\texttt{nil} : \texttt{List})$ and $(\texttt{cons} : E \to \texttt{List} \to \texttt{List})$ in a straightforward manner.

Lemma 4.3.3 (`LimList0`)**.** The proposition $(\texttt{LimList}\ \texttt{nil}^*)$ holds.

Proof. Let $(X, Y : \mathcal{U})$, $(x : X)$, $(y : Y)$, $(f : X \to Y)$, $(g : E \to X \to X)$ and $(h : E \to Y \to Y)$. We assume that we have a proof term of $(\texttt{MorphList}\ X\ x\ g\quad Y\ y\ h\quad f)$. Therefore we obtain that $f(x) = y$. By the β-rules of $\texttt{rec}_1^*$ (4.1.6), we have that $\texttt{rec}_1^*\ X\ x\ g\ \texttt{nil}^* \overset{\beta}{=} x$ and $\texttt{rec}_1^*\ Y\ y\ h\ \texttt{nil}^* \overset{\beta}{=} y$. We conclude that $f(\texttt{rec}_1^*\ X\ x\ g\ \texttt{nil}^*) \overset{\beta}{=} f(x) = y \overset{\beta}{=} \texttt{rec}_1^*\ Y\ y\ h\ \texttt{nil}^*$. □

Lemma 4.3.4 (`LimListC`)**.** For $(e : E)$ and $(l : \texttt{List})$ we have that $(\texttt{LimList}\ (\texttt{cons}^*\ e\ (\texttt{pr1}\ l)))$ holds.

Proof. Let $(X, Y : \mathcal{U})$, $(x : X)$, $(y : Y)$, $(f : X \to Y)$, $(g : E \to X \to X)$ and $(h : E \to Y \to Y)$. We assume that we have a proof term of $(\texttt{MorphList}\ X\ x\ g\quad Y\ y\ h\quad f)$. Therefore we have $(\texttt{morph} : f \circ (g\ e) = (h\ e) \circ f)$. Let $(e : E)$ and $(l : \texttt{List})$. We have that $(\texttt{LimList}\ (\texttt{pr1}\ l))$ holds. Note that $(\texttt{pr2}\ l)$ has type $(\texttt{LimList}\ (\texttt{pr1}\ l))$. We can apply $(\texttt{LimList}\ (\texttt{pr1}\ l))$ with the appropriate variables (X, Y, x, y, f, g, h). We obtain $\big(\texttt{comm} : f(\texttt{rec}_1^*\ X\ x\ g\ (\texttt{pr1}\ l)) = \texttt{rec}_1^*\ Y\ y\ h\ (\texttt{pr1}\ l)\big)$. We derive the following:

$$\begin{aligned} f(\texttt{rec}_1^*\ X\ x\ g\ (\texttt{cons}^*\ e\ (\texttt{pr1}\ l))) &\overset{\beta}{=} f(g\ e\ (\texttt{rec}_1^*\ X\ x\ g\ (\texttt{pr1}\ l))) \\ &\overset{\texttt{morph}}{=} h\ e\ (f(\texttt{rec}_1^*\ X\ x\ g\ (\texttt{pr1}\ l))) \\ &\overset{\texttt{comm}}{=} h\ e\ (\texttt{rec}_1^*\ Y\ y\ h\ (\texttt{pr1}\ l)) \\ &\overset{\beta}{=} \texttt{rec}_1^*\ Y\ y\ h\ (\texttt{cons}^*\ e\ (\texttt{pr1}\ l)) \end{aligned}$$ □

Now that we have proven that `nil` and `cons` satisfy the `LimList` predicate, we can create new introduction rules for the inductive list type `List`. Note that we actually embed the proofs of the `LimList` predicate within the definition!

Definition 4.3.5. We define $\texttt{nil}$, $\texttt{cons}$ and $\texttt{rec}_1$ for the $\texttt{List}$ type using $\texttt{LimList0}$ and $\texttt{LimListC}$.

$$\texttt{nil} := \langle \texttt{nil}^*, \texttt{LimList0} \rangle \qquad \text{(introduction)}$$
$$\texttt{cons}\ (e : E)\ (l : \texttt{List}) := \langle \texttt{cons}^*\ e\ (\texttt{pr1}\ l), \texttt{LimListC}\ e\ l \rangle \qquad \text{(introduction)}$$

We can also define a new recursor that simply uses the old recursor.

$$\texttt{rec}_1 : \prod (X : \mathcal{U}).X \to (E \to X \to X) \to \texttt{List} \to X \qquad \text{(elimination)}$$
$$\texttt{rec}_1\ X\ x\ c\ l := \texttt{rec}_1^*\ X\ x\ c\ (\texttt{pr1}\ l)$$

Given these new introduction and elimination rules, we again need to check that the computation rules (β-rules) are still satisfied.

Proposition 4.3.6. The list recursor $\texttt{rec}_1$ satisfies the following β-rules. Let $(X : \mathcal{U})$, $(x : X)$ and $(f : E \to X \to X)$.

$$\texttt{rec}_1\ X\ x\ f\ \texttt{nil} \overset{\beta}{=} x \qquad \text{(computation)}$$
$$\texttt{rec}_1\ X\ x\ f\ (\texttt{cons}\ e\ l) \overset{\beta}{=} f\ e\ (\texttt{rec}_1\ X\ x\ f\ l) \qquad \text{(computation)}$$

Proof. By β-reduction. □

With this recursor, we have that the triple $\langle \texttt{List}, \texttt{nil}, \texttt{cons} \rangle$ acts as the initial $\mathcal{L}$-algebra. This means that for all triples $\langle X, x, g \rangle$ that act like $\mathcal{L}$-algebra, we have that $(\texttt{rec}_1\ X\ x\ g : \texttt{List} \to X)$ acts as the unique morphism u_X. We ensure that $(\texttt{rec}_1\ X\ x\ g)$ is an $\mathcal{L}$-algebra morphism by proving the following lemma.

Lemma 4.3.7. For all $(X : \mathcal{U})$, $(x : X)$ and $(g : E \to X \to X)$ we have that $(\texttt{rec}_1\ X\ x\ g)$ is an $\mathcal{L}$-algebra morphism. We thus have a proof of the following proposition:

$$\begin{array}{ccc} 1 + (E \times \texttt{List}) & \xrightarrow{[\texttt{nil},\texttt{cons}]} & \texttt{List} \\ {\scriptstyle \mathcal{L}(\texttt{rec}_1\ X\ x\ g)}\downarrow & = & \downarrow{\scriptstyle \texttt{rec}_1\ X\ x\ g} \\ 1 + (E \times X) & \xrightarrow[[x,g]]{} & X \end{array}$$

$$\big(\texttt{MorphList List nil cons}\quad X\ x\ g\quad (\texttt{rec}_1\ X\ x\ g)\big)$$

Proof. Let $(X : \mathcal{U})$, $(x : X)$ and $(g : E \to X \to X)$. For the left conjunct, we have $(\texttt{rec}_1\ X\ x\ g)\ \texttt{nil} \overset{\beta}{=} x$ by the first β-rule. For the right conjunct, we have the following reduction using beta equality and function extensionality.

$$\begin{array}{lll} & (\texttt{rec}_1\ X\ x\ g) \circ (\texttt{cons}\ e) = (g\ e) \circ (\texttt{rec}_1\ X\ x\ g) & \overset{\texttt{FunExt}}{\Longleftrightarrow} \\ \forall (l : \texttt{List}). & (\texttt{rec}_1\ X\ x\ g)(\texttt{cons}\ e\ l) = g\ e\ (\texttt{rec}_1\ X\ x\ g\ l) & \overset{\beta}{\Longleftrightarrow} \\ \forall (l : \texttt{List}). & g\ e\ (\texttt{rec}_1\ X\ x\ g\ l) = g\ e\ (\texttt{rec}_1\ X\ x\ g\ l) & \end{array}$$

Thus we conclude that $(\texttt{MorphList List nil cons}\quad X\ x\ g\quad (\texttt{rec}_1\ X\ x\ g))$ holds □

Before we can prove the η-rule for the recursor, we need the following helper lemma.

Lemma 4.3.8 (RecId)**.** $\mathtt{rec_1\ List\ nil\ cons} = \mathtt{id_{List}}$

$$\begin{array}{ccc} \mathbf{1} + (E \times \mathtt{List}) & \xrightarrow{[\mathtt{nil},\mathtt{cons}]} & \mathtt{List} \\ \downarrow & = & \mathtt{id_{List}} \downarrow = \searrow \mathtt{rec_1\ List\ nil\ cons} \\ \mathbf{1} + (E \times \mathtt{List}) & \xrightarrow[[\mathtt{nil},\mathtt{cons}]]{} & \mathtt{List} \end{array}$$

Proof. We first reduce the statement using function extensionality and β-reduction. In the $(\hookrightarrow)$ step, we use the fact that $\mathtt{List}^* \hookrightarrow \mathtt{List}$ is an embedding.

$$\begin{array}{lll} & \mathtt{rec_1\ List\ nil\ cons} = \mathtt{id_{List}} & \overset{\text{FunExt}}{\iff} \\ \forall\ (l : \mathtt{List}). & \mathtt{rec_1\ List\ nil\ cons}\ l = l & \overset{\hookrightarrow}{\iff} \\ \forall\ (l : \mathtt{List}). & \mathtt{pr1}\ (\mathtt{rec_1\ List\ nil\ cons}\ l) = \mathtt{pr1}\ l & \overset{\text{FunExt}}{\iff} \\ \forall\ {\scriptstyle (l:\mathtt{List}),(X:\mathcal{U}),\atop (x:X),(g:E\to X\to X)}. & \big(\mathtt{pr1}\ (\mathtt{rec_1\ List\ nil\ cons}\ l)\big)\ X\ x\ g = (\mathtt{pr1}\ l)\ X\ x\ g & \end{array} \tag{4.2}$$

By 4.3.7 we known that $((\mathtt{rec_1}\ X\ x\ g) : \mathtt{List} \to X)$ is an $\mathcal{L}$-morphism. By looking at $(p := \mathtt{pr2}\ l)$, which has type $\mathtt{LimList}(\mathtt{pr1}\ l)$, we obtain the following equation:

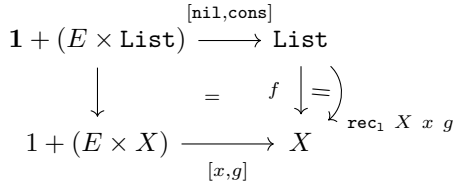
$$(\mathtt{rec_1}\ X\ x\ g)(\mathtt{rec_1^*\ List\ nil\ cons}(\mathtt{pr1}\ l)) = (\mathtt{rec_1^*}\ X\ x\ g\ (\mathtt{pr1}\ l)) \tag{4.3}$$

We conclude the proof by showing Equation (4.2). Let $(l : \mathtt{List}), (X : \mathcal{U}), (x : X)$, and $(g : E \to X \to X)$.

$$\begin{aligned} \big(\mathtt{pr1}\ (\mathtt{rec_1\ List\ nil\ cons}\ l)\big)\ X\ x\ g &\overset{\beta}{=} \mathtt{rec_1^*}\ X\ x\ g\ \big(\mathtt{pr1}\ (\mathtt{rec_1\ List\ nil\ cons}\ l)\big) \\ &\overset{\beta}{=} \mathtt{rec_1}\ X\ x\ g\ (\mathtt{rec_1\ List\ nil\ cons}\ l) \\ &= (\mathtt{rec_1}\ X\ x\ g)\ (\mathtt{rec_1\ List\ nil\ cons}\ l) \\ &:= (\mathtt{rec_1}\ X\ x\ g)\ (\mathtt{rec_1^*\ List\ nil\ cons}\ (\mathtt{pr1}\ l)) \\ &\overset{4.3}{=} (\mathtt{rec_1^*}\ X\ x\ g(\mathtt{pr1}\ l)) \\ &:= (\mathtt{pr1}\ l)\ X\ x\ g \end{aligned}$$ □

Theorem 4.3.9. The encoding List satisfies the η-rule which states that for all $(X : \mathcal{U})$, $(x : X)$, $(g : E \to X \to X)$ and $(f : \mathtt{List} \to X)$ we have:

$$(\mathtt{MorphList\ \ List\ nil\ cons}\quad X\ x\ g\quad f) \implies f = \mathtt{rec_1}\ X\ x\ g \quad \text{(uniqueness)}$$



Proof. Let $(X : \mathcal{U}), (x : X), (g : E \to X \to X), (f : \texttt{List} \to X)$ and $(l : \texttt{List})$. Assume we have a proof term of $(\texttt{MorphList}\ \ \texttt{List}\ \texttt{nil}\ \texttt{cons}\quad X\ x\ g\quad f)$. By looking at $(p := \texttt{pr2}\ l)$, which has type $\texttt{LimList}(\texttt{pr1}\ l)$, we obtain the following equation:

$$f\ (\texttt{rec}_\texttt{l}^*\ \texttt{List}\ \texttt{cons}\ \texttt{nil}\ (\texttt{pr1}\ l)) = \texttt{rec}_\texttt{l}^*\ X\ g\ (\texttt{pr1}\ l) \tag{4.4}$$

We derive the following:

$$\begin{aligned} f\ l &\overset{\texttt{RecId}}{=} f\ (\texttt{rec}_\texttt{l}\ \texttt{List}\ \texttt{nil}\ \texttt{cons}\ l) \\ &:= f\ (\texttt{rec}_\texttt{l}^*\ \texttt{List}\ \texttt{nil}\ \texttt{cons}\ (\texttt{pr1}\ l)) \\ &\overset{p}{=} \texttt{rec}_\texttt{l}^*\ X\ x\ g\ (\texttt{pr1}\ l) \\ &=: \texttt{rec}_\texttt{l}\ X\ x\ g\ l \end{aligned}$$

By functional extensionality, we conclude that $f = \texttt{rec}_\texttt{l}\ X\ x\ g$. □

To summerize this section: we employed the technique of Awodey, Frey and Speight to encode the impredicative list type of the previous section togeter with a proof that the recursor is unique. This created an embedding $\texttt{List}^* \hookrightarrow \texttt{List}$. We showed that this definition was well defined by updating the `nil` and `cons` constructors. We showed that the β-rules and the η-rule are satisfied by this new definition.

4.4 List induction principle

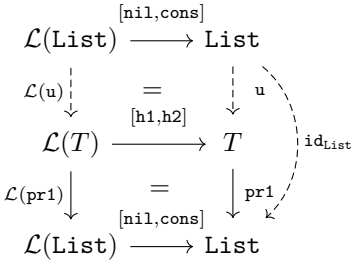
We are now ready to show that we have an induction principle for the new `List` type. Just like induction for the natural numbers, the list induction principle is used to prove facts about *all* lists. If some predicate $(P : \texttt{List} \to \mathcal{U})$ holds for `nil` and it is preserved under `cons`, then we expect the predicate to hold for all lists.

Definition 4.4.1. We define the **list induction principle**. Let $(P : \texttt{List} \to \mathcal{U})$ be a predicate on lists. If P holds for `nil` and is preserved under `cons`, then P holds for all lists. Formally, we have:

$$\texttt{IndList}\ P := (P\ \texttt{nil}) \to \Big(\prod_{l:\texttt{List}} \prod_{e:E} (P\ l) \to (P\ (\texttt{cons}\ e\ l))\Big) \to \prod_{l:\texttt{List}} P\ l$$

To prove the list induction principle, we make use of a helper type $T := (\sum(l : \texttt{List}).P\ l)$. This type is a pair of a list l and a proof that P holds for l. We show the induction principle in the following steps.

1. We show that $T := (\sum(l : \texttt{List}).P\ l)$ forms an $\mathcal{L}$-algebra.
2. We have that $\texttt{pr1}$ is an $\mathcal{L}$-morpism ($\texttt{pr1} : T \to \texttt{List}$).
3. By initiality of $\texttt{List}$ we have an $\mathcal{L}$-morpism ($\texttt{u} : \texttt{List} \to T$).
4. We have that ($\texttt{pr1} \circ \texttt{u}$) is a $\mathcal{L}$-morphism from $\texttt{List}$ to $\texttt{List}$.
5. We have that $\texttt{id}_{\texttt{List}}$ is also an $\mathcal{L}$-morphism from $\texttt{List}$ to $\texttt{List}$.
6. By the of $\texttt{List}$ we conclude that $\texttt{pr1} \circ u = \texttt{id}_{\texttt{List}}$.
7. For all l we have that $\texttt{pr1}\ (u\ l) = l$ and thus conclude that $(\texttt{pr2}\ (u\ l) : P\ l)$.



Lemma 4.4.2. Let us assume a predicate $(P : \texttt{List} \to \mathcal{U})$. Furthermore, we assume the following:

$$\texttt{p0} : P\ \texttt{nil}$$
$$\texttt{pn} : \prod_{l:\texttt{List}} \prod_{e:E} (P\ l) \to (P\ (\texttt{cons}\ e\ l))$$

Let $(T := \sum(l : \texttt{List}).P\ l)$. We have that $\langle T, \langle \texttt{h1}, \texttt{h2} \rangle \rangle$ forms an $\mathcal{L}$-algebra, where $\texttt{h1}$ and $\texttt{h2}$ are defined as follows:

$$\texttt{h1} := \langle \texttt{nil}, \texttt{p0} \rangle$$
$$\texttt{h2} := \lambda(e : E)(t : T).\langle \texttt{cons}\ e\ (\texttt{pr1}\ t), \texttt{pn}\ (\texttt{pr1}\ t)\ e\ (\texttt{pr2}\ t) \rangle$$

Proof. We have to show that the following two equations hold:

$$\texttt{pr1}\ \texttt{h1} = \texttt{nil} \tag{4.5}$$
$$\texttt{pr1} \circ (\texttt{h2}\ e) = (\texttt{cons}\ e) \circ \texttt{pr1} \tag{4.6}$$

Both equations hold by simple β-reduction. For Equation (4.5) we have:

$$\texttt{pr1}\ \texttt{h1} := \texttt{pr1}\ \langle \texttt{nil}, \texttt{p0} \rangle \overset{\beta}{=} \texttt{nil}$$

For Equation (4.6) we have:

$$\begin{aligned} \texttt{pr1} \circ (\texttt{h2}\ e) &:= \texttt{pr1}\ \langle \texttt{cons}\ e\ (\texttt{pr1}\ t), \texttt{pn}\ (\texttt{pr1}\ t)\ e\ (\texttt{pr2}\ t) \rangle \\ &\overset{\beta}{=} \texttt{cons}\ e\ (\texttt{pr1}\ t) \end{aligned}$$

□

Lemma 4.4.3. We have that ($\texttt{pr1} : T \to \texttt{List}$) is an $\mathcal{L}$-morphism. Formally, we have that the following holds:

$$\big(\texttt{MorphList}\ T\ \texttt{h1}\ \texttt{h2}\quad \texttt{List}\ \texttt{nil}\ \texttt{cons}\quad \texttt{pr1}\big) \tag{4.7}$$

Proof. We have to show that the following two equations hold:

$$\texttt{pr1}\ \texttt{h1} = \texttt{nil} \tag{4.8}$$

$$\texttt{pr1}\ \circ (\texttt{h2}\ e) = (\texttt{cons}\ e) \circ f \tag{4.9}$$

Both hold by simple β-reduction, where you need to instantiate Equation (4.9) with $(t : T)$. □

We can now combine the previous two lemmas to show that the induction principle holds for `List`.

Theorem 4.4.4 (`IndList`)**.** The list induction principle `IndList` described in 4.4.1 holds for the `List` type.

Proof. Let us assume a predicate $(P : \texttt{List} \to \mathcal{U})$. Furthermore, we assume the following:

$$\texttt{p0} : P\ \texttt{nil}$$

$$\texttt{pn} : \prod_{l:\texttt{List}} \prod_{e:E} (P\ l) \to (P\ (\texttt{cons}\ e\ l))$$

We follow the steps described before.

1. By 4.4.2 we have that $T := (\sum(l : \texttt{List}).P\ l)$ forms an $\mathcal{L}$-algebra.
2. By 4.4.3 we have that ($\texttt{pr1} : T \to \texttt{List}$) is an $\mathcal{L}$-morphism.
3. By initiality of `List`, we have the unique morphism $(u : \texttt{List} \to T)$ given by $u := (\texttt{rec}_\texttt{l}\ T\ \texttt{h1}\ \texttt{h2})$.
4. By a straightforward adaptation of 2.2.4 we have that ($\texttt{pr1}\ \circ \texttt{u}$) is an $\mathcal{L}$-morphism.
5. By a straightforward adaptation of 02.2.3 we have that $\texttt{id}_\texttt{List}$ is an $\mathcal{L}$-morphism.
6. By 4.3.9 we have that $\texttt{pr1} \circ \texttt{u} = (\texttt{rec}_\texttt{l}\ \texttt{List}\ \texttt{nil}\ \texttt{cons})$. By 4.3.8 we have that $(\texttt{rec}_\texttt{l}\ \texttt{List}\ \texttt{nil}\ \texttt{cons}) = \texttt{id}_\texttt{List}$. We combine these to obtain that $\texttt{pr1} \circ \texttt{u} = \texttt{id}_\texttt{List}$.
7. Let $(l : \texttt{List})$ be any list. We want to show that $(P\ l)$ holds. We just saw that $\texttt{pr1}\ (\texttt{u}\ l) = l$. We can thus obtain a proof of $(P\ l)$ by looking at the second projection $\big(\texttt{pr2}\ (\texttt{u}\ l) : P\ l\big)$.

□

5 Quotients

In this chapter, we discuss the concept of quotient types. Quotient types are a way to define types where some values of a type $(D : \mathcal{U})$ are identified by a relation $(R : D \to D \to \mathcal{U})$. You can compare this to the concept of quotients in set theory. It is possible to define quotient types in impredivative type theory. Similarly to lists, this definition does satisfy the expected β-rule, but fails to satisfy the η-rule. We show that it is possible to extend the technique of [AFS18] to define quotients with an η-rule. We follow the same basic approach as for lists. We use this improved quotient type to show that the class-function, that lifts an element of the base type to an element of the quotient type, is indeed a surjective function. In chapter 6 we use the quotients as defined here to create stream types.

5.1 Impredicative quotient type

Quotient types can best be understood through an example.

Example 5.1.1. Let us create a quotient type that identifies all even numbers. In set theory, we would write this as $\mathbb{N}/\texttt{even} = \{[0], [1]\}$. We define the relation `even` as follows:

$$\begin{aligned} \texttt{even} &: \mathbb{N} \to \mathbb{N} \to \mathcal{U} \\ \texttt{even}\ n\ m &:= \texttt{isEven}\ n \iff \texttt{isEven}\ m \end{aligned}$$

The quotient type $\texttt{quot}^*\ N\ \texttt{even}$ should be similar to the set-theoretic quotient $\mathbb{N}/\texttt{even}$. We want a function $(\texttt{cls}^* : \mathbb{N} \to \texttt{quot}\ \mathbb{N}\ \texttt{even})$ that maps a natural number to its 'equivalence class' such that:

$$\forall (n, m : \mathbb{N}) \qquad \texttt{even}\ n\ m \to \texttt{cls}^*\ n = \texttt{cls}^*\ m$$

Let us now give a more formal definition of quotient types. But first, we define a predicate that states that a function behaves the same on all equivalence classes of a relation R.

Definition 5.1.2. A function f behaves the same on all equivalence classes of R if the following predicate holds:

$$\texttt{EqCls}\ (f : D \to E)\ (R : D \to D \to \mathcal{U}) := \prod (x, y : D).R\ x\ y \implies f\ x = f\ y$$

Definition 5.1.3. We define **quotient types**. Let $(D : \mathcal{U})$ and $(R : D \to D \to \mathcal{U})$.

$$\texttt{quot}^*\ D\ R := \prod (C : \mathcal{U}).\prod (f : D \to C).\texttt{EqCls}\ f\ R \to C \qquad \text{(formation)}$$

We create the class function as follows:

$$\begin{aligned} \texttt{cls}^* &: D \to \texttt{quot}^*\ D\ R \qquad \text{(introduction)} \\ \texttt{cls}^*\ (d : D) &:= \lambda C.\lambda f.\lambda H.f\ d \end{aligned}$$

We are able to lift any function $(f : D \to E)$ that behaves the same on all equivalence classes to a function $(\overline{f} : \mathtt{quot}^*\ D\ R \to E)$. This lifting is done by the recursor for quotient types.

Definition 5.1.4. We define the **recursor for quotient types** as the function that lifts functions that behave the same on all equivalence classes.

$$\begin{aligned} \mathtt{rec}^*_{\mathtt{q}} &: \prod(C : \mathcal{U}).\prod(f : D \to C). && \text{(elimination)} \\ &\quad \prod(H : \mathtt{EqCls}\ f\ R).\prod(q : \mathtt{quot}^*\ D\ R).\ C \\ \mathtt{rec}^*_{\mathtt{q}}\ C\ f\ H\ q &:= q\ C\ f\ H \end{aligned}$$

Notation 5.1.5. We often write $\overline{f} := (\lambda q.\ \mathtt{rec}^*_{\mathtt{q}}\ C\ f\ H\ q)$ if C and H are clear from the context.

We expect that the lifted function satisfies $\overline{f} \circ \mathtt{cls}^* = f$. This gives us the β-rule for quotient types as shown in Figure 4.

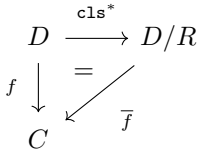


Figure 4: Commutative diagram of the β-rule for quotient types.

Lemma 5.1.6. The β-rule for quotient types is satisfied. For $(D : \mathcal{U}), (R : D \to D \to \mathcal{U}), (f : D \to C)$ and $(H : (\mathtt{EqCls}\ f\ R))$ we have that $\overline{f} \circ \mathtt{cls}^* = f$. In other words:

$$(\mathtt{rec}^*_{\mathtt{q}}\ C\ f\ H) \circ \mathtt{cls}^* = f \qquad \text{(computation)}$$

Proof. Let D, R, f and H be as described above. Let $(x : D)$.

$$\begin{aligned} \overline{f}\ (\mathtt{cls}^*\ x) &:= \mathtt{rec}^*_{\mathtt{q}}\ C\ f\ H\ (\mathtt{cls}^*\ x) \\ &\overset{\beta}{=} (\mathtt{cls}^*\ x)\ C\ f\ H \\ &\overset{\beta}{=} f\ x \end{aligned}$$

□

Lemma 5.1.7 (EqCls1)**.** Let $(D : \mathcal{U})$ and $(R : D \to D \to \mathcal{U})$. For $(x, y : D)$ we have:

$$R\ x\ y \implies \mathtt{cls}^*\ x = \mathtt{cls}^*\ y$$

Proof. Let D and R be as described above. Let $(x, y : D)$ and assume $(R\ x\ y)$. We initialize $(\mathtt{cls}^*\ x)$ using $(C : \mathcal{U})$, $(f : D \to C)$ and $(H : (\mathtt{EqCls}\ f\ R))$. The statement follows by function extensionality.

$$
\begin{aligned}
(\texttt{cls}^*\ x)\ C\ f\ H &\overset{\beta}{=} f\ x \overset{H}{=} f\ y \\
&\overset{\beta}{=} (\texttt{cls}^*\ y)\ C\ f\ H \\
&\overset{\beta}{=} f\ y
\end{aligned}
$$

□

Note 5.1.8. Before we continue, here is a quick note about the relation R. From set theory, we expect that R needs to be an equivalence relation. We expect it to be *reflexive*, *symmetric*, and *transitive*. However, none of the definitions given in this section enforce this expectation. Nevertheless, by definition, any function f that acts on elements of $\texttt{quot}^* D\ R$ needs to satisfy $\texttt{EqCls}\ f\ R$. If R is not reflexive, so that we don't have $R\ x\ x$ for some $(x : D)$, we obviously still have that $f\ x = f\ x$. The same goes for symmetry: if $R\ x\ y$ holds, but, $R\ y\ x$ does not hold for some $(x, y : D)$, we still have that $R\ x\ y \implies f\ x = f\ y$ by $\texttt{EqCls}\ f\ R$. Therefore, we also have $f\ y = f\ x$, by the reflexivity of $=$. To complete this reasoning, if we have $R\ x\ y$ and $R\ y\ z$, but don't have $R\ x\ z$ for some $(x, y, z : D)$, we still have that $f\ x = f\ y$ and $f\ y = f\ z$. By transitivity of $=$ we thus have that $f\ x = f\ z$. The behavior of $\texttt{quot}^*$ can only be observed as if R were an equivalence relation. This definition of quotients is thus built on the universal property in a categorical setting, rather than the set-theoretic equivalence class definition. Taking the reflexive, symmetric, and transitive closure of R would thus not matter (although we shall not explore this avenue further).

We would hope to have that the function $\overline{f}$ is indeed *the* lifted function and that any other function h that satisfies the β-rule is equal to this f. This uniqueness requirement or η-rule is unfortunately not satisfied. In the next section, we take a look at a categorical definition of quotients. Using this definition and its uniqueness requirement, we can apply the same encoding technique as before to create a quotient type that satisfies this uniqueness property.

5.2 Categorical quotients

We take a quick look at the categorical requirements for a quotient type that satisfies the η-rule. Just like we have seen for lists, we want quotients to satisfy the following uniqueness rule:

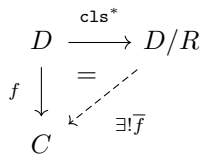


Figure 5: Uniqueness rule for quotients.

Just as seen before, we can give an alternative definition. Suppose we have morphisms $(g : D \to X)$ and $(g' : D \to Y)$ that both satisfy the additional requirement that $(\texttt{EqCls}\ g\ R)$ and $(\texttt{EqCls}\ g'\ R)$. By the β-rule, we know that

$\overline{g} \circ \texttt{cls}^* = g$ and $\overline{g'} \circ \texttt{cls}^* = g'$. If we have a morphism $(f : X \to Y)$, such that $g' = f \circ g$, we want to have that $\overline{g'} = f \circ \overline{g}$ (the bottom right triangle in Figure 6). This is the η-rule for quotient types.

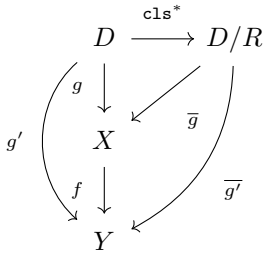


Figure 6: Commutative diagram of the η-rule for quotient types.

5.3 Impredicative encoding of quotients

We are now ready to define an impredicative encoding that satisfies the η-rule. Note that we follow the same approach as we did for encoding `List` in the previous chapter. We encode the uniqueness rule of Figure 6. As an additional requirement, we need to have that the morphisms satisfy the `EqCls` predicate.

Definition 5.3.1. We define an **inductive quotient type** `quot`.

$$\texttt{quot}\ (D : \mathcal{U})\ (R : D \to D \to \mathcal{U}) := \sum (q : \texttt{quot}^*\ D\ R).\texttt{LimQuot}\ q \qquad \text{(formation)}$$

$$\begin{aligned}\texttt{LimQuot}\ (q : \texttt{quot}^*\ D\ R) := \prod (X, Y : \mathcal{U}).\ \prod \tbinom{g : D \to X}{g' : D \to Y}.\ \prod (f : X \to Y).\\ (H : (\texttt{EqCls}\ g\ R)) \wedge (H' : (\texttt{EqCls}\ g'\ R)) \wedge f \circ g = g'\\ \Longrightarrow\\ f\ (\texttt{rec}^*_{\texttt{q}}\ E\ g\ H\ q) = \texttt{rec}^*_{\texttt{q}}\ E\ g'\ H'\ q\end{aligned}$$

We define the quotient recursor as follows:

$$\texttt{rec}_{\texttt{q}}\ C\ f\ H\ q := \texttt{rec}^*_{\texttt{q}}\ C\ f\ H\ (\texttt{pr1}\ q)$$

Note the similarity to the definition of `List`. To define the new `cls` function, we need to show that $(\texttt{LimQuot}\ (\texttt{cls}\ d))$ holds for all $(d : D)$.

Lemma 5.3.2 (`LimQuotCls`)**.** For all $(D : \mathcal{U})$, we have that $(\texttt{LimQuot}\ (\texttt{cls}\ d))$ holds.

Proof. Let $(d : D)$. Let $(X, Y : \mathcal{U})$, $(g : D \to X)$, $(g' : D \to Y)$ and $(f : X \to Y)$. Assume $(H : (\texttt{EqCls}\ g\ R))$, $(H' : (\texttt{EqCls}\ g'\ R))$ and $(*) := f \circ g = g'$. We need to show that $f\ (\texttt{rec}^*_{\texttt{q}}\ E\ g\ H\ (\texttt{cls}^*\ d)) = \texttt{rec}^*_{\texttt{q}}\ E\ g'\ H'\ (\texttt{cls}^*\ d)$.

$$\begin{aligned} f\ (\mathtt{rec}_\mathtt{q}^*\ E\ g\ H\ (\mathtt{cls}^*\ d)) &\overset{\beta}{=} f\ ((\mathtt{cls}^*\ d)\ E\ g\ H) \\ &\overset{\beta}{=} f\ (g\ d) \\ &\overset{(*)}{=} g'\ d \\ &\overset{\beta}{=} (\mathtt{cls}^*\ d)\ E\ g'\ H' \\ &\overset{\beta}{=} \mathtt{rec}_\mathtt{q}^*\ E\ g'\ H'\ (\mathtt{cls}^*\ d) \end{aligned}$$ □

Definition 5.3.3. Let $(D : \mathcal{U})$ and $(R : D \to D \to \mathcal{U})$. We define the **class function** `cls` for `quot` $D\ R$ as follows:

$$\mathtt{cls}\ (d : D) := \langle \mathtt{cls}^*\ d,\ \mathtt{LimQuotCls}\ d \rangle \qquad \text{(introduction)}$$

We also still have that `cls` $x =$ `cls` y for all $(x, y : D)$ such that $R\ x\ y$.

Lemma 5.3.4 (`EqCls2`)**.** Let $(D : \mathcal{U})$ and $(R : D \to D \to \mathcal{U})$. We have that (`EqCls cls` R) holds. In other words, for $(x, y : D)$ we have:

$$R\ x\ y \implies \mathtt{cls}\ x = \mathtt{cls}\ y$$

Proof. Assume $(R\ x\ y)$. By 5.1.7 we have that $\mathtt{cls}^*\ x = \mathtt{cls}^*\ y$. We also have that $\mathtt{LimQuot}(\mathtt{cls}^*\ x) = \mathtt{LimQuot}(\mathtt{cls}^*\ y)$ by `UIP`.
We conclude that $\mathtt{cls}\ x = \langle \mathtt{cls}^*\ x, \mathtt{LimQuot}(\mathtt{cls}^*\ x) \rangle = \langle \mathtt{cls}^*\ y, \mathtt{LimQuot}(\mathtt{cls}^*\ y) \rangle =$ `cls` y. □

We are now ready to show that the η-rule is satisfied by the new quotient type. We first need a few lemmas. We show that the β-rule remains satisfied for `quot`. We then show that the identity case of the η-rule holds: $\overline{\mathtt{cls}} = \mathtt{id}_{\mathtt{quot}\ D\ R}$. Finally, we show the complete η-rule.

Lemma 5.3.5. The β-rule for `quot` is satisfied. Let $(D : \mathcal{U})$ and $(R : D \to D \to \mathcal{U})$ form a quotient `quot` $D\ R$ and let $(C : \mathcal{U})$, $(g : D \to C)$ and $(H : \mathtt{EqCls}(g, R))$ we have that $\overline{g} \circ \mathtt{cls} = g$. In other words, we have:

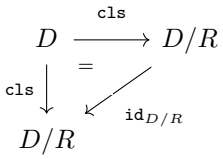
$$(\mathtt{rec}_\mathtt{q}\ C\ g\ H) \circ \mathtt{cls} = g \qquad \text{(computation)}$$

Proof. Let D, R, g and H be as described above. Let $(d : D)$.

$$\begin{aligned} (\overline{g} \circ \mathtt{cls})\ d &:= \mathtt{rec}_\mathtt{q}\ C\ g\ H\ (\mathtt{cls}\ d) \\ &:= \mathtt{rec}_\mathtt{q}^*\ C\ g\ H\ (\mathtt{pr1}\ (\mathtt{cls}\ d)) \\ &\overset{\beta}{=} \mathtt{rec}_\mathtt{q}^*\ C\ g\ H\ (\mathtt{pr1}\ \langle \mathtt{cls}^*\ d, \mathtt{LimQuot}(\mathtt{cls}^*\ d) \rangle) \\ &\overset{\beta}{=} \mathtt{rec}_\mathtt{q}^*\ C\ g\ H\ (\mathtt{cls}^*\ d) \\ &\overset{\beta}{=} g\ d \end{aligned}$$ □

Lemma 5.3.6 (IdLift)**.** Let $(D : \mathcal{U})$ with $(R : D \to D \to \mathcal{U})$ be a quotient $(D/R := \mathtt{quot}\ D\ R)$. We have $\overline{\mathtt{cls}} = \mathtt{id}_{D/R}$.

$$\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2} = \mathtt{id}_{D/R}$$



Proof. We first reduce the statement using function extensionality and β-reduction. In the $(\hookrightarrow)$ step, we use the fact that $\mathtt{quot}^*\ D\ R \hookrightarrow \mathtt{quot}\ D\ R$ is an embedding.

$$\begin{aligned}
& \mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2} = \mathtt{id}_{D/R} && \overset{\text{FunExt}}{\iff} \\
\forall\ (q : D/R).\quad & \mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q = q && \overset{\hookrightarrow}{\iff} \\
\forall\ (q : D/R).\quad & \mathtt{pr1}\ (\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q) = \mathtt{pr1}\ q && \overset{\text{FunExt}}{\iff} \\
\forall_{\substack{(q:D/R),(C:\mathcal{U}),\\ (g:D\to C),(H:\mathtt{EqCls}\ g\ R)}}.\quad & \big(\mathtt{pr1}\ (\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q)\big)\ C\ f\ H = (\mathtt{pr1}\ q)\ C\ f\ H && (5.1)
\end{aligned}$$

We want to apply the $(\mathtt{LimQuot}\ (\mathtt{pr1}\ q))$ proposition. The function $((\mathtt{rec_1}\ X\ x\ g) : \mathtt{List} \to X)$ will play the role of f.

1. We have $(\mathtt{EqCls}\ g\ R)$ by H.
2. We have $(\mathtt{EqCls}\ \mathtt{cls}\ R)$ by 5.3.4.
3. We have $(\mathtt{rec_q}\ C\ g\ H) \circ \mathtt{cls} = g$ by the β-rule of 5.3.5.

We can thus use the $(\mathtt{LimQuot}\ (\mathtt{pr1}\ q))$ proposition to show that the following equation holds:

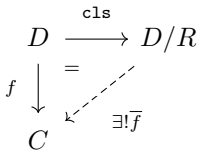
$$(\mathtt{rec_q}\ C\ g\ H)\ (\mathtt{rec_q^*}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ (\mathtt{pr1}\ q)) = (\mathtt{rec_q^*}\ C\ g\ H\ (\mathtt{pr1}\ q)) \quad (5.2)$$

We conclude the proof by showing Equation (5.1). Let $(q : D/R), (C : \mathcal{U}), (f : D \to C)$, and $(H : \mathtt{EqCls}\ f\ R)$.

$$\begin{aligned}
\big(\mathtt{pr1}\ (\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q)\big)\ C\ g\ H &\overset{\beta}{=} \mathtt{rec_q^*}\ C\ g\ H\ \big(\mathtt{pr1}\ (\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q)\big) \\
&\overset{\beta}{=} \mathtt{rec_q}\ C\ g\ H\ (\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q) \\
&= (\mathtt{rec_q}\ C\ g\ H)\ (\mathtt{rec_q}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ q) \\
&:= (\mathtt{rec_q}\ C\ g\ H)\ (\mathtt{rec_q^*}\ D/R\ \mathtt{cls}\ \mathtt{EqCls2}\ (\mathtt{pr1}\ q)) \\
&\overset{5.2}{=} (\mathtt{rec_q^*}\ C\ g\ H\ (\mathtt{pr1}\ q)) \\
&:= (\mathtt{pr1}\ q)\ C\ g\ H \qquad \square
\end{aligned}$$

Using the previous lemma, we show that the general η-rule is satisfied.

Theorem 5.3.7. The η-rule holds for `quot`. Let $(D : \mathcal{U})$ with $(R : D \to D \to \mathcal{U})$ be a quotient $D/R := \texttt{quot}\ D\ R$. Suppose $(g : D \to C)$ with $(H : (\texttt{EqCls}\ g\ R))$ and $(C : \mathcal{U})$. Furthermore, suppose we have a function $(f : D/R \to C)$ with $f \circ \texttt{cls} = g$. Then we have that $f = \overline{g}$. In formulas:



$$\prod(H : \texttt{EqCls}(g, R)).(f \circ \texttt{cls} = g) \implies f = \texttt{rec}_\texttt{q}\ C\ g\ H$$
(uniqueness)

Proof. Assume we have D, R, C, g, H and f as described. We want to show that $f = \texttt{rec}_\texttt{q}\ C\ g\ H$.
Let $(\langle q, p\rangle : D/R)$ where $(q : \texttt{quot}^* D\ R)$ and $(p : \texttt{LimQuot}(q))$. We have that $f \circ \texttt{cls} = g$ by definition. Obviously we have $(H' : (\texttt{EqCls}\ f \circ \texttt{cls}\ R))$. We thus have that all requirements of p are satisfied. The statement follows by function extensionality.

$$\begin{aligned} f\ \langle q, p\rangle &\overset{\texttt{IdLift}}{=} f\ (\texttt{rec}_\texttt{q}\ D/R\ \texttt{cls}\ \texttt{EqCls2}\ \langle q, p\rangle) \\ &\overset{p}{=} \texttt{rec}_\texttt{q}\ C\ g\ H\ \langle q, p\rangle \end{aligned}$$ □

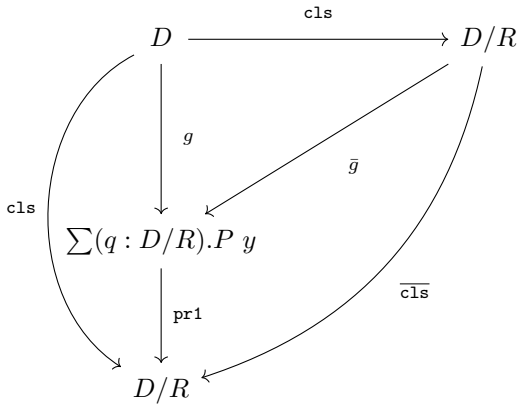
5.4 Quotient induction priciple

Whenever we can show that some property P holds for all $((\texttt{cls}\ d) : D)$ we of course want to obtain that P holds for all $(q : D/R)$. This fact does not hold for the $\texttt{quot}^*$ type. The improved `quot` type does satisfy this property since it follows from the η-rule. We show this in the following theorem.

Theorem 5.4.1. Let $(D : \mathcal{U})$ with $(R : D \to D \to \mathcal{U})$ be a quotient $D/R := \texttt{quot}\ D\ R$. Let $(P : D/R \to \mathcal{U})$ be some property. We then have the that:

$$\prod(x : D).\ P\ (\texttt{cls}\ x) \implies \prod(q : D/R).\ P\ q$$

Proof. Assume we have $(A : \prod(x : D).\ P\ (\texttt{cls}\ x))$. Define $(\sum(q : D/R).P\ y)$ and $g(x) := \langle \texttt{cls}\ x, A\ (\texttt{cls}\ x)\rangle$.
In this proof, we shall make the following diagram commute in all configurations:



We have that $R\ x\ y \implies g\ x = g\ y$ by 5.1.7 and because $A\ x$ is a proof of $P\ x$. We thus have that $\overline{g}$ is well-defined and we have that $\overline{g} \circ \texttt{cls} = g$. Furthermore, we notice that $(\texttt{cls} : D \to D/R)$ can be lifed to $(\overline{\texttt{cls}} : D/R \to D/R)$ such that $\overline{\texttt{cls}} \circ \texttt{cls} = \texttt{cls}$ which is well defined by 5.1.7. Finally, we notice that $\texttt{cls} = \texttt{pr1} \circ g$. By 5.3.6 we have that $\overline{\texttt{cls}} = \texttt{id}_{D/R}$. By 5.3.7 we have that $\texttt{pr1} \circ \overline{g} = \texttt{id}_{D/R}$. We fix a quotient $(q : D/R)$. We then have that $\texttt{pr1}(\overline{g}\ q) = q$. But then we find that $\texttt{pr2}\ (\overline{g}\ q)$ is of type $P\ q$. We conclude that we have a proof of type $P\ q$ for any $(q : D/R)$. □

In the next chapter, we take a look at the coinductive stream type. Like before, we encode the uniqueness property of streams within their definition. In our continuous quest to satisfy the η-rules, we shall make use of the improved quotient types to show that our stream type satisfies the η-rule of streams.

6 Streams

In this chapter, we develop an impredicative encoding of a stream type that satisfies the expected β-rules and the η-rule. These streams are an example of a coinductive type. Coinductive types are the dual notion of inductive types and describe (possibly) infinite data structures. These streams, or infinite lists, shall range over some element type $(E : \mathcal{U})$. As before, we assume that $(\Gamma \vdash E : \mathcal{U})$ holds globally. The definitions in this chapter can be straightforwardly generalized to any element type.
To create streams, we need the concept of existential types. They are the type theoretical analog of existential quantifiers in set theory. We use them to hide the implementation of streams. We continue by giving a common impredicative definition of streams and show that the β-rules are satisfied for this definition. Next, we take a look at the categorical definition of coinductive streams using a suitable final coalgebra. We combine these definitions into an impredicative encoding of a stream type that also satisfies the η-rule and prove the coinduction principle for streams.

6.1 Existential types

In System F, it is possible to create existential types [Geu14b]. They can be used to *hide* the implementation of a type and only give access to an interface. An existential type $\exists(X : \mathcal{U}).P(X)$ is constructed using some type $(X : \mathcal{U})$ and a type family $(P : \mathcal{U} \to \mathcal{U})$. Just like an existential quantifier, we can create an instance of the existential type by giving a sample $(X : \mathcal{U})$ and a witness $(p : P(X))$. Once X has been packed, you cannot retrieve it anymore. In this way, the implementation (the concrete value of X) is hidden from the outside world and only the interface (the type family P) is accessible. This means we can operate on P, but not retrieve information about X.

Definition 6.1.1. We define the **existential type**. Let $(P : \mathcal{U} \to \mathcal{U})$.

$$\exists(X : \mathcal{U}).P(X) := \prod(Y : \mathcal{U}).\Big(\prod(X : \mathcal{U}).P(X) \to Y\Big) \to Y \qquad \text{(formation)}$$

We denote these existential types using the shorthand notation $\exists X.P := \exists(X : \mathcal{U}).P(X)$. Inhabitants of these existential types are created using the `pack` function.

$$\texttt{pack} \ : \prod(X : \mathcal{U}).\prod(p : P(X)).\ \exists X.A \qquad \text{(introduction)}$$
$$\texttt{pack}\ X\ p := \lambda(Y : \mathcal{U}).\lambda\big(k : \prod(X : \mathcal{U}).P(X) \to Y\big).k\ X\ p$$

We can operate on the existential type using the following recursor. Note that $(\widetilde{X} : \mathcal{U})$, has no connection to the original value of $(X : \mathcal{U})$ that was used to `pack` the existential type. Therefore, the output type $(Y : \mathcal{U})$ cannot depend on the concrete implementation of X.

$$\texttt{rec}_\exists := \prod(Y : \mathcal{U}).\Big(\prod(\widetilde{X} : \mathcal{U}).P(\widetilde{X}) \to Y\Big).\exists X.P \to Y \qquad \text{(elimination)}$$
$$\texttt{rec}_\exists\ Y\ f\ e := e\ Y\ f$$

We can adapt this definition for bounded existential types like $\exists(a : A).P(a)$ by replacing $(X : \mathcal{U})$ with $(A : a)$ and taking the type $(P : A \to \mathcal{U})$.

This definition is best understood through an example. We sketch why we cannot retrieve the value of X once it has been packed.

Example 6.1.2. Define $\exists X.(X \times X)$. We can create a pair of natural numbers and encapsulate it in the existential type. We thus have $(\texttt{pack}\ \mathbb{N}\ \langle 3, 3\rangle : \exists X.(X \times X))$. When we try to get the value 3 out of the existential type, we quickly see that this is not possible. The recursor only allows us to operate on the value for *some* type X. We have forgotten that this type was $\mathbb{N}$. Below is a failed attempt to extract 3 from the existential type. This will not type-check because x is not of type $\mathbb{N}$.

$$\texttt{rec}_\exists\ \mathbb{N}\ (\lambda X.\lambda(\langle x, y\rangle : X \times X).\quad \boxed{x}\quad)\ (\texttt{pack}\ \mathbb{N}\ \langle 3, 3\rangle)$$

Example 6.1.3. We define the existential type $\exists X.X \times (X \to X)$ that encapsulates the behaviour of the natural numbers. This definition hides the concrete implementation from the interface. We have for example $N := \texttt{pack}\ \mathbb{N}^*\ \langle \texttt{0}^*, \texttt{S}^*\rangle$. When we operate on N, there is no way to see that we used $\mathbb{N}$. If we somehow define another implementation of the natural numbers, for example using binary lists, we can use `pack` to hide this implementation.

Before we continue, we first have to add an axiom about equality on existential types. When we *unpack* an existential type $(t : \exists X.P)$ using the recursor $\texttt{rec}_\exists$, we obtain values $(X : \mathcal{U})$ and $(p : P(X))$. These values are unrelated to the values we used to create the member t. This is by design. There is however a problem. If we use the `pack` function, to re-package the X and p values $t' := \texttt{pack}\ X\ p$, we need to have that $t = t'$. In other words, if we unpack an existential type $(t : \exists X.P)$, and then re-pack it, we would like it to be equal to t.

Axiom 6.1.4. Let $(P : \mathcal{U} \to U)$. If we apply the pack function to the recursor, this yields the identity.

$$\texttt{ExistsId} \quad : \quad \texttt{rec}_\exists\ (\exists X.P)\ \texttt{pack} = \texttt{id}_{\exists X.P}$$

We shall use this axiom in 6.4.12, where we also give some more details. We do not exclude that this axiom can be proven as a theorem by creating an improved variant of the existential type. This gives a good opportunity for further research.

6.2 Impredicative stream type

Like before, we first give an impredicative definition of a stream type. We will show that this definition satisfies the computation rules (β-rules).

Definition 6.2.1. The **impredicative stream type**.

$$\texttt{Stream}^* := \exists(X : \mathcal{U}).X \times (X \to E) \times (X \to X) \qquad \text{(formation)}$$

We first give the elimination rules for the stream type. The $\texttt{hd}^*$ function gives the head of the stream. The $\texttt{tl}^*$ function gives the tail of the stream. Both make use of the existential recursor $\texttt{rec}_\exists$ to look inside the existential type. The tail function furthermore uses the `pack` function to create a new inhabitant of the existential type.

$$\texttt{hd}^*(s : \texttt{Stream}^*) := \texttt{rec}_\exists\ E\ (\lambda X.\lambda\langle x, h, t\rangle.\ h\ x)\ s \qquad : E \quad \text{(elimination)}$$

$$\texttt{tl}^*(s : \texttt{Stream}^*) := \texttt{rec}_\exists\ E\ (\lambda X.\lambda\langle x, h, t\rangle.\ \texttt{pack}\ X\ \langle t\ x, h, t\rangle)\ s \qquad : \texttt{Stream}^* \quad \text{(elimination)}$$

A stream is build using some internal type $(X : \mathcal{U})$. It consists of triples $\langle x, h, t\rangle$ where x is some element to boot-strap the stream, h is a function that computes the head of the stream and t is a function that computes the tail of the stream. For example, we could have $\langle 0, \lambda n.n, \lambda n.n + 1\rangle$, where we use $\mathbb{N}$ both internally as our type X and externally as our type E. To compute the head of this stream, we use the second element of the triple $(\lambda n.n)$ and apply it to the first element 0 to obtain $(\lambda n.n)\ 0 \stackrel{\beta}{=} 0$. To obtain the tail of this stream, we use the third element $(\lambda n.n + 1)$ to update the first element $((\lambda n.n + 1)\ 0 \stackrel{\beta}{=} 1)$. To construct a stream, we define the corecursor. This is the dual notion of a recursor and is used not to destruct a type, but to construct one. We use the `pack` function to create an inhabitant of the existential type.

Definition 6.2.2. The **impredicative stream corecursor**.

$$\texttt{corec}^*_{\texttt{s}} : \prod(X : \mathcal{U}).(X \to E) \to (X \to X) \to X \to \texttt{Stream}^* \quad \text{(introduction)}$$

$$\texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ x := \texttt{pack}\ X\ \langle x, h, t\rangle$$

We can check that the introduction rule (the corecursor) and elemination rules (the head and tail functions) compute as one would expect.

Proposition 6.2.3. The stream corecursor satisfies the following β-rules. Let $(X : \mathcal{U})$, $(h : X \to E)$, $(t : X \to X)$ and $(x : X)$.

$$\texttt{hd}^*(\texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ x) \stackrel{\beta}{=} h\ x \qquad \text{(computation)}$$

$$\texttt{tl}^*(\texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ x) \stackrel{\beta}{=} \texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ (t\ x) \qquad \text{(computation)}$$

Proof. We unfold the definitions of $\texttt{rec}_\exists$, `pack`, $\texttt{hd}^*$ and $\texttt{tl}^*$. To distinguish between between the outer variables X, x, h and t, we rename the bounded inner variables to A, a, b and c.

$$
\begin{aligned}
\texttt{hd}^*(\texttt{corec}_\texttt{s}^*\ X\ h\ t\ x) &:= \texttt{rec}_\exists\ E\ (\lambda A.\lambda\langle a,b,c\rangle.\ b\ a)\ (\texttt{corec}_\texttt{s}^*\ X\ h\ t\ x)\\
&:= (\texttt{corec}_\texttt{s}^*\ X\ h\ t\ x)\ E\ (\lambda X.\lambda\langle a,b,c\rangle.\ (b\ a))\\
&:= (\texttt{pack}\ X\ \langle x,h,t\rangle)\ E\ (\lambda X.\lambda\langle a,b,c\rangle.\ (b\ a))\\
&:= (\lambda Y.\lambda k.k\ X\ \langle x,h,t\rangle)\ E\ (\lambda X.\lambda\langle a,b,c\rangle.\ (b\ a))\\
&\overset{\beta}{=} (\lambda X.\lambda\langle a,b,c\rangle.\ (b\ a))\ X\ \langle x,h,t\rangle\\
&\overset{\beta}{=} h\ x
\end{aligned}
$$

$$
\begin{aligned}
\texttt{tl}^*(\texttt{corec}_\texttt{s}^*\ X\ h\ t\ x) &:= \texttt{rec}_\exists\ E\ (\lambda A.\lambda\langle a,b,c\rangle.\ \texttt{pack}\ A\ \langle c\ a,b,c\rangle)\ (\texttt{corec}_\texttt{s}^*\ X\ h\ t\ x)\\
&:= (\texttt{corec}_\texttt{s}^*\ X\ h\ t\ x)\ E\ (\lambda A.\lambda\langle a,b,c\rangle.\ \texttt{pack}\ A\ \langle c\ a,b,c\rangle)\\
&:= (\texttt{pack}\ X\ \langle x,h,t\rangle)\ E\ (\lambda A.\lambda\langle a,b,c\rangle.\ \texttt{pack}\ A\ \langle c\ a,b,c\rangle)\\
&:= (\lambda Y.\lambda k.k\ X\ \langle x,h,t\rangle)\ E\ (\lambda A.\lambda\langle a,b,c\rangle.\ \texttt{pack}\ A\ \langle c\ a,b,c\rangle)\\
&\overset{\beta}{=} (\lambda A.\lambda\langle a,b,c\rangle.\ \texttt{pack}\ A\ \langle c\ a,b,c\rangle)\ X\ \langle x,h,t\rangle\\
&\overset{\beta}{=} \texttt{pack}\ X\ \langle t\ x,h,t\rangle\\
&=: \texttt{corec}_\texttt{s}^*\ X\ h\ t\ (t\ x)
\end{aligned}
$$

□

We show a few examples of how to define streams using the corecursor.

Example 6.2.4. Let $E := \mathbb{N}$. We can define the following streams:

$$
\begin{aligned}
\texttt{zeroes} &:= \texttt{corec}_\texttt{s}^*\ \mathbb{N}\ (\lambda n.n)\ (\lambda n.0)\ 0\\
\texttt{ones} &:= \texttt{corec}_\texttt{s}^*\ \mathbb{N}\ (\lambda n.n)\ (\lambda n.1)\ 0\\
\texttt{incr} &:= \texttt{corec}_\texttt{s}^*\ \mathbb{N}\ (\lambda n.n)\ (\lambda n.n+1)\ 0
\end{aligned}
$$

Here `zeroes` is an infinite stream of zeroes, `ones` is an infinite stream of ones and `incr` is an infinite stream of natural numbers starting at zero and incrementing by one each time. We can see that `incr` behaves as expected by evaluating the first few elements of the stream using the β-rules of 6.2.3.

$$
\begin{aligned}
\texttt{hd}^*\ \texttt{tl}^*\ \texttt{incr} &:= \texttt{hd}^*\Big(\texttt{tl}^*\ (\texttt{corec}_\texttt{s}^*\ \mathbb{N}\ (\lambda n.n)\ (\lambda n.n+1)\ 0)\Big)\\
&\overset{\beta}{=} \texttt{hd}^*\Big(\texttt{corec}_\texttt{s}^*\ \mathbb{N}\ (\lambda n.n)\ (\lambda n.n+1)\ \big((\lambda n.n+1)\ 0\big)\Big)\\
&\overset{\beta}{=} \texttt{hd}^*\Big(\texttt{corec}_\texttt{s}^*\ \mathbb{N}\ (\lambda n.n)\ (\lambda n.n+1)\ 1\Big)\\
&\overset{\beta}{=} (\lambda n.n)\ 1 \overset{\beta}{=} 1
\end{aligned}
$$

We can also use $\texttt{corec}_\texttt{s}^*$ to define functions on streams.

Example 6.2.5. Let $E := \mathbb{N}$. We define a function `square` that takes a stream of natural numbers and returns a stream of the squares of those numbers.

$$
\texttt{square}\ (s : \texttt{Stream}^*) := \texttt{corec}_\texttt{s}^*\ \texttt{Stream}^*\ (\lambda x.(\texttt{hd}^*\ x) * (\texttt{hd}^*\ x))\ (\lambda x.\texttt{tl}^*\ x)\ s
$$

At a first glance, these stream types seem to be working quite well. We can construct them using the corecursor and destruct them using the head and tail functions. We have a way to compute using these streams because the β-rules are satisfied. However, we will see that the stream type does not satisfy the η-rule. If we construct two streams that *behave* exactly the same, but use a different internal representation, we would like to have that these streams are *propositionally* equal. Take for example `zeroes` and $\big(\texttt{corec}^*_{\texttt{s}}\ \mathbb{N}\ (\lambda n.n * 2)\ (\lambda n.0)\ 0\big)$. It is obvious that these streams will behave the same, therefore we would like to have an equality type between these different representations. This problem is usually solved using *bisimulation*, sometimes called *coinduction* [JR12]. Using the bisimulation principle, two streams that *behave* the same, actually *are* the same (propositionally). In the remainder of this chapter, we construct an impredicative encoding that satisfies the η-rule and use this to prove the bisimulation principle.

6.3 Categorical streams

Within category theory, we can define streams as the final co-algebra of the endofunctor $\mathcal{S}$. We instantiate the definitions of section 2.3. We again leave the category unspecified, but one can think of the category **Set**.

$$\begin{aligned} \mathcal{S}(X) &:= E \times X \\ \mathcal{S}(f : X \to Y) &:= \langle \mathrm{id}, f \rangle \end{aligned}$$

As shown in 2.3.5, we have a category of $\mathcal{S}$-coalgebras denoted as $\mathcal{S}$-**CoAlg** with objects $\langle X, \langle h, t \rangle \rangle$ where X is a set and $(h : X \to E)$ and $(t : X \to X)$ are functions. Morphisms in $\mathcal{S}$-**CoAlg** satisfy the following commutative diagram:

$$\begin{array}{ccc} X & \xrightarrow{\langle h,t \rangle} & E \times X \\ {\scriptstyle f}\downarrow & = & \downarrow{\scriptstyle \mathcal{S}(f)} \\ Y & \xrightarrow[\langle h',t' \rangle]{} & E \times Y \end{array}$$

Figure 7: Commutative diagram of $\mathcal{S}$-**CoAlg** morphisms.

In formulas this comes down to the following:

$$h' \circ f = h \wedge t' \circ f = f \circ t$$

The final object $\langle F, \langle \mathrm{hd}, \mathrm{tl} \rangle \rangle$ of the category of $\mathcal{S}$-coalgebras $\mathcal{S}$-**CoAlg** has exectly one morphism $\big(u_X : \langle X, \langle h, t \rangle \rangle \to \langle F, \langle \mathrm{hd}, \mathrm{tl} \rangle \rangle\big)$ for each object $\langle X, \langle h, t \rangle \rangle$ in the category $\mathcal{S}$-**CoAlg**. In addition to Figure 7, these morphisms need to satisfy the uniqueness requirement of 2.3.7. We make use of requirement 2). In formulas, this boils down to the following: for each $\mathcal{S}$-**CoAlg** morphism $(f : X \to Y)$, we have that $u_Y \circ f = u_X$.

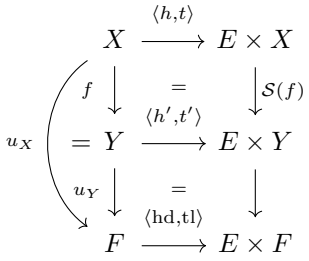


Figure 8: Diagram of the final $\mathcal{S}$-coalgebra.

6.4 Impredicative encoding of streams

As we saw before with lists, we shall now translate the categorical notion of final $\mathcal{S}$-coalgebra back to type theory. We define a predicate that states that $(f : X \to Y)$ forms an $\mathcal{S}$-coalgebra morphism.

Definition 6.4.1. Let $(X, Y : \mathcal{U})$, $(h : X \to E)$, $(t : X \to X)$, $(h' : Y \to E)$, $(t' : Y \to Y)$ and $(f : X \to Y)$. We define the predicate `MorphStream` that states that f forms an $\mathcal{S}$-coalgebra morphism. This predicate mirrors the commutative diagram from the categorical definition in Figure 7.

$$\Big(\texttt{MorphStream}\ X\ h\ t \quad Y\ h'\ t' \quad f : X \to Y := h' \circ f = h\ \wedge\ t' \circ f = f \circ t\Big)$$

Let us take a moment to think about what we want to achieve. We want to define an encoding of $\texttt{Stream}^*$ that satisfies the η-rule. This η-rule is taken from the final coalgebra of $\mathcal{S}$ as shown in Figure 8. Given a morphism $(f : X \to Y)$, we thus want that $u_Y \circ f = u_X$ for morphisms $(u_X : X \to F)$ and $(u_Y : Y \to F)$. In the case of `List`, the morphisms u_X and u_Y were given by the recursor. Taking the dual notion, we can see that these morphisms are created using the corecursor for $\texttt{Stream}^*$. We thus have $u_X := \texttt{corec}^*_{\texttt{s}}\ X\ h\ t$ and $u_Y := \texttt{corec}^*_{\texttt{s}}\ Y\ h'\ t'$.
In the case of `List`, we created a subtype of $\texttt{List}^*$, by pairing each list with a proof that the list satisfies the η-rule. This made $\texttt{List}^*$ an embedding of `List`. In the case of streams, we take the dual notion of an embedding: a quotient. We explain the duality and the general constructions more in chapter 7.
The quotient should equate two streams that are related by $u_Y \circ f = u_X$. To define this quotient, we create a relation `CoLimStr`. We relate two streams σ and τ if they can be translated into each other by some $\mathcal{S}$-morphism f. The relation `CoLimStr` is given below.

Definition 6.4.2. We define the relation `CoLimStr` between streams $(\sigma, \tau : \texttt{Stream}^*)$ where $(\texttt{CoLimStr}\ \sigma\ \tau)$ holds if σ and τ are related by some $\mathcal{S}$-coalgebra morphism.

$$\texttt{CoLimStr}\ \sigma\ \tau := \exists(X, Y : \mathcal{U}).\exists^{(h:X\to E)}_{(h':Y\to E)}.\exists^{(t:X\to X)}_{(t':Y\to Y)}.\exists(f : X \to Y).\exists(x : X) \tag{6.1}$$

$$\big(\texttt{MorphStream}\ X\ h\ t \quad Y\ h'\ t' \quad f\big) \wedge \tag{6.2}$$

$$\sigma = \texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ x \wedge \tag{6.3}$$

$$\tau = \texttt{corec}^*_{\texttt{s}}\ Y\ h'\ t'\ (f\ x) \tag{6.4}$$

Notation 6.4.3. We use the infix notation $(\sigma\ \equiv\ \tau)$ to denote that $(\texttt{CoLimStr}\ \sigma\ \tau)$ holds.

Lemma 6.4.4. The relation `CoLimStr` is neither *symmetric* nor *transitive*. By 5.1.8, this does not hinder our further progress since the equality relation on $(\texttt{cls}\ x)$ is an equivalence relation. When we speak about 'the equivalence classes of `CoLimStr`', we thus mean the equivalence classes of $(\texttt{cls}\ x)$.

Proof. We elaborate more on why none of these properties hold in chapter D. □

Lemma 6.4.5 $(\texttt{EqHd} : (\texttt{EqCls}\ \texttt{hd}^*\ \equiv))$**.** The head function $\texttt{hd}^*$ is constant on all equivalence classes of `CoLimStr`.

$$\prod(\sigma, \tau : \texttt{Stream}^*).\ (\sigma \equiv \tau)\ \Longrightarrow\ (\texttt{hd}^*\ \sigma = \texttt{hd}^*\ \tau)$$

Proof. Let $(\sigma, \tau : \texttt{Stream}^*)$ and suppose that $(\sigma \equiv \tau)$ holds. We then have that the following exists: $(X, Y : \mathcal{U}), {}^{(h:X\to E)}_{(h':Y\to E)}, {}^{(t:X\to X)}_{(t':Y\to Y)}, (f : X \to Y)$, and $(x : X)$ with

$$\sigma = \texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ x \tag{6.5}$$

$$\tau = \texttt{corec}^*_{\texttt{s}}\ Y\ h'\ t'\ (f\ x) \tag{6.6}$$

$$h \circ f = h' \tag{6.7}$$

$$t' \circ f = f \circ t$$

We conclude the following:

$$\begin{aligned}
\texttt{hd}^*\sigma &\overset{6.5}{=} \texttt{hd}^*(\texttt{corec}^*_{\texttt{s}}\ X\ h\ t\ x) \\
&\overset{\beta}{=} h\ x \\
&\overset{6.7}{=} h'\ (f\ x) \\
&\overset{\beta}{=} \texttt{hd}^*\ (\texttt{corec}^*_{\texttt{s}}\ Y\ h'\ t'\ (f\ x)) \\
&\overset{6.6}{=} \texttt{hd}^*\ \tau
\end{aligned}$$ □

Lemma 6.4.6 $(\texttt{EqTl} : (\texttt{EqCls}\ (\texttt{cls}\circ\texttt{tl}^*)\ \equiv))$**.** The lifted tail function $\texttt{cls}\circ\texttt{tl}^*$ is constant on all equivalence classes of `CoLimStr`.

$$\prod(\sigma, \tau : \texttt{Stream}^*).\ (\texttt{CoLimStr}\ \sigma\ \tau)\ \rightarrow\ \texttt{cls}\ (\texttt{tl}^*\ \sigma) = \texttt{cls}\ (\texttt{tl}^*\ \tau)$$

Proof. Let $(\sigma, \tau : \texttt{Stream}^*)$ and suppose $(\texttt{CoLimStr}\ \sigma\ \tau)$. We then have that the following exists: $(X, Y : \mathcal{U}), \substack{h:X\to E\\ h':Y\to E}, \substack{t:X\to X\\ t':Y\to Y}, (f : X \to Y)$ and $(x : Y)$ with:

$$\sigma = \texttt{corec}^*_\texttt{s}\ X\ h\ t\ x \tag{6.8}$$
$$\tau = \texttt{corec}^*_\texttt{s}\ Y\ h'\ t'\ (f\ x) \tag{6.9}$$
$$h \circ f = h'$$
$$t' \circ f = f \circ t \tag{6.10}$$

We conclude the following:

$$\begin{aligned}
\texttt{tl}^*\sigma &\overset{6.8}{=} \texttt{tl}^*(\texttt{corec}^*_\texttt{s}\ X\ h\ t\ x)\\
&\overset{\beta}{=} \texttt{corec}^*_\texttt{s}\ X\ h\ t\ (t\ x)\\
&\equiv \texttt{corec}^*_\texttt{s}\ Y\ h'\ t'\ (f \circ (t\ x))\\
&\overset{6.10}{=} \texttt{corec}^*_\texttt{s}\ Y\ h'\ t'\ (t' \circ (f\ x))\\
&\overset{\beta}{=} \texttt{tl}^*\ (\texttt{corec}^*_\texttt{s}\ Y\ h'\ t'\ (f\ x))\\
&\overset{6.9}{=} \texttt{tl}^*\ \tau
\end{aligned}$$

Because we have that $\big(\texttt{CoLimStr}\ (\texttt{tl}^*\ \sigma)\ (\texttt{tl}^*\ \tau)\big)$, it follows from 5.3.4 that $\texttt{cls} \circ \texttt{tl}^*$ is constant on all equivalence classes of $\texttt{CoLimStr}$. □

We are now ready to define the impredicative encoding of coinductive streams as the quotient of the $\texttt{CoLimStr}$ equivalence relation. We use our previously defined $\texttt{quot}$ type.

Definition 6.4.7. We define a **coinductive stream type** $\texttt{Stream}$.

$$\texttt{Stream} := \texttt{quot}\ \texttt{Stream}^*\ \texttt{CoLimStr}$$

We define the new head and tail functions $\texttt{hd}$ and $\texttt{tl}$ as the following lifted functions:

$$\begin{aligned}
\texttt{hd} &:= \overline{\texttt{hd}^*} &&:= \texttt{rec}_\texttt{q}\ E\ \texttt{hd}^*\ \texttt{EqHd}\\
\texttt{tl} &:= \overline{\texttt{cls} \circ \texttt{tl}^*} &&:= \texttt{rec}_\texttt{q}\ \texttt{Stream}\ (\texttt{cls} \circ \texttt{tl}^*)\ \texttt{EqTl}
\end{aligned}$$

We define the coinductive constructor $\texttt{corec}_\texttt{s}$ by lifting the corecursive constructor $\texttt{corec}^*_\texttt{s}$:

$$\begin{aligned}
&\texttt{corec}_\texttt{s} : \prod(X : \mathcal{U}).(X \to E) \to (X \to X) \to X \to \texttt{Stream}\\
&\texttt{corec}_\texttt{s} := \texttt{cls} \circ \texttt{corec}^*_\texttt{s}
\end{aligned} \tag{6.11}$$

In the remainder of this section, we show that $\texttt{Stream}$ is the final $\mathcal{S}$-coalgebra in the category $\mathcal{S}$-**CoAlg**. We first show that this new stream type still satisfies the β-rules.

Proposition 6.4.8. The new stream corecursor satisfies the following β-rules:

$$\begin{aligned}
&\mathtt{hd}\ (\mathtt{corec_s}\ X\ h\ t\ x) \overset{\beta}{=} h\ x \\
&\mathtt{tl}\ (\mathtt{corec_s}\ X\ h\ t\ x) \overset{\beta}{=} \mathtt{corec_s}\ X\ h\ t\ (t\ x)
\end{aligned}$$

Proof. The β rules for `Stream` follow from the β-rules of $\mathtt{Stream}^*$ and `quot`.

$$\begin{aligned}
\mathtt{hd}\ (\mathtt{corec_s}\ X\ h\ t\ x) &:= \overline{\mathtt{hd}^*}\ (\mathtt{cls} \circ \mathtt{corec_s^*}\ X\ h\ t\ x) \\
&\overset{\beta-\mathtt{quot}}{=} \mathtt{hd}^*(\mathtt{corec_s^*}\ X\ h\ t\ x) \\
&\overset{\beta-\mathtt{Stream}^*}{=} h\ x \\
\mathtt{tl}(\mathtt{corec_s}\ X\ h\ t\ x) &:= (\overline{\mathtt{cls} \circ \mathtt{tl}^*})\ (\mathtt{cls} \circ \mathtt{corec_s^*}\ X\ h\ t\ x) \\
&\overset{\beta-\mathtt{quot}}{=} (\mathtt{cls} \circ \mathtt{tl}^*)\ (\mathtt{corec_s^*}\ X\ h\ t\ x) \\
&\overset{\beta-\mathtt{Stream}^*}{=} \mathtt{cls}\ (\mathtt{corec_s^*}\ X\ h\ t\ (t\ x)) \\
&=: \mathtt{corec_s}\ X\ h\ t\ (t\ x)
\end{aligned}$$

□

Next, we show that $(\mathtt{corec_s^*}\ X\ h\ t)$ and `cls` are $\mathcal{S}$-morphisms.

Lemma 6.4.9. Let $(X : \mathcal{U})$, $(h : X \to E)$, $(t : X \to X)$ and $(x : X)$. We have that $(\mathtt{corec_s^*}\ X\ h\ t)$ forms an $\mathcal{S}$-morphism $(X \to \mathtt{Stream}^*)$:

$$\Big(\mathtt{MorphStream}\ X\ h\ t \quad \mathtt{Stream}^*\ \mathtt{hd}^*\ \mathtt{tl}^* \quad (\mathtt{corec_s^*}\ X\ h\ t)\Big)$$

Proof. The claim follows naturally from the β-rules of 6.4.8 and function extensionality. □

Lemma 6.4.10. The class function $(\mathtt{cls} : \mathtt{Stream}^* \to \mathtt{Stream})$ forms an $\mathcal{S}$-morphism.

$$\Big(\mathtt{MorphStream}\ \mathtt{Stream}^*\ \mathtt{hd}^*\ \mathtt{tl}^* \quad \mathtt{Stream}\ \mathtt{hd}\ \mathtt{tl} \quad \mathtt{cls}\Big)$$

Proof. We need to show $\mathtt{hd} = \mathtt{hd}^* \circ \mathtt{cls}$ and that $\mathtt{tl} \circ \mathtt{cls} = \mathtt{cls} \circ \mathtt{tl}^*$. Recall the β-rule for $\mathtt{rec_q}$, described in 5.3.5. We have that $\mathtt{hd} := \overline{\mathtt{hd}^*} \overset{\beta}{=} \mathtt{hd}^* \circ \mathtt{cls}$. Similarly, we have $\mathtt{tl} \circ \mathtt{cls} := \overline{\mathtt{cls} \circ \mathtt{tl}^*} \circ \mathtt{cls} \overset{\beta}{=} \mathtt{cls} \circ \mathtt{tl}^*$. □

Before we show that the η-rule holds for this improved `Stream` type, we first need to prove the following helper lemmas.

Lemma 6.4.11. Let $(\sigma : \mathtt{Stream}^*)$ be a stream. Then $(\mathtt{corec_s^*}\ \mathtt{Stream}^*\ \mathtt{hd}^*\ \mathtt{tl}^*\ (\mathtt{cls}\ \sigma))$ and $(\mathtt{corec_s^*}\ \mathtt{Stream}\ \mathtt{hd}\ \mathtt{tl}\ \sigma)$ are related by `CoLimStr`.

$$(\mathtt{corec_s^*}\ \mathtt{Stream}^*\ \mathtt{hd}^*\ \mathtt{tl}^*\ \sigma) \equiv (\mathtt{corec_s^*}\ \mathtt{Stream}\ \mathtt{hd}\ \mathtt{tl}\ (\mathtt{cls}\ \sigma)) \tag{6.12}$$

Proof. By 6.4.10, we have that `cls` is a morphism. Therefore, the `CoLimStr` predicate, instantiated using the obvious choices for X, Y, h, h', t, t', f and x follows directly. □

Lemma 6.4.12. Let $(\sigma : \texttt{Stream}^*)$ be a stream. Then $(\texttt{corec}_\texttt{s}^*\ \texttt{Stream}^*\ \texttt{hd}^*\ \texttt{tl}^*\ \sigma)$ and σ are related by `CoLimStr`.

$$(\texttt{corec}_\texttt{s}^*\ \texttt{Stream}^*\ \texttt{hd}^*\ \texttt{tl}^*\ \sigma) \equiv \sigma \tag{6.13}$$

Proof. We shall make this proof as precise as possible to highlight the inner workings of the existential type used to define $\texttt{Stream}^*$. To show that the above relation holds, we need to find suitable instances of X, Y, h, h', t, t', f and x for 6.4.2. We take $\sigma := \sigma$ and $\tau := (\texttt{corec}_\texttt{s}^*\ \texttt{Stream}^*\ \texttt{hd}^*\ \texttt{tl}^*\ \sigma)$. We immediately see some good candidates for $Y := \texttt{Stream}^*$, $h' := \texttt{hd}^*$ and $t' := \texttt{tl}^*$. We can destruct the existential type σ using $\texttt{rec}_\exists$ to obtain $(X : \mathcal{U})$, $(h : Y \to E)$, $(t : Y \to Y)$ and $(x : X)$. We are left to show the following three equations:

$$\texttt{MorphStream}\ X\ h\ t \quad \texttt{Stream}^*\ \texttt{hd}^*\ \texttt{tl}^* \quad f \tag{6.14}$$

$$\sigma = \texttt{corec}_\texttt{s}^*\ X\ h\ t\ x \tag{6.15}$$

$$\tau = (\texttt{corec}_\texttt{s}^*\ \texttt{Stream}^*\ \texttt{hd}^*\ \texttt{tl}^*\ (f\ x)) \tag{6.16}$$

Let's take a look at Equation (6.15). By destructing σ, we obtained X, h, t, and x. There is however a fundamental problem. We cannot relate these choices of X, h, t, and x back to the original existential type σ. For this, we need 6.1.4. We apply the axiom below:

$$\sigma \overset{\texttt{ExistsId}}{=} \texttt{pack}\ X\ \langle x, h, t\rangle =: \texttt{corec}_\texttt{s}^*\ X\ h\ t\ x \tag{6.17}$$

We are now ready to look for a suitable candidate for $(f : X \to Y)$. We use $(f := \texttt{corec}_\texttt{s}^*\ X\ h\ t)$. By 6.4.9 we know that $(\texttt{corec}_\texttt{s}^*\ X\ h\ t)$ is a morphism. We thus have that Equation (6.14) is satisfied. Additionally, by Equation (6.17), we have that $(f\ x) := (\texttt{corec}_\texttt{s}^*\ X\ h\ t)\ x \overset{6.17}{=} \sigma$. Therefore, Equation (6.16) follows trivially. □

Lemma 6.4.13. The stream recursor, together with the head function `hd` and the tail function `tl` form the identity.

$$\texttt{corec}_\texttt{s}\ \texttt{Stream}\ \texttt{hd}\ \texttt{tl} = \texttt{id}_{\texttt{Stream}}$$

Proof. By 5.3.6, we know that $\overline{\texttt{cls}} = \texttt{id}_{\texttt{Stream}}$, where $\overline{\texttt{cls}} := \texttt{rec}_\texttt{q}\ \texttt{Stream}\ \texttt{cls}\ \texttt{EqCls2}$. If we then apply the η-rule of the quotient type (5.3.7), we see that, in order to show that $\texttt{corec}_\texttt{s}\ \texttt{Stream}\ \texttt{hd}\ \texttt{tl} = \texttt{id}_{\texttt{Stream}} = \texttt{rec}_\texttt{q}\ \texttt{Stream}\ \texttt{cls}\ \texttt{EqCls2}$, it is enough to show the following equation:

$$(\texttt{corec}_\texttt{s}\ \texttt{Stream}\ \texttt{hd}\ \texttt{tl}) \circ \texttt{cls} = \texttt{cls} \tag{6.18}$$

We reduce Equation (6.18) using function extensionality, the definition of $\texttt{corec}_\texttt{s}$ and 5.3.4.

$$
\begin{array}{lll}
 & \mathtt{corec_s\ Stream\ hd\ tl} \circ \mathtt{cls} = \mathtt{cls} & \overset{\text{FunExt}}{\Longleftrightarrow} \\
\forall (s : \mathtt{Stream}^*) & \mathtt{corec_s\ Stream\ hd\ tl}\ (\mathtt{cls}\ s) = \mathtt{cls}\ s & \overset{6.11}{\Longleftrightarrow} \\
\forall (s : \mathtt{Stream}^*) & \mathtt{cls}\ \left(\mathtt{corec_s^*\ Stream\ hd\ tl}\ (\mathtt{cls}\ s)\right) = \mathtt{cls}\ s & (6.19)
\end{array}
$$

We shall prove Equation (6.19). Assume a stream $(s : \mathtt{Stream}^*)$. We use the existential recursor $\mathtt{rec}_\exists$ and 6.1.4 to destruct s. We thus destruct $(s = \mathtt{corec_s^*}\ X\ h\ t\ x)$ for some $(X : \mathcal{U})$, $(h : X \to E)$, $(t : X \to X)$ and $(x : X)$. We use 5.3.4 for the equational reasoning below:

$$
\begin{aligned}
\mathtt{cls}\ \left(\mathtt{corec_s^*\ Stream\ hd\ tl}\ (\mathtt{cls}\ s)\right) &\overset{6.4.11}{=} \mathtt{cls}\ \left(\mathtt{corec_s^*\ Stream^*\ hd^*\ tl^*}\ s\right) \\
&\overset{6.4.12}{=} \mathtt{cls}\ s
\end{aligned}
$$

□

We are now ready to show that the recursor is unique.

Theorem 6.4.14. For all $(X : \mathcal{U}), (h : X \to E), (t : X \to X)$ and $(f : X \to \mathtt{Stream})$ we have:

$$
\left(\mathtt{MorphStream}\ X\ h\ t \quad \mathtt{Stream\ hd\ tl} \quad f\right) \implies (f = \mathtt{corec_s}\ X\ h\ t)
$$

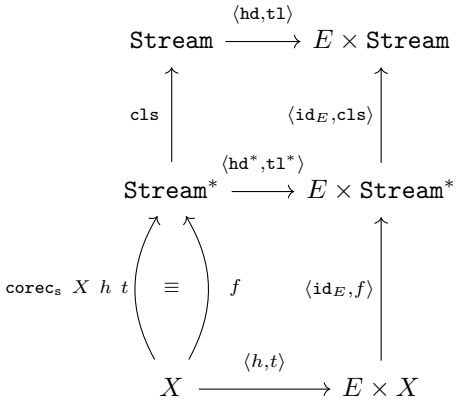
Proof. Assume the following:

$$
\begin{array}{l}
(X : \mathcal{U}) \\
(h : X \to E) \\
(t : X \to X) \\
(f : X \to \mathtt{Stream}) \\
\left(\mathtt{MorphStream}\ X\ h\ t \quad \mathtt{Stream\ hd\ tl} \quad f\right)
\end{array}
$$

By function extensionality, it is enough to prove $(f\ x = \mathtt{corec_s}\ X\ h\ t\ x)$ for all $(x : X)$. We can apply the result from 6.4.13 and the fact that $(\mathtt{corec_s^*}\ X\ h\ t\ x)$ and $(\mathtt{corec_s^*\ Stream\ hd\ tl}\ (f\ x))$ are related by `CoLimStr`.

$$
\begin{aligned}
f\ x &\overset{6.4.13}{=} \mathtt{corec_s^*\ Stream\ hd\ tl}\ (f\ x) \\
&= \mathtt{corec_s^*}\ X\ h\ t\ x
\end{aligned}
$$

□



6.5 Stream bisimulation principle

In this section, we show that the coinduction principle, also known as the bisimulation principle, holds for the newly defined stream type `Stream`. The bisimulation principle states that two streams are equal if they have the same behavior, i.e. if they are related by a bisimulation relation. We define a bisimulation as a relation `BiSimStream` and show that two streams are equal if and only if they are related by this relation.

Definition 6.5.1. We define the bisimulation relation `BiSim` that relates two streams if they can be related via some bisimulation R. A relation R is a bisimulation if the predicate (`IsBiSim` R) holds. A bisimulation between streams $(\sigma, \tau : \texttt{Stream})$ is a relation that implies the heads of both streams are equal and the tails are related.
We define the bisimulation relation that relates streams with the same behavior. Plainly, two streams are bisimilar when their heads are equal and their tails are bisimilar:

$$\texttt{BiSim}\ \sigma\ \tau := \sum(R : \texttt{Stream} \to \texttt{Stream} \to \mathcal{U}).\texttt{IsBiSim}\ R \wedge R\ \sigma\ \tau$$
$$\texttt{IsBiSim}\ R := \prod(\sigma, \tau : \texttt{Stream}).R\ \sigma\ \tau \to \texttt{hd}\ \sigma = \texttt{hd}\ \tau \wedge R\ (\texttt{tl}\ \sigma)\ (\texttt{tl}\ \tau)$$

Notation 6.5.2. We denote the relation `BiSim` using the infix symbol $\sim$, so instead of writing (`BiSim` $\sigma\ \tau$) we write $(\sigma \sim \tau)$.

Lemma 6.5.3. The propositional equality relation on streams is a bisimulation on streams. So for all $(\sigma, \tau : \texttt{Stream})$ we have $(\sigma = \tau) \implies (\sigma \sim \tau)$. We thus have that the predicate `IsBiSim` holds for $=$.

$$\texttt{IsBiSim}\ (\lambda\sigma.\lambda\tau.\sigma = \tau) \tag{6.20}$$

Proof. Define $R := (\lambda\sigma.\lambda\tau.\ \sigma = \tau)$. Let $(\sigma, \tau : \texttt{Stream})$ and suppose that $(R\ \sigma\ \tau)$. By definition of R we have that $\sigma = \tau$. It follows that $\texttt{hd}\ \sigma = \texttt{hd}\ \tau$. We also have that $\texttt{tl}\ \sigma = \texttt{tl}\ \tau$, by which we have that $R\ (\texttt{tl}\ \sigma)\ (\texttt{tl}\ \tau)$. We conclude that $\texttt{hd}\ \sigma = \texttt{hd}\ \tau \wedge R\ (\texttt{tl}\ \sigma)\ (\texttt{tl}\ \tau)$. □

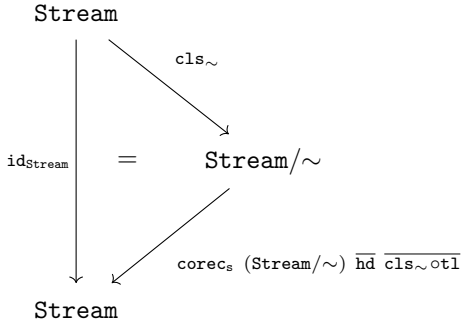
Theorem 6.5.4. We have the following *coinduction* proof principle `CoIndStream`.

$$\texttt{CoIndStream}\ \sigma\ \tau := \sigma \sim \tau \iff \sigma = \tau$$

Proof. ($\Longleftarrow$) This follows from 6.5.3.
($\Longrightarrow$) Let $(\sigma, \tau : \texttt{Stream})$. Assume we have $\sigma \sim \tau$.
We define the following quotient:

$$\texttt{Stream}/\sim := \texttt{quot Stream BiSim}$$

The proofs that $\overline{\texttt{hd}}$ and $\overline{\texttt{cls}_\sim \circ \texttt{tl}}$ are well-defined are very similar to 6.4.5 and 6.4.6 and have been omitted. We show that the following diagram commutes.



By $\sigma \sim \tau$ there exists a bisimulation $(R : \texttt{Stream} \to \texttt{Stream} \to \mathcal{U})$ with $(R\ \sigma\ \tau)$ and $(\texttt{IsBiSim}\ R)$. So we have $\texttt{hd}\ \sigma = \texttt{hd}\ \tau$ and $R\ (\texttt{tl}\ \sigma)\ (\texttt{tl}\ \tau)$.
Define $f := \left(\texttt{corec}_\texttt{s}\ (\texttt{Stream}/\sim)\ \overline{\texttt{hd}}\ \overline{\texttt{cls}_\sim \circ \texttt{tl}}\right) \circ \texttt{cls}_\sim$. We prove that f forms an $\mathcal{S}$-morphism by showing that the predicate $\left(\texttt{MorphStream Stream hd tl}\quad \texttt{Stream hd tl}\quad f\right)$ holds.

$$\begin{aligned}
\texttt{hd}\ (f\ \sigma) &:= \texttt{hd}\ (\texttt{corec}_\texttt{s}\ (\texttt{Stream}/\sim)\ \overline{\texttt{hd}}\ \overline{\texttt{cls}_\sim \circ \texttt{tl}}\ (\texttt{cls}_\sim\ \sigma)) \\
&\overset{\beta}{=} \overline{\texttt{hd}}\ (\texttt{cls}_\sim\ \sigma) \\
&\overset{5.3.5}{=} \texttt{hd}\ \sigma
\end{aligned}$$

$$\begin{aligned}
\texttt{tl}\ (f\ \sigma) &:= \texttt{tl}\ (\texttt{corec}_\texttt{s}\ (\texttt{Stream}/\sim)\ \overline{\texttt{hd}}\ \overline{\texttt{cls}_\sim \circ \texttt{tl}}\ (\texttt{cls}_\sim\ \sigma)) \\
&\overset{\beta}{=} \texttt{corec}_\texttt{s}\ (\texttt{Stream}/\sim)\ \overline{\texttt{hd}}\ \overline{\texttt{cls}_\sim \circ \texttt{tl}}\ (\overline{\texttt{cls}_\sim \circ \texttt{tl}}\ (\texttt{cls}_\sim\ \sigma)) \\
&\overset{5.3.5}{=} \texttt{corec}_\texttt{s}\ (\texttt{Stream}/\sim)\ \overline{\texttt{hd}}\ \overline{\texttt{cls}_\sim \circ \texttt{tl}}\ (\texttt{cls}_\sim\ (\texttt{tl}\ \sigma)) \\
&\overset{\beta}{=} f\ (\texttt{tl}\ \sigma)
\end{aligned}$$

By the application of the η-rule for streams (6.4.14), we find that $f = \texttt{corec}_\texttt{s}\ \texttt{Stream hd tl}$. We thus have for all streams $(\sigma : \texttt{Stream})$, that the following holds:

$$f\ \sigma := (\texttt{corec}_\texttt{s}\ (\texttt{Stream}/\sim)\ \overline{\texttt{hd}}\ \overline{\texttt{cls}_\sim \circ \texttt{tl}}\ (\texttt{cls}_\sim\ \sigma)) = (\texttt{corec}_\texttt{s}\ \texttt{Stream hd tl}\ \sigma) \tag{6.21}$$

Because $(\sigma \sim \tau)$, it follows from 5.1.7, that:

$$\mathtt{cls}_{\sim}\ \sigma = \mathtt{cls}_{\sim}\ \tau \tag{6.22}$$

We can now derive the following:

$$\begin{aligned}
\sigma &\overset{6.4.13}{=} \mathtt{corec_s\ Stream\ hd\ tl}\ \sigma \\
&\overset{6.21}{=} \mathtt{corec_s}\ (\mathtt{Stream}/\sim)\ \overline{\mathtt{hd}}\ \overline{\mathtt{cls}_{\sim} \circ \mathtt{tl}}\ (\mathtt{cls}_{\sim}\ \sigma) \\
&\overset{6.22}{=} \mathtt{corec_s}\ (\mathtt{Stream}/\sim)\ \overline{\mathtt{hd}}\ \overline{\mathtt{cls}_{\sim} \circ \mathtt{tl}}\ (\mathtt{cls}_{\sim}\ \tau) \\
&\overset{6.21}{=} \mathtt{corec_s\ Stream\ hd\ tl}\ \tau \\
&\overset{6.4.13}{=} \tau
\end{aligned}$$

We conclude that $\sigma = \tau$. □

7 Generalization

In this chapter, we zoom out and look at the general construction that was used to define `List` and `Stream`. We first look at the general construction of creating inductive and coinductive types in System F. We then look at the generic requirements of the initiality and finality of the $\mathcal{L}$-algebra and $\mathcal{S}$-coalgebra respectively. We shall see that the encodings are defined as well-chosen limit and colimit respectively.
Let's summarize the constructions of the previous chapters. We defined `List` as the initial algebra of the functor $\mathcal{L}(X) := 1 + E \times X$ and `Stream` as the final coalgebra of the functor $\mathcal{S}(X) := E \times X$. Both already satisfied the expected β-rules, (which made them weakly initial and weakly final respectively). We had to encode the η-rules so the uniqueness of the defining morphisms was guaranteed. We used a subtype to encode this for `List` and a quotient type to encode this for `Stream`. For `List` this gave us the induction principle and for `Stream` this gave us the coinduction principle (often called the bisimulation principle).

7.1 Impredicative definition

The first question that might arise is: given a functor F, how can we find the right impredicative definition? The answer can be found in a general construction of free data structures as described in [GTL93, 11.4. Representation of free structure]. We can describe any data structure given by a positive endofunctor F using the following System F type:

$$\prod(X : \mathcal{U})X^{F(X)} \to X \tag{7.1}$$

Let's look at how we can obtain the definition of $\texttt{List}^*$ from Equation (7.1). Before we continue, we need the following currying isomorphism:

$$\big(\sum(a : A).B(a) \to X\big) \to X \cong \prod(a : A).(B(a) \to X) \to X \tag{7.2}$$

This isomorphism holds in a very general setting [nLa24]. It is essentially the same idea that lets us consider a function that takes two arguments to be isomorphic to a function that takes one argument and returns a function that takes the second argument, also known as currying [Rij22, Section 4.6].
Using this definition, we can describe inductive types. Let us look at the definition of `List` with the functor $\mathcal{L}(X) := 1 + E \times X$.

$$\begin{aligned}
\prod(X : \mathcal{U})X^{\mathcal{L}(X)} \to X &:= \prod(X : \mathcal{U}).X^{1+E\times X} \to X \\
&:= \prod(X : \mathcal{U}).\big((1 + E \times X) \to X\big) \to X \\
&\cong \prod(X : \mathcal{U}).\big(X \times (E \times X \to X)\big) \to X \\
&\overset{7.2}{\cong} \prod(X : \mathcal{U}).X \to (E \to X \to X) \to X \\
&=: \texttt{List}^*
\end{aligned}$$

For coinductive data type, the story is reversed. We can describe any data structure given by a positive endofunctor F using the following System F type [Geu14b, p. 4.2]:

$$\exists(X : \mathcal{U}).X \times (F(X))^X \tag{7.3}$$

Using this definition, we can describe coinductive types. Let us look at the definition of `Stream` with the functor $\mathcal{S}(X) := E \times X$. We can describe the type of streams as follows:

$$\begin{aligned}\exists(X : \mathcal{U}).X \times (\mathcal{S}(X))^X &:= \exists(X : \mathcal{U}).X \times (E \times X)^X \\ &:= \exists(X : \mathcal{U}).X \times \big(X \to (E \times X)\big) \\ &\cong \exists(X : \mathcal{U}).X \times (X \to E) \times (X \to X) \\ &=: \texttt{Stream}^*\end{aligned}$$

7.2 Ensuring the η-rules

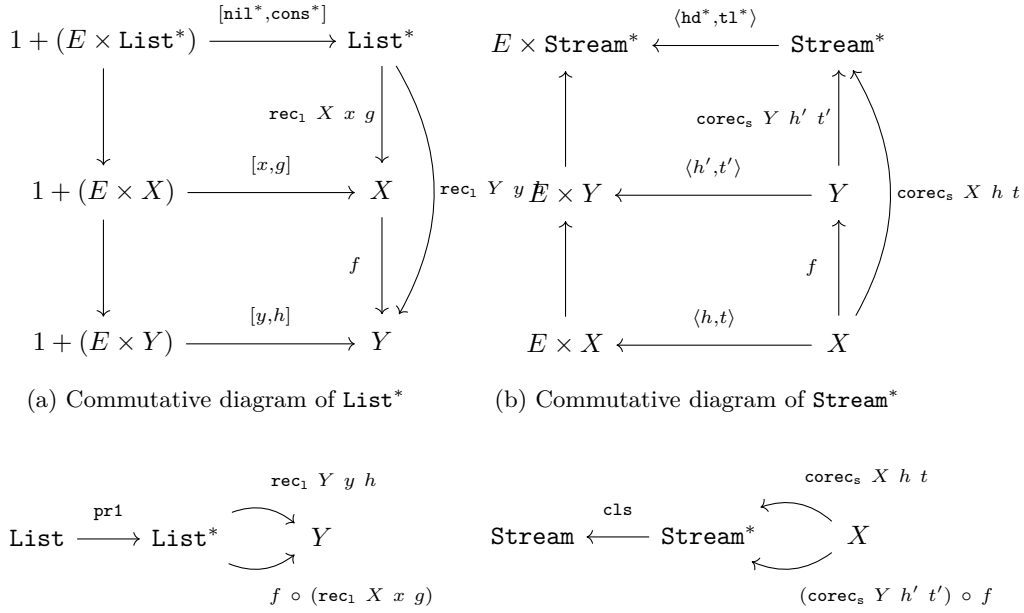
For `List` we wanted to guarantee the uniqueness of $(\texttt{rec}_\texttt{l}\ X\ x\ g)$. We ensured this by requiring that the following equality holds for $\mathcal{L}$-morphisms f:

$$f \circ (\texttt{rec}_\texttt{l}\ X\ x\ g) = \texttt{rec}_\texttt{l}\ Y\ y\ h$$

For `Stream` we wanted to guarantee the uniqueness of $(\texttt{corec}_\texttt{s}\ X\ h\ t)$. We ensured this by requiring that the following equality holds for $\mathcal{S}$-morphisms f:

$$(\texttt{corec}_\texttt{s}\ Y\ h'\ t') \circ f = \texttt{corec}_\texttt{s}\ X\ h\ t$$

We can describe these constructions using equalizers (Figure 9(c)) and coequalizers (Figure 9(d)). To obtain the initial object of $\mathcal{L}$-**Alg**, we take the equalizer of the two morphisms $f \circ (\texttt{rec}_\texttt{l}\ X\ x\ g)$ and $\texttt{rec}_\texttt{l}\ Y\ y\ h$, which is $\langle \texttt{List}, \texttt{pr1} \rangle$. To obtain the final object of $\mathcal{S}$-**CoAlg**, we take the coequalizer of the two morphisms $\texttt{corec}_\texttt{s}\ Y\ h'\ t' \circ f$ and $\texttt{corec}_\texttt{s}\ X\ h\ t$, which is $\langle \texttt{Stream}, \texttt{cls} \rangle$.
To be precise, we actually need a (co)equalizer for each free variable that appears in the diagram (for each of the quantifiers in the definition of `LimList` and `CoLimStr`). This means that `List` and `Stream` are the (co)limits of some big commutative diagrams. Since (co)limits are nothing more than a product of (co)equalizers, we can focus on the given (co)equalizers and leave the quantification implicit. This significantly reduces the complexity of the diagrams below.



(a) Commutative diagram of $\texttt{List}^*$

(b) Commutative diagram of $\texttt{Stream}^*$

(c) Equalizer diagram of `List`

(d) Co-equalizer diagram of `Stream`

Figure 9: Diagrams of `List` and `Stream`

7.3 Alternative predicates

In this thesis, we used the limit and colimit to encode the (co)inductive data types similar to [AFS18]. It is also possible to take different predicates/relations. One can for example directly encode the induction principle within the embedding, by creating a predicate `Ind` as was done in [Rip23]. It also seems possible to directly encode the η-rule in a way that is closer to Requirement 1) of 2.2.7 and Requirement 1) of 2.3.7 using a predicate `Unq`. Such a predicate would closely match the statement of the various η-rules in this thesis. In the table below, we summarized the different encodings and how they relate to the different theorems/definitions in this thesis.

`Lim`	8.4.2	`CoLim`	9.4.2
`Unq`	8.4.8	`Unq`	9.4.14
`Ind`	8.5.5	`CoInd`	9.5.4

Regardless of the exact choice of the embedding or quotient; the principle idea remains the same: we encode the (co)limit/uniqueness/(co)induction principles within the definition of the type.
In the next chapter, we use this knowledge about the general construction to create an embedding of a W-type. This shows that the technique of [AFS18] can be generalized to arbitrary (non-higher) inductive types.

8 W-types

In this chapter, we introduce W-types. W-types generalize inductive types such as natural numbers, lists, and trees. They were originally introduced by Martin-Löf [MS84, Wellorderings] as a way to define inductive data structures within type theory. In this chapter, we give an impredicative definition of W-types. This impredicative definition yields the expected β-rule, just like we saw for the `List`* type in chapter 4. We again create an impredicative encoding that ensures the η-rule also holds. We finish by using this η-rule to prove the induction principle for W-types.

8.1 Introduction to W-types

Before we give a formal definition of W-types, we first sketch the intuitive idea. For further reading, we recommend [HOT13, 5.3 W-Types].
The essential property of inductive data structures is of course their self-referential nature. Natural numbers are created by repeatedly taking the successors of natural numbers and lists are created by appending elements to lists. This self-referential property is quite explicit within functional programming languages like Haskell. Take for example a Haskell definition of natural numbers and lists:

```
data Nat = Z | S Nat
data List = Nil | Cons E List
```

In this context, the explicit usage of `Nat` and `List` on both sides of the definition highlights the inherent recursive nature of inductive types. It also highlights that inductive types are created using several constructors. For example, natural numbers have two constructors: `Z` and `S`. Each of these constructors has an arity. This is the number of arguments that the constructor requires. The constructor `Z` has arity 0 since it does not require any arguments. The constructor S has arity 1 because we require one argument. For the list type, we could say that it also has two constructors. One can also consider `Cons` as a *family* of constructors: one for each $(e : E)$. So if E has infinitely many inhabitants, then the list type has infinitely many constructors. Each of these constructors combines the arguments in some way to produce an element of the inductive type. An inductive type can thus be defined using a set of constructors that each have an arity.
The concept of W-types captures this idea. We have a type $(A : \mathcal{U})$ of *labels*. Each label represents one of the constructors of the inductive type. For each label $(a : A)$, we have a type $(B(a) : \mathcal{U})$ that describes the arity of the constructor.

Notation 8.1.1. To denote the W-type with label type $(A : \mathcal{U})$ and arity type $(B : A \to \mathcal{U})$, we write $W(a : A).B(a)$, or W_{AB}.

Note that the existence of W-types is often postulated as an axiom. The beauty of the following approach is that we get W-types as a constructable type. Let us take a look at the W-types for natural numbers and lists.

Example 8.1.2. We can define the natural numbers as a W-type. We need to find suitable instances of A and B. As we have seen, the natural numbers are made up of two constructors. Therefore, we define our label type A as a

type that has two inhabitants. We use ($A := \mathbf{1} + \mathbf{1}$). For each of the labels (($\texttt{inl}\ *) : \mathbf{1} + \mathbf{1}$) and (($\texttt{inr}\ *) : \mathbf{1} + \mathbf{1}$) we define the arity. We construct the arity type ($a : A \vdash B(a)$) using the recursor of $+$. We map ($\texttt{inl}\ *$) to $\mathbf{0}$ and ($\texttt{inr}\ *$) to $\mathbf{1}$. Our type $B(a)$ then becomes ($B := [\lambda x.\mathbf{0}, \lambda x.\mathbf{1}]$). We thus define the W-variant of natural numbers as follows:

$$\mathbb{N}^W := W(a : \mathbf{1} + \mathbf{1}).[\lambda x.\mathbf{0}, \lambda x.\mathbf{1}]\ a$$

Example 8.1.3. Let us also define lists using W-types. We need a suitable label type that captures that we either create an empty list or append some element ($e : E$) to the list. For this purpose we define ($A := \mathbf{1} + E$). The empty list does not require any recursive arguments. The cons constructor requires one. The resulting W-type thus becomes:

$$\texttt{List}^W := W(a : \mathbf{1} + E).[\lambda x.\mathbf{0}, \lambda e.\mathbf{1}]\ a$$

Next, we take a look at how we construct elements of W-types. We have one constructor called `sup`. This function simply takes one of the constructors, and its arguments, and creates an element of the W-type. The basic idea is as follows: In the case of $\mathbb{N}^W$, we create 0^W using `sup(Z, *)` and 1^W using `sup(S, sup(Z,*))`. Likewise, we create the list $[1, 2]^W$ as follows:

$$[1, 2]^W := \texttt{sup(Cons1, sup(Cons2, sup(Nil, *)))}$$

The `sup` constructor is nothing more than an interface for the original constructors.

8.2 Impredicative W-type

In this section, we give an impredicative definition of W-types. Before we begin, we would like to remind the reader that, when we write $(\sum(a : A).B(a) \to X)$, the binding is as follows: $(\sum(a : A).(B(a) \to X))$. In other words, arrows bind stronger than sigma-types.
In the literature, e.g. [HOT13, 5.3 W-Types], we see that W-types can be defined as the initial algebra of the functor $\mathcal{W}$ below.

Definition 8.2.1. We define the endofunctor ($\mathcal{W} : \mathcal{C} \to \mathcal{C}$). We leave the category $\mathcal{C}$ unspecified.

$$\mathcal{W}(X) := \sum(a : A).B(a) \to X$$
$$\mathcal{W}(f) := \langle \pi_1, f \circ \pi_2 \rangle$$

Of course, we aim at *defining* W-types, not *postulating* them. To that effect, we follow the same approach as we did for $\texttt{List}^*$ in chapter 4 and detailed in chapter 7. We first define the type $W^*(a : A).B(a)$, which serves as our impredicative basis. We again need the currying isomorphism of Equation (7.2). We instantiate Equation (7.1) using the functor $\mathcal{W}$ and apply Equation (7.2).

$$\begin{aligned}\prod(X:\mathcal{U}).X^{\mathcal{W}(X)} \to X &:= \prod(X:\mathcal{U}).X^{\sum(a:A).B(a)\to X} \to X \\ &:= \prod(X:\mathcal{U}).\Big(\big(\sum(a:A).B(a)\to X\big)\to X\Big)\to X \\ &\overset{7.2}{\cong} \prod(X:\mathcal{U}).\Big(\prod(a:A).(B(a)\to X)\to X\Big)\to X\end{aligned}$$

We are able to give an impredicative definition of W-types.

Definition 8.2.2. We define **impredicative W-types**. Let $(A:\mathcal{U})$ be a label type and $(B:A\to\mathcal{U})$ be a type describing the arities of these labels.

$$W^*(a:A).B(a) := \prod(X:\mathcal{U}).\Big(\prod(a:A).(B(a)\to X)\to X\Big)\to X \qquad \text{(formation)}$$

We define the $\mathtt{sup}^*$ constructor.

$$\begin{aligned}\mathtt{sup}^* :&\ \prod(a:A).\prod(r:B(a)\to W^*_{AB})\to W^*_{AB} \qquad \text{(introduction)}\\ \mathtt{sup}^*\ a\ r :=&\ \lambda(X:\mathcal{U}).\lambda\big(g:\prod(a:A).(B(a)\to X)\to X\big).\\ &\ g\ a\ \big(\lambda b.r\ b\ X\ g\big)\end{aligned}$$

Example 8.2.3. Using the formally defined $\mathtt{sup}^*$ function, we define 0^{W^*} and 1^{W^*} and $[1,2]^{W^*}$. First, we would like to remind the reader that we can always create an inhabitant of the type $\mathbf{0}\to X$, using $\mathtt{rec_0}$, as defined in chapter B.

$$\begin{aligned}0^{W^*} &:= \mathtt{sup}^*\ (\mathtt{inl}\ *)\ (\lambda(b:\mathbf{0}).\mathtt{rec_0}\ \mathbb{N}^{W^*}\ b)\\ 1^{W^*} &:= \mathtt{sup}^*\ (\mathtt{inr}\ *)\ (\lambda(b:\mathbf{1}).0^{W^*})\\ 1^{W^*} &:= \mathtt{sup}^*\ (\mathtt{inr}\ *)\ (\lambda(b:\mathbf{1}).1^{W^*})\\ [1,2]^{W^*} &:= \mathtt{sup}^*\ (\mathtt{inr}\ 1)\ \Big(\lambda(b:\mathbf{1}).\mathtt{sup}^*\ (\mathtt{inr}\ 2)\ \big(\lambda(b:\mathbf{1}).\mathtt{sup}^*\ (\mathtt{inl}\ *)\ \big(\lambda(b:\mathbf{0}).\mathtt{rec_0}\ \mathtt{List}^{W^*}\ b\big)\big)\Big)\end{aligned}$$

It is time to create a way to compute using the W-types by giving the recursion principle.

Definition 8.2.4. We define the W**-recursor**.

$$\mathtt{rec}^*_{\mathtt{W}} : \prod(X:\mathcal{U}).\Big(\prod(a:A).(B(a)\to X)\to X\Big)\to W^*_{AB}\to X \qquad \text{(elimination)}$$

$$\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ w := w\ X\ g$$

This recursion principle behaves as one would expect. It thus satisfies the β-rule below.

Lemma 8.2.5. The following β-rule holds for W^*_{AB}. Let $(X:\mathcal{U})$, $(a:A)$ and $(r:B(a)\to W^*_{AB})$ be given. Suppose $\big(g:\prod(a:A).(B(a)\to X)\to X\big)$.

$$\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{sup}^*\ a\ r) \overset{\beta}{=} g\ a\ ((\mathtt{rec}^*_{\mathtt{W}}\ X\ g)\circ r) \qquad \text{(computation)}$$

Proof.

$$\begin{aligned}
\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{sup}^*\ a\ r) &:= (\mathtt{sup}^*\ a\ r)\ X\ g \\
&\overset{\beta}{=} g\ a\ (\lambda b.\ r\ b\ X\ g) \\
&\overset{\beta}{=} g\ a\ (\lambda b.\ \mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (r\ b)) \\
&\overset{\beta}{=} g\ a\ ((\mathtt{rec}^*_{\mathtt{W}}\ X\ g) \circ r)
\end{aligned}$$

□

Example 8.2.6. We define a function `doubleW` that doubles a natural number. The recursive schema is as follows:

$$\begin{aligned}
\mathtt{0} &\longmapsto \mathtt{0} \\
\mathtt{S}\ n &\longmapsto \mathtt{S}\ (\mathtt{S}\ n)
\end{aligned}$$

The function is defined using the recursor for W-types:

$$\begin{aligned}
\mathtt{doubleW} &: \mathbb{N}^{W^*} \to \mathbb{N}^{W^*} \\
\mathtt{doubleW}\ n &:= \mathtt{rec}^*_{\mathtt{W}}\ \mathbb{N}^{W^*}\ g\ n
\end{aligned}$$

We have to define g. We first look at the type of g. We named the inner function t for convenience.

$$g : \prod(a : \mathbf{1} + \mathbf{1}).\ \prod\Big(t : \big([\lambda x.\mathbf{0}, \lambda x.\mathbf{1}]\ a\big) \to \mathbb{N}^{W^*}\Big) \to \mathbb{N}^{W^*}$$

There are two cases, either $a = (\mathtt{inl}\ *)$ or $a = (\mathtt{inr}\ *)$.

- If $a = (\mathtt{inl}\ *)$, we have $(t : \mathbf{0} \to \mathbb{N}^{W^*})$. This t is thus empty. In this base case, we want to return 0^{W^*}.
- If $a = (\mathtt{inr}\ *)$, we have $(t : \mathbf{1} \to \mathbb{N}^{W^*})$. In other words, we have $\big((t\ \star) : \mathbb{N}^{W^*}\big)$ and can obtain a value of $\mathbb{N}^{W^*}$. This is the recursive call. We want to map this value to its double. In other words, we map $(t\ \star)$ to $\mathtt{sup}^*\ (\mathtt{inr}\ *)\ \big(\lambda b.\mathtt{sup}^*\ (\mathtt{inr}\ *)\ (\lambda b.t\ \star)\big)$.

All together, we have the following definition of g:

$$\begin{aligned}
g := \lambda a.[\ & \\
& \lambda(s : \star).\lambda(t : \mathbf{0} \to \mathbb{N}^{W^*}).0^{W^*}, \\
& \lambda(s : \star).\lambda(t : \mathbf{1} \to \mathbb{N}^{W^*}).\mathtt{sup}^*\ (\mathtt{inr}\ *)\ (\lambda b.\mathtt{sup}^*\ (\mathtt{inr}\ *)\ (\lambda b.(t\ \star)) \\
]\ a &
\end{aligned}$$

Let us finish by computing the value of $(\mathtt{doubleW}\ 1^{W^*})$.

$$
\begin{aligned}
\texttt{doubleW}\ 1^{W^*} &:= \texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g\ 1^{W^*} \\
&:= \texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g\ \left(\texttt{sup}^*\ (\texttt{inr}\ *)\ (\lambda b.0^{W^*})\right) \\
&\overset{\beta}{=} g\ (\texttt{inr}\ *)\ \left((\texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g) \circ (\lambda b.0^{W^*})\right) \\
&= g\ (\texttt{inr}\ *)\ \left(\lambda(b:\star).\texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g\ ((\lambda b.0^{W^*})\ b)\right) \\
&\overset{\beta}{=} g\ (\texttt{inr}\ *)\ \left(\lambda b.\texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g\ 0^{W^*}\right) \\
&\overset{\beta}{=} \texttt{sup}^*\ (\texttt{inr}\ *)\ \left(\lambda b.\texttt{sup}^*\ (\texttt{inr}\ *)\ \left(\lambda b.(\lambda b.\texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g\ 0^{W^*})\ \star\right)\right) \\
&\overset{\beta}{=} \texttt{sup}^*\ (\texttt{inr}\ *)\ \left(\lambda b.\texttt{sup}^*\ (\texttt{inr}\ *)\ \left(\lambda b.\texttt{rec}^*_{\texttt{W}}\ \mathbb{N}^{W^*}\ g\ 0^{W^*}\right)\right) \\
&\overset{\beta}{=} \texttt{sup}^*\ (\texttt{inr}\ *)\ \left((\lambda b.\texttt{sup}^*\ (\texttt{inr}\ *)\ (\lambda b.0^{W^*})\right) \\
&=: 2^{W^*}
\end{aligned}
$$

In this section, we have seen that we can create W-types using an impredicative definition. Even though this definition looks quite complicated, this is mostly due to the nature of formal type-theoretic definitions rather than an inherent complexity of W-types. As mentioned in section 8.1, W-types are usually defined as the initial algebra of the $\mathcal{W}$-endofunctor. This initiality property, and in particular the uniqueness requirement of initiality, allows us to prove the induction principle of W-types. We shall again continue by taking a look at the categorical definition of W-types. We then encode the uniqueness requirement within the definition of an inductive W-type. We finish by showing that the induction principle of W-types holds for this improved definition.

8.3 Categorical W-type

In this section, we take a look at the categorical definition of W-types as the initial algebra of the $\mathcal{W}$-functor of 8.2.1. We first look at the commutative diagram of $\mathcal{W}$-morphisms.

$$
\begin{array}{ccc}
\sum(a:A).B(a) \to X & \xrightarrow{\quad g \quad} & X \\
{\scriptstyle \langle \pi_1, f\circ\pi_2\rangle}\ \Big\downarrow & = & \Big\downarrow\ {\scriptstyle f} \\
\sum(a:A).B(a) \to Y & \xrightarrow[\quad g' \quad]{} & Y
\end{array}
$$

Figure 10: Commutative diagram of $\mathcal{W}$-morphisms.

In formulas, a function $(f : X \to Y)$ is a $\mathcal{W}$-morphism if the following holds:

$$
\begin{aligned}
f \circ g = g' \circ \langle \pi_1, f \circ \pi_2 \rangle \qquad &\iff \\
\forall (a:A), (t : B(a) \to X). \qquad f \circ g \circ \langle a, t\rangle = g' \circ \langle a, f \circ t\rangle \qquad &(8.1)
\end{aligned}
$$

As we saw before, we write out the uniqueness requirement of the initial $\mathcal{W}$-algebra. In formulas, this η-rule states that for each $\mathcal{W}$-morphism $(f : X \to Y)$, we have $f \circ u_X = u_Y$. In the next section, we encode this requirement within the definition of an inductive W-type.

$$\begin{array}{ccc}
\sum(a : A).B(a) \to I & \longrightarrow & I \\
\downarrow & = & \downarrow{\scriptstyle u_X} \\
\sum(a : A).B(a) \to X & \longrightarrow & X \\
\downarrow & = & \downarrow{\scriptstyle f} \\
\sum(a : A).B(a) \to Y & \longrightarrow & Y
\end{array}
\qquad
\begin{array}{l}
{\scriptstyle g} \\
\\
\\
{\scriptstyle g'} \\
\\
\\
{\scriptstyle u_Y}
\end{array}$$

Figure 11: Uniqueness requirement of the initial $\mathcal{W}$-algebra.

8.4 Impredicative encoding of W-types

In this section we give an impredicative encoding of the W-type that we defined in section 8.2. To start, we give a predicate `MorphW` that states that a function f is a $\mathcal{W}$-morphism.

Definition 8.4.1. Let $(X, Y : \mathcal{U})$, $\big(g : \prod(a : A).(B(a) \to X) \to X\big)$ and $\big(g' : \prod(a : A).(B(a) \to Y) \to Y\big)$. We define a predicate `MorphW` that states that $(f : X \to Y)$ forms a $\mathcal{W}$-morphism based on Equation (8.1).

$$\texttt{MorphW}\ X\ g\quad Y\ g'\quad f := \prod(a : A).\prod(t : B(a) \to X).f\ (g\ a\ t) = g'\ a\ (f \circ t)$$

We are now ready to define an inductive W-type. This is an embedding of the previously defined impredicative definition 8.2.2. We thus have that $W(a : A).B(a) \hookrightarrow W^*(a : A).B(a)$. We use a Σ-type to encode a proof term of the uniqueness requirement of Figure 11.

Definition 8.4.2. We define an **inductive W-type**. Let $(A : \mathcal{U})$ be a label type and let $(B : A \to \mathcal{U})$ be the type of arities.

$$\begin{aligned}
W(a : A).B(a) &:= \sum\big(w : W^*(a : A).B(a)\big).\texttt{LimW}\ w && \text{(formation)} \\
\texttt{LimW}\ w &:= \prod(X, Y : \mathcal{U}) \\
&\quad \prod\Big(g : \big(\prod(a : A).B(a) \to X\big) \to X\Big) \\
&\quad \prod\Big(g' : \big(\prod(a : A).B(a) \to Y\big) \to Y\Big). \\
&\quad \prod(f : X \to Y). \\
&\quad (\texttt{MorphW}\ X\ g\ Y\ g'\ f) \implies f\ (\texttt{rec}^*_{\texttt{W}}\ X\ g\ w) = \texttt{rec}^*_{\texttt{W}}\ Y\ g'\ w
\end{aligned}$$

We can create a recursor for this new W-type.

$$\mathtt{rec_W} : \prod(X : \mathcal{U}).\Big(\prod(a : A).(B(a) \to X) \to X\Big) \to W(a : A).B(a) \to X \qquad \text{(elimination)}$$

$\mathtt{rec_W}\ X\ g\ w := \mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{pr1}\ w)$

We have to show that elements of the old W-type satisfy the `LimW` predicate. Because we have just one introduction rule, namely $\mathtt{sup}^*$, we just have to check that `LimW` holds for this constructor.

Lemma 8.4.3 (`LimWSup`)**.** For $(a : A)$ and $\big(r : B(a) \to W_{AB}\big)$ we have that the $\mathtt{sup}^*$ constructor satisfies the `LimW` predicate.

$$\mathtt{LimW}\ (\mathtt{sup}^*\ a\ (\mathtt{pr1}\ \circ r))$$

Proof. Let $(X, Y : \mathcal{U})$, $(g : \prod(a : A).(B(a) \to X) \to X)$, $(g' : \prod(a : A).(B(a) \to Y) \to Y)$ and $(f : X \to Y)$ be given. Suppose $(\mathtt{MorphW}\ X\ g\ Y\ g'\ f)$ holds. We need to show that $f\ (\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ \mathtt{sup}^*(a, r)) = \mathtt{rec}^*_{\mathtt{W}}\ Y\ g'\ \mathtt{sup}^*(a, r)$ holds. Note that by instantiating
$(\mathtt{MorphW}\ X\ g\ Y\ g'\ f)$ with a and $(g\ a\ (\mathtt{rec}^*_{\mathtt{W}}(X, g) \circ r))$ we obtain the following equation:

$$f\ \big(g\ a\ ((\mathtt{rec}^*_{\mathtt{W}}\ X\ g) \circ r)\big) = g'\ a\ (f \circ (\mathtt{rec}^*_{\mathtt{W}}\ X\ g) \circ r) \tag{8.2}$$

For all $(a : A)$ and $(b : B(a))$ we have that $(r\ b)$ is of the type W_{AB}. Thus, we can use the proof obtained from $(\mathtt{pr2}\ (r\ b))$. In other words, we have that $(\mathtt{LimW}\ (\mathtt{pr1}\ (r\ b)))$ holds. We instantiate this with the variables X, Y, g, g' and f to obtain the following equation:

$$\prod(a : A).\prod(b : B(a)).f\ \big(\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{pr1}\ (r\ b))\big) = \mathtt{rec}^*_{\mathtt{W}}\ Y\ g'\ (\mathtt{pr1}\ (r\ b)) \iff f \circ (\mathtt{rec}^*_{\mathtt{W}}\ X\ g) \circ (\mathtt{pr1}\ \circ r) = (\mathtt{rec}^*_{\mathtt{W}}\ Y\ g') \circ (\mathtt{pr1}\ \circ r) \tag{8.3}$$

We conclude:

$$\begin{aligned} f\ \big(\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{sup}^*\ a\ (\mathtt{pr1}\ \circ r))\big) &\overset{\beta}{=} f\ \big(g\ a\ ((\mathtt{rec}^*_{\mathtt{W}}\ X\ g) \circ (\mathtt{pr1}\ \circ r))\big) \\ &\overset{(8.2)}{=} g'\ a\ (f \circ (\mathtt{rec}^*_{\mathtt{W}}\ X\ g)) \circ (\mathtt{pr1}\ \circ r)) \\ &\overset{(8.3)}{=} g'\ a\ (\mathtt{rec}^*_{\mathtt{W}}\ Y\ g') \circ (\mathtt{pr1}\ \circ r)) \\ &=: \mathtt{rec}^*_{\mathtt{W}}\ Y\ g'\ (\mathtt{sup}^*\ a\ (\mathtt{pr1}\ \circ r)) \end{aligned}$$ □

Definition 8.4.4. We define the introduction rule for the new inductive W-type.

$$\mathtt{sup} : \prod(a : A).\prod(r : B(a) \to W_{AB}) \to W_{AB} \qquad \text{(introduction)}$$
$$\mathtt{sup}\ a\ r := \langle \mathtt{sup}^*\ a\ (\mathtt{pr1}\ \circ r),\ \mathtt{LimWSup}\ a\ (\mathtt{pr1}\ \circ r)\rangle$$

Lemma 8.4.5. The β-rule hold for $W(a : A).B(a)$. Let $(X : \mathcal{U})$, $(a : A)$ and $(r : B(a) \to W_{AB})$ be given. Suppose $\left(g : \prod(a : A).(B(a) \to X) \to X\right)$. We have the following equation:

$$\mathtt{rec_W}\ X\ g\ (\mathtt{sup}\ a\ r) \overset{\beta}{=} g\ a\ ((\mathtt{rec_W}\ X\ g) \circ r) \qquad \text{(computation)}$$

Proof.

$$\begin{aligned}
\mathtt{rec_W}\ X\ g\ (\mathtt{sup}\ a\ r) &:= \mathtt{rec}^*_\mathtt{W}\ X\ g\ (\mathtt{pr1}(\mathtt{sup}\ a\ r)) \\
&:= \mathtt{rec}^*_\mathtt{W}\ X\ g\ (\mathtt{sup}^*\ a\ (\mathtt{pr1}\ \circ r)) \\
&\overset{\beta}{=} g\ a\ ((\mathtt{rec}^*_\mathtt{W}\ X\ g) \circ (\mathtt{pr1}\ \circ r)) \\
&= g\ a\ (\lambda b.\ \mathtt{rec}^*_\mathtt{W}\ X\ g\ (\mathtt{pr1}\ (r\ b))) \\
&=: g\ a\ (\lambda b.\ \mathtt{rec_W}\ X\ g\ (r\ b)) \\
&= g\ a\ ((\mathtt{rec_W}\ X\ g) \circ r)
\end{aligned}$$

□

Now that we have shown the formation, introduction, elimination and computation rules for the inductive W-type, we are ready to show that this type satisfies the uniqueness or η-rule. We start by showing that $(\mathtt{rec_W}\ X\ g)$ is a $\mathcal{W}$-morphism. Like before, these recursors shall play the role of the unique morphisms u_X of the initial object of the $\mathcal{W}$-algebra.

Lemma 8.4.6. Let $(X : \mathcal{U})$ and $\left(g : \prod(a : A).(B(a) \to X) \to X\right)$. Then we have:

$$\mathtt{MorphW}\ \ W_{AB}\ \mathtt{sup}\quad X\ g\quad (\mathtt{rec_W}\ X\ g)$$

Proof. Let $(X : \mathcal{U})$ and $(g : \prod(a : A).(B(a) \to X) \to X)$. By filling `MorphW` with the right arguments, we see that the required equation is equal to the β-rule for W_{AB} as shown in 8.4.5.

$$\mathtt{rec_W}\ X\ g\ (\mathtt{sup}\ a\ r) \overset{\beta}{=} g\ a\ ((\mathtt{rec_W}\ X\ g) \circ r) \qquad \square$$

To prove the complete η-rule, we first show a special case.

Lemma 8.4.7 (`RecWId`)**.** The following holds:

$$\mathtt{rec_W}\ W_{AB}\ \mathtt{sup} = \mathtt{id}_{W_{AB}}$$

Proof. Let $(w : W_{AB})$. We first reduce the statement using function extensionality and β-reduction. In the $(\hookrightarrow)$ step, we use the fact that $W^*(a : A).B(a)^* \hookrightarrow W_{AB}$ is an embedding.

$$\begin{aligned}
& \mathtt{rec_W}\ W_{AB}\ \mathtt{sup} = \mathtt{id}_{W_{AB}} && \overset{\text{FunExt}}{\Longleftrightarrow} \\
\forall\ (w : W_{AB}).\quad & \mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w = \mathtt{id}_{W_{AB}}\ w && \overset{\beta}{\Longleftrightarrow} \\
\forall\ (w : W_{AB}).\quad & \mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w = w && \overset{\hookrightarrow}{\Longleftrightarrow} \\
\forall\ (w : W_{AB}).\quad & \mathtt{pr1}\ \left(\mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w\right) = \mathtt{pr1}\ w && \overset{\text{FunExt}}{\Longleftrightarrow} \\
\forall_{\substack{(w:W_{AB}),(X:\mathcal{U}),\\ (g:\prod(a:A).(B(a)\to X)\to X)}}.\quad & \left(\mathtt{pr1}\ \left(\mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w\right)\right)\ X\ g = (\mathtt{pr1}\ w)\ X\ g &&
\end{aligned} \tag{8.4}$$

By 8.4.6 we known that $\big((\mathtt{rec_W}\ X\ g) : W_{AB} \to X\big)$ is a $\mathcal{W}$-morphism. By looking at $(p := \mathtt{pr2}\ w)$, which has type $\mathtt{LimW}(\mathtt{pr1}\ w)$, we obtain the following equation:

$$(\mathtt{rec_W}\ X\ g)\ (\mathtt{rec}^*_{\mathtt{W}}\ W_{AB}\ \mathtt{sup}\ (\mathtt{pr1}\ w)) = (\mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{pr1}\ w)) \tag{8.5}$$

We conclude the proof by showing Equation (8.4). Let $(w : W_{AB}), (X : \mathcal{U})$, and $\big(g : \prod(a : A).(B(a) \to X) \to X\big)$.

$$\begin{aligned}
\Big(\mathtt{pr1}\ \big(\mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w\big)\Big)\ X\ g &=: \mathtt{rec}^*_{\mathtt{W}}\ X\ g\ \Big(\mathtt{pr1}\ \big(\mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w\big)\Big) \\
&=: \mathtt{rec_W}\ X\ g\ \big(\mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w\big) \\
&=: \mathtt{rec_W}\ X\ g\ \big(\mathtt{rec}^*_{\mathtt{W}}\ W_{AB}\ \mathtt{sup}\ (\mathtt{pr1}\ w)\big) \\
&\overset{(8.5)}{=} \mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{pr1}\ w) \\
&= (\mathtt{pr1}\ w)\ X\ g
\end{aligned}$$

□

We are now ready to show the η-rule for the inductive W-type. This rule states that the recursors $(\mathtt{rec_W}\ X\ g)$ are unique.

Theorem 8.4.8. The η-rule holds. Let $(X : \mathcal{U})$, $\big(g : \prod(a : A).(B(a) \to X) \to X\big)$ and $(f : W_{AB} \to X)$ be given. We have the following equation:

$$\mathtt{MorphW}(W_{AB},\ \mathtt{sup},\quad X,\ g,\quad f) \implies f = \mathtt{rec_W}\ X\ g$$

Proof. Let $(X : \mathcal{U})$, $\big(g : \prod(a : A).(B(a) \to X) \to X\big)$ and $(f : W_{AB} \to X)$. Suppose $\mathtt{MorphW}(W_{AB},\ \mathtt{sup},\ X,\ g,\ f)$ holds. Fix $(w : W_{AB})$. By looking at $(p := \mathtt{pr2}\ w)$, which has type $\mathtt{LimW}(\mathtt{pr1}\ w)$, we obtain the following equation:

$$f\ (\mathtt{rec}^*_{\mathtt{W}}\ W_{AB}\ \mathtt{sup}\ (\mathtt{pr1}\ w)) = \mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{pr1}\ w) \tag{8.6}$$

We derive the following:

$$\begin{aligned}
f\ w &\overset{\mathtt{RecWId}}{=} f\ \big(\mathtt{rec_W}\ W_{AB}\ \mathtt{sup}\ w\big) \\
&=: f\ \big(\mathtt{rec}^*_{\mathtt{W}}\ W_{AB}\ \mathtt{sup}\ (\mathtt{pr1}\ w)\big) \\
&\overset{(8.6)}{=} \mathtt{rec}^*_{\mathtt{W}}\ X\ g\ (\mathtt{pr1}\ w) \\
&=: \mathtt{rec_W}\ X\ g\ w
\end{aligned}$$

By function extensionality we conclude that $(f = \mathtt{rec_W}\ X\ g)$. □

8.5 W-induction principle

In this section, we show that we have an induction principle on our impredicative encoding. We follow the same approach as in the proof for list induction in 4.4.4. Before we begin, we first take a moment to investigate what an induction principle for W-types would look like.

Induction is used to prove properties about *all* members of some type. We thus assume some predicate $(P : W_{AB} \to \mathcal{U})$ that we want to show. The end result of the induction principle is thus a proof that this predicate holds for all members: $\left(\prod(w : W_{AB}).P\ w\right)$. What requirements do we need to satisfy in order to prove this? We know that W-types are constructed using just one constructor: $(\mathtt{sup}\ a\ r)$. The induction hypothesis is therefore as follows:

Definition 8.5.1. Induction **hypothesis for W-induction**. Let $(a : A)$, and $(r : (B\ a) \to W_{AB})$.

$$\mathtt{Hyp}_W\ a\ r := \prod\left(h : \prod(b : B\ a).P\ (r\ b)\right) \implies P\ (\mathtt{sup}\ a\ r)$$

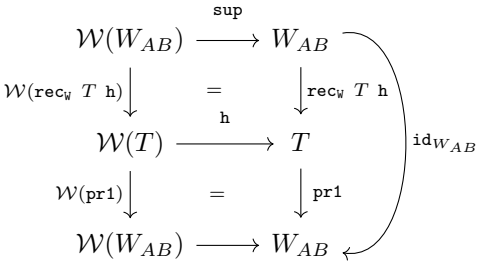
The combined induction principle for W-types is quite straightforward:

Definition 8.5.2. We define the **induction principle of W-types**.

$$\mathtt{Ind}_W := \left(\prod(a : A).\prod(r : (B\ a) \to W_{AB}).\ \mathtt{Hyp}_W\ a\ r\right) \implies \prod(w : W_{AB}).P\ w$$

To show that this induction principle holds, we make use of the η-rule of 8.4.8. We first formulate our plan of attack. Note the similarity to what we have seen in 4.4.4.

1. We show that $(T := \sum(w : W_{AB}).P\ w)$ together with some function $\mathtt{h}$ forms a $\mathcal{W}$-algebra.
2. We have that $\mathtt{pr1}$ is a $\mathcal{W}$-morphism $(\mathtt{pr1} : T \to W_{AB})$.
3. We have the unique $\mathcal{W}$-morphism $(\mathtt{rec_W}\ T\ \mathtt{h} : W_{AB} \to T)$.
4. We have that $(\mathtt{pr1} \circ (\mathtt{rec_W}\ T\ \mathtt{h}))$ is a $\mathcal{W}$-morphsim $\left(\mathtt{pr1} \circ (\mathtt{rec_W}\ T\ \mathtt{h}) : W_{AB} \to W_{AB}\right)$.
5. We have that $\mathtt{id_{List}}$ is also an $\mathcal{L}$-morphism from $\mathtt{List}$ to $\mathtt{List}$.
6. By the η-rule we conclude that that $\mathtt{pr1} \circ (\mathtt{rec_W}\ T\ \mathtt{h}) = \mathtt{id}_{W_{AB}}$.
7. For all $(w : W_{AB})$ we have that $\mathtt{pr1}\ (\mathtt{rec_W}\ T\ \mathtt{h}\ w) = w$ and thus conclude that $(\mathtt{pr2}\ (\mathtt{rec_W}\ T\ \mathtt{h}\ w) : P\ w)$.



Lemma 8.5.3. Assume a predicate $(P : W_{AB} \to \mathcal{U})$. We assume the W-induction hypothesis of 8.5.1 holds. Then, e have that $(T := \sum(w : W_{AB}).P\ w)$ together with some function $\mathtt{h}$ forms a $\mathcal{W}$-algebra.

Proof. We define $(T := \sum(w : W_{AB}).P\ w)$. To make this a W-algebra, we need a function $(\mathtt{h} : \left(\sum(a : A).B(a) \to T\right) \to T)$. We define $\mathtt{h}$ as follows:

$$\texttt{h} : \Big(\sum(a : A).B(a) \to \sum(w : W_{AB}).P\ w\Big) \to \sum(w : W_{AB}).P\ w$$
$$\texttt{h} := \lambda\langle a, r\rangle.\langle \texttt{sup}\ a\ (\texttt{pr1}\ \circ r),\ H\ a\ (\texttt{pr1}\ \circ r)\ (\texttt{pr2}\ \circ r)\rangle$$

Let's take a moment to decipher this definition. First, we destruct the left-most sigma type. This yields a tuple $\langle a, r\rangle$ where $(a : A)$ and $(r : B(a) \to \sum(w : W_{AB}).P\ w)$. Next, we need to create a value of type $(\sum(w : W_{AB}).P\ w)$. We thus need to create a tuple. The left element needs to be of type W_{AB}. We use `sup` to construct the required W-type. Note that we need to take the first projection of r.
For the right element of the tuple, we need a proof of type $(P\ w)$. Because we are working with a dependent sum type, this w is the value we just gave for the left element of the tuple. We thus need to give a proof of $(P\ (\texttt{sup}\ a\ (\texttt{pr1}\ \circ r)))$. We use the induction hypothesis. We first initialize H with the right types:

$$(H\ a\ (\texttt{pr1}\ \circ r)) : \prod\Big(k : \prod(b : B\ a).P\ ((\texttt{pr1}\ \circ r)\ b)\Big) \implies P\ (\texttt{sup}\ a\ (\texttt{pr1}\ \circ r))$$

If we find a suitable value for k, the required result follows directly. We are thus left with finding a suitable value of k.

$$k : \prod(b : B\ a).P\ ((\texttt{pr1}\ \circ r)\ b)$$

Assume $(b : B\ a)$. Note that $\big((r\ b) : \sum(w : W_{AB}).P\ w\big)$, where $\big((\texttt{pr1}\ (r\ b)) : W_{AB}\big)$ and therefore $\big((\texttt{pr2}\ (r\ b)) : P\ (\texttt{pr1}\ (r\ b))\big)$. We thus see that $(\texttt{pr2}\ \circ r)$ gives the desired value for k. □

Lemma 8.5.4. We have that $(\texttt{pr1} : T \to W_{AB})$ is a $\mathcal{W}$-morphism. Formally, the following holds:

$$\big(\texttt{MorphW}\ T\ \texttt{h}\quad W_{AB}\ \texttt{sup}\quad \texttt{pr1}\big)$$

Proof. Let $(a : A)$ and $(t : (B\ a) \to T)$. We have to show that $\texttt{pr1}\ (\texttt{h}\ a\ t) = \texttt{sup}\ a\ (\texttt{pr1}\ \circ t)$. This follows directly by looking at (the first projection of) the definition of $(\texttt{h}\ a\ t)$. □

Theorem 8.5.5. Induction holds for W_{AB}.
Suppose $(P : W_{AB} \to \mathcal{U})$ is a predicate on W_{AB}. Suppose we have for all $(a : A)$ and $\big(r : (B\ a) \to W_{AB}\big)$ that $(\texttt{Hyp}_W\ a\ r)$ holds, then we conclude $\prod\big(w : W_{AB}\big).P\ w$.

Proof. Let us assume a predicate $(P : W_{AB} \to \mathcal{U})$. We assume that the induction hypothesis $(\texttt{Hyp}_W\ a\ r)$ holds for all $(a : A)$ and $\big(r : (B\ a) \to W_{AB}\big)$. This means that we have a proof $(H\ a\ r : (\texttt{Hyp}_W\ a\ r))$ for all $(a : A)$ and $\big(r : (B\ a) \to W_{AB}\big)$.

1. By 8.5.3 we have that $(T := \sum(w : W_{AB}).P\ w)$ forms a $\mathcal{W}$-algebra.
2. By 8.5.4 we have that $(\texttt{pr1} : T \to W_{AB})$ formas a $\mathcal{W}$-morphism.

3. By 8.4.6 have a $\mathcal{W}$-morphism $((\mathtt{rec_W}\ T\ \mathtt{h}) : W_{AB} \to T)$, which is unique by the η-rule of 8.4.8.

4. By a straightforward adaptation of 2.2.4 we have that $\big((\mathtt{pr1} \circ (\mathtt{rec_W}\ T\ \mathtt{h})) : W_{AB} \to W_{AB}\big)$ is a $\mathcal{W}$-morphism.

5. By a straightforward adaptation of 2.2.3 we have that $\mathtt{id_{List}}$ is a $\mathcal{W}$-morphism.

6. By 8.4.8 we have that $(\mathtt{pr1} \circ (\mathtt{rec_W}\ T\ \mathtt{h})) = (\mathtt{rec_W}\ W_{AB}\ \text{sup})$. By 8.4.7 we have that $(\mathtt{rec_W}\ W_{AB}\ \text{sup}) = \mathtt{id}_{W_{AB}}$. We combine these to obtain that $\mathtt{pr1}\ (\mathtt{rec_W}\ T\ \mathtt{h}) = \mathtt{id_{List}}$.

7. Let $(w : W_{AB})$ be any member of W_{AB}. We want to show that $(P\ w)$ holds. We just saw that $(\mathtt{pr1} \circ (\mathtt{rec_W}\ T\ \mathtt{h}))\ w = w$. We can thus obtain a proof of $(P\ l)$ by looking at the second projection $\big((\mathtt{pr2}\ (\mathtt{rec_W}\ T\ \mathtt{h}))\ w : P\ w\big)$. $\square$

9 M-types

In this chapter, we introduce M-types. M-types are the dual notion of W-types and generalize coinductive types such as streams, possibly infinite lists, and non-wellfounded trees. In this chapter, we give an impredicative definition of M-types. This definition yields the expected β-rules just like we saw for $\texttt{Stream}^*$ in chapter 6. We again create an impredicative encoding that ensures that the η-rule also holds. We finish by using this η-rule to prove the coinduction principle (also called bisimulation) for M-types.

9.1 Introduction to M-types

Before we give a formal definition, we first sketch the idea behind M-types. Coinductive data types are structures that are possibly infinite. Take for example the type of streams. These streams are infinite in the sense that one can take the tail of the stream any amount of times without depleting the stream. It is also possible to create streams that might terminate at some point. They are often called possibly infinite lists. Inhabitants of this type might be infinite or might terminate at some point. Another example are non-wellfounded trees, in other words, trees that might go on forever.
Just like a W-type has a number of constructors, an M-type has a number of ways it can destruct. We saw that streams have two destructors: ($\texttt{hd} : \texttt{Stream} \to E$) and ($\texttt{tl} : \texttt{Stream} \to \texttt{Stream}$). We can also combine these into a new destructor: ($\texttt{destr} : \texttt{Stream} \to (E \times \texttt{Stream})$) which simultaneously takes the first element of the stream and returns the tail of the stream. Note that we can define `hd` and `tl` in terms of `destr` by taking the first and second projections respectively. Just like we could describe `cons` as a family of constructors (one for each $(e : E)$), we can also describe `destr` as a family of destructors $\texttt{destr}_E$. For each particular $(e : E)$, we have a destructor ($\texttt{destr}_E : \texttt{Stream} \to \texttt{Stream}$). To define M-types, we take some type $(A : \mathcal{U})$ of *labels*. Each label represents one of the destructors of the coinductive type. For each label $(a : A)$, we have a type $(B(a) : \mathcal{U})$ that describes the arity of the destructor.

Notation 9.1.1. To denote the M-type with label type $(A : \mathcal{U})$ and arity type $(B : A \to \mathcal{U})$, we write $M(a : A).B(a)$ or M_{AB}.

Example 9.1.2. We define infinite streams as an M-type. Define $A := E$ and $B := \lambda e.\mathbf{1}$.

$$\texttt{Stream}^M := M(e : E).(\lambda e.\mathbf{1})\ e = M(e : E).\mathbf{1}$$

For each M-type, we have an elimination principle `elimM` that destructs that M-type. The type definition of this elimination principle is as follows. Note that the scope of the sigma type extends all the way to the right.

$$\texttt{elimM} : \quad M_{AB} \to \sum(a : A).B(a) \to M_{AB}$$

In the case of $\texttt{Stream}^M$, we thus have the following type definition.

$$\texttt{elimM}: \quad \texttt{Stream}^M \to \sum(e : E).\mathbf{1} \to \texttt{Stream}^M$$

Because B does not depend on $(e : E)$, we equivalently have:

$$\texttt{elimM}: \quad \texttt{Stream}^M \to (E \times \texttt{Stream}^M)$$

We thus see that at least the types of the destructors of $\texttt{Stream}$ and $\texttt{Stream}^M$ coincide.

Example 9.1.3. Let us give another example where the type B does depend on $(a : A)$. We define possibly infinite lists over some element type E. This means that if we destruct the possibly infinite list, we either find that it continues, or that it terminates. We define $A := E + \mathbf{1}$ such that the list continues if it destructs to the left ($\texttt{inl}\ e$), and it terminates if it destructs to the right ($\texttt{inr}\ \star$). If the list continues, we again obtain an element of the possibly infinite list from the destructor. If it terminates, we obtain nothing. We define $(B\ (\texttt{inl}\ e)) := \mathbf{1}$ and $(B\ (\texttt{inr}\ \star)) := \mathbf{0}$. We can thus define the type of possibly infinite lists as follows:

$$\texttt{PIList}^M := M(a : E + \mathbf{1}).[\lambda e.\mathbf{1}, \lambda x.\mathbf{0}]\ a$$

We take a look at the elimination principle:

$$\texttt{elimM}: \quad \texttt{PIList}^M \to \sum(e : E).\mathbf{1} \to \texttt{PIList}^M$$

If we abuse notation, we see that there are two cases:

$$\texttt{elimM}\ (l : \texttt{PIList}^M)\ \langle e : E, - \rangle : \mathbf{1} \to \texttt{PIList}^M$$
$$\texttt{elimM}\ (l : \texttt{PIList}^M)\ \langle \star : \mathbf{1}, - \rangle : \mathbf{0} \to \texttt{PIList}^M$$

In other words, if we destruct l, we either find that it continues, or that it terminates:

$$\texttt{elimM}\ (l : \texttt{PIList}^M)\ \langle e : E, - \rangle : \texttt{PIList}^M$$
$$\texttt{elimM}\ (l : \texttt{PIList}^M)\ \langle \star : \mathbf{1}, - \rangle : \mathbf{1}$$

9.2 Impredicative M-types

In this section, we give an impredicative definition of M-types. M-types can be defined as the final coalgebra of the functor $\mathcal{M}$ below [Ott20].

Definition 9.2.1. We define the endofunctor $(\mathcal{M} : \mathcal{C} \to \mathcal{C})$. We leave the category $\mathcal{C}$ unspecified.

$$\begin{aligned}\mathcal{M}(X) &:= \sum(a : A).B(a) \to X \\ \mathcal{M}(f) &:= \langle \pi_1, f \circ \pi_2 \rangle\end{aligned}$$

To define an impredicative M-type, we instantiate Equation (7.3) using the functor $\mathcal{M}$. Just like the coinductive type of streams, we also define M-types using existential types.

$$\begin{aligned}\exists(X : \mathcal{U}).X \times (\mathcal{M}(X))^X &:= \exists(X : \mathcal{U}).X \times \left(\sum(a : A).B(a) \to X\right)^X \\ &= \exists(X : \mathcal{U}).X \times \left(X \to \sum(a : A).B(a) \to X\right)\end{aligned}$$

We are able to give an impredicative definition of M-types.

Definition 9.2.2. We define **impredicative M-types**. Let $(A : \mathcal{U})$ be a label type and $(B : A \to \mathcal{U})$ be a type describing the arities of these labels.

$$M^*(a : A).B(a) := \exists(X : \mathcal{U}).X \times \left(X \to \sum(a : A).B(a) \to X\right) \quad \text{(formation)}$$

The following corecursor serves as our introduction rule.

$$\texttt{corec}^*_{\texttt{M}} : \prod(X : \mathcal{U}).\left(X \to \sum(a : A).B(a) \to X\right) \to X \to M^*_{AB} \quad \text{(introduction)}$$

$$\texttt{corec}^*_{\texttt{M}}\ X\ r\ x := \texttt{pack}\ X\ \langle x, r \rangle$$

As we outlined before, we need a destructor, or elimination principle. Because we are working with existential types, we need to use the $\texttt{rec}_\exists$ recursor of the existential type to operate on the contents of the M-type.

Definition 9.2.3. For each M-type M_{AB}, we define the **M-eliminator** as follows:

$$\texttt{elimM}^* : M^*_{AB} \to \sum(a : A).B(a) \to M^*_{AB} \quad \text{(elimination)}$$

$$\begin{aligned}\texttt{elimM}^*\ m := \texttt{rec}_\exists\ &\left(\sum(a : A).B(a) \to M^*_{AB}\right) \\ &\Big(\lambda X.\lambda\langle x, f\rangle.\ \langle \texttt{pr1}\ (f\ x),\ \lambda b.\texttt{pack}\ X\ \langle \texttt{pr2}\ (f\ x)\ b, f\rangle\ \rangle\ \Big)\end{aligned}$$

Let us dissect this definition. We want to create a function that takes an M-type $(m : M^*_{AB})$ and produces a value of type $\left(\sum(a : A).B(a) \to M^*_{AB}\right)$. We use the existential recursor $\texttt{rec}_\exists$ with output type $(\sum(a : A).B(a) \to M^*_{AB})$. We can thus create a function where we get some type $(X : \mathcal{U})$ and a tuple $\langle x, f\rangle$, where $(x : X)$ and $\left(f : X \to \sum(a : A).B(a) \to X\right)$ and return some value $(\langle l, r\rangle : \sum(a : A).B(a) \to M^*_{AB})$:

$$\texttt{elimM}^*\ m := \texttt{rec}_\exists\ \left(\sum(a:A).B(a) \to M^*_{AB}\right)$$
$$\left(\lambda X.\lambda\langle x, f\rangle.\ \langle \underbrace{\texttt{pr1}\ (f\ x)}_{l\ :\ A},\ \underbrace{\lambda b.\texttt{pack}\ X\ \langle(\ \overbrace{\texttt{pr2}\ (f\ x)}^{:\ B(a)\to W_{AB}}\ )\ b, f\rangle}_{r\ :\ B(a)\to M^*_{AB}}\ \rangle\ \right)$$

For the left-hand side l, we extract the value of 'a' from the first projection of $(f\ x)$. To create a value for the right-hand side r, we need to create a function that takes a value of type $(b : B(a))$ and yields a value of type M^*_{AB}. To this effect, we use the right-hand projection of $(f\ x)$ and apply b.

Now that we have both an introduction and an elimination principle, we can check if the computation rule or β-rule holds.

Lemma 9.2.4. The β-rule holds for M^*_{AB}. Let $(X : \mathcal{U})$, $(r : X \to \sum(a : A).B(a) \to X)$ and $(x : X)$. We have that the following equation holds:

$$\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ r\ x) \overset{\beta}{=} \langle \texttt{pr1}\ (r\ x),\ (\texttt{corec}^*_{\texttt{M}}\ X\ r) \circ (\texttt{pr2}\ (r\ x))\rangle \qquad \text{(computation)}$$

Proof. Let X, r, and x be as described. The desired result follows by folding and unfolding the definitions and using 6.1.4.

$$\begin{aligned}
\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ r\ x) &:= \texttt{elimM}^*\ (\texttt{pack}\ X\ \langle x, r\rangle)\\
&\overset{\texttt{ExistsId}}{=} \langle \texttt{pr1}\ (r\ x), \lambda b.\texttt{pack}\ X\ \langle \texttt{pr2}\ (r\ x)\ b, r\rangle\rangle\\
&=: \langle \texttt{pr1}\ (r\ x), \lambda b.\texttt{corec}^*_{\texttt{M}}\ X\ r\ (\texttt{pr2}\ (r\ x)\ b)\rangle\\
&=: \langle \texttt{pr1}\ (r\ x),\ (\texttt{corec}^*_{\texttt{M}}\ X\ r) \circ (\texttt{pr2}\ (r\ x))\rangle \qquad \square
\end{aligned}$$

Now that we have an impredicative definition of M-types, we can follow the approach outlined in chapter 7 to create an impredicative encoding. We first take a look at the categorical definitions.

9.3 Categorical M-type

In this section, we take a look at the categorical definition of M-types as the final coalgebra of the $\mathcal{M}$-functor of 9.2.1. We first look at the commutative diagram of $\mathcal{M}$-morphisms.
In formulas, a function $(f : X \to Y)$ is an $\mathcal{M}$-morphism, if the following holds:

$$\begin{aligned}
& g' \circ f = \langle \pi_1, f \circ \pi_2\rangle \circ g \iff\\
\forall(x : X).\qquad & g'\ (f\ x) = \langle \pi_1(g\ x), f \circ (\pi_2\ (g\ x))\rangle \iff\\
\forall(x : X).\qquad & \pi_1(g'\ (f\ x)) = \pi_1(g\ x) \quad \wedge \quad \pi_2(g'\ (f\ x)) = f \circ (\pi_2\ (g\ x)) \qquad (9.1)
\end{aligned}$$

We take a look at the uniqueness requirement of the final $\mathcal{M}$-coalgebra. In formulas, this η-rule states that for each $\mathcal{M}$-morphism $(f : X \to Y)$, we have $u_X = f \circ u_Y$.

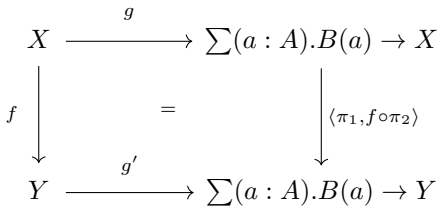


Figure 12: Commutative diagram of $\mathcal{M}$-morphisms.

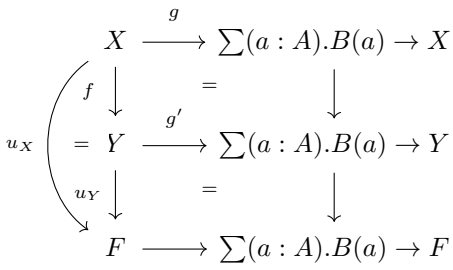


Figure 13: Uniqueness requirement of the final $\mathcal{M}$-coalgebra.

9.4 Impredicative encoding of M-types

In this section, we give an impredicative encoding of the M-type that we defined in section 9.2. We first give a predicate `MorphM` that states that a given function $(f : X \to Y)$ is an $\mathcal{M}$-morphism.

Definition 9.4.1. Let $(X, Y : \mathcal{U})$, $(g : X \to \sum(a : A).B(a) \to X)$, $(g' : Y \to \sum(a : A).B(a) \to Y)$ and $(f : X \to Y)$. We define a predicate `MorphM` that states that $(f : X \to Y)$ forms an $\mathcal{M}$-morphism based on Equation (9.1).

$$\begin{aligned}\texttt{MorphM}\ X\ g\quad Y\ g'\quad f :\equiv \prod(x : X).\quad & \texttt{pr1}\ (g'\ (f\ x)) = \texttt{pr1}\ (g\ x)\quad \wedge \\ & \texttt{pr2}\ (g'\ (f\ x)) = f \circ (\texttt{pr2}\ (g\ x))\end{aligned}$$

As we saw with $\texttt{Stream}^*$, we shall define a colimit relation `CoLimM` that relates two M-types if they are related by an $\mathcal{M}$-morphism. We then create a quotient using this relation such that two M-types that are related by `CoLimM` are equated. We finally prove that this quotient satisfies the η-rule.

Definition 9.4.2. We define the relation `CoLimM`. Let $(m, n : M_{AB})$. The predicate $(\texttt{CoLimM}\ m\ n)$ holds if m and n are related by some $\mathcal{M}$-morphism.

$$\begin{aligned}\texttt{CoLimM}\ m\ n :\equiv \exists(X, Y : \mathcal{U}).\exists^{g : X \to \sum(a:A).B(a) \to X}_{g' : Y \to \sum(a:A).B(a) \to Y}.\exists(f : X \to Y).\exists(x : X).& \\ (\texttt{MorphM}\ X\ g\quad Y\ g'\quad f) \wedge & \\ m = \texttt{corec}^*_{\texttt{M}}\ X\ g\ x \wedge & \\ n = \texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ x) &\end{aligned}$$

Notation 9.4.3. We use the infix notation $(m \equiv n)$ to denote that $(\texttt{CoLimM}\ m\ n)$ holds.

Even though this relation is not an equivalence relation (see note 5.1.8), we shall speak about the equivalence classes in the sense of the elements that are equal when taking the quotient `quot`.

Definition 9.4.4. We define the **coinductive M-type** M_{AB}.

$$M(a : A).B(a) := \texttt{quot}\ (M^*(a : A).B(a))\ \texttt{CoLimM} \qquad \text{(formation)}$$

We define the corecursor as follows:

$$\texttt{corec}_{\texttt{M}} : \prod(X : \mathcal{U}).\big(X \to \sum(a : A).B(a) \to X\big) \to X \to M_{AB} \qquad \text{(introduction)}$$

$$\texttt{corec}_{\texttt{M}}\ X\ r\ x := \texttt{cls}\ (\texttt{corec}^*_{\texttt{M}}\ X\ r\ x)$$

In order to lift the $\texttt{elimM}^*$ function, we need to show that it is constant under the equivalence classes of `CoLimM`. We show this in two parts: first, we show that the left projection is constant under equivalence and then we show that the right projection is constant under `cls`.

Lemma 9.4.5 ($\texttt{ElimEq1} : (\texttt{EqCls}\ (\texttt{pr1} \circ \texttt{elimM}^*)\ \equiv)$)**.** The left projection of the M-eliminator $\texttt{elimM}^*$ is constant on all equivalence classes of `CoLimM`.

$$\prod(m, n : M_{AB}).m \equiv n \implies (\texttt{pr1}\ (\texttt{elimM}^*\ m)) = (\texttt{pr1}\ (\texttt{elimM}^*\ n))$$

Proof. Let $(m, n : M_{AB})$ be M-types. Suppose that $(\texttt{CoLimM}\ m\ n)$ holds. We thus have some $(X, Y : \mathcal{U})$, $(g : X \to \sum(a : A).B(a) \to X)$, $(g' : Y \to \sum(a : A).B(a) \to Y)$, $(x : X)$ and $(f : X \to Y)$ such that the following hold:

$$\texttt{pr1}\ (g'\ (f\ x)) = \texttt{pr1}\ (g\ x) \tag{9.2}$$
$$\texttt{pr2}\ (g'\ (f\ x)) = f \circ (\texttt{pr2}\ (g\ x)) \tag{9.3}$$
$$m = \texttt{corec}^*_{\texttt{M}}\ X\ g\ x \tag{9.4}$$
$$n = \texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ x) \tag{9.5}$$

We derive the following:

$$\begin{aligned}
\texttt{pr1}\ (\texttt{elimM}^*\ m) &\overset{9.4}{=} \texttt{pr1}\ (\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ g\ x)) \\
&\overset{\beta}{=} \texttt{pr1}\ \big\langle \texttt{pr1}\ (g\ x),\ \lambda b.\texttt{corec}^*_{\texttt{M}}\ X\ g\ (\texttt{pr2}\ (g\ x)\ b) \big\rangle \\
&\overset{\beta}{=} \texttt{pr1}\ (g\ x) \\
&\overset{9.2}{=} \texttt{pr1}\ (g'\ (f\ x)) \\
&\overset{\beta}{=} \texttt{pr1}\ \big\langle \texttt{pr1}\ (g'\ (f\ x)),\ \lambda b.\texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (\texttt{pr2}\ (g'\ (f\ x))\ b) \big\rangle \\
&\overset{\beta}{=} \texttt{pr1}\ (\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ x))) \\
&\overset{9.5}{=} \texttt{pr1}\ (\texttt{elimM}^*\ n)
\end{aligned}$$

□

Lemma 9.4.6 ($\texttt{ElimEq2} : (\texttt{EqCls}\ (\texttt{cls} \circ \texttt{pr2} \circ \texttt{elimM}^*)\ \equiv)$)**.** The right projection of the M-eliminator $\texttt{elimM}^*$ is constant on all equivalence classes of $\texttt{CoLimM}$.

$$\prod(m, n : M_{AB}).m \equiv n \implies (\texttt{cls}\ (\texttt{pr2}\ (\texttt{elimM}^*\ m))) = (\texttt{cls}\ (\texttt{pr2}\ (\texttt{elimM}^*\ n)))$$

Proof. Let $(m, n : M_{AB})$ be M-types. Suppose that $(\texttt{CoLimM}\ m\ n)$ holds. We thus have some $(X, Y : \mathcal{U})$, $(g : X \to \sum(a : A).B(a) \to X)$, $(g' : Y \to \sum(a : A).B(a) \to Y)$, $(x : X)$, and $(f : X \to Y)$ such that the following hold:

$$\texttt{pr1}\ (g'\ (f\ x)) = \texttt{pr1}\ (g\ x) \tag{9.6}$$
$$\texttt{pr2}\ (g'\ (f\ x)) = f \circ (\texttt{pr2}\ (g\ x)) \tag{9.7}$$
$$m = \texttt{corec}^*_{\texttt{M}}\ X\ g\ x \tag{9.8}$$
$$n = \texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ x) \tag{9.9}$$

We derive the following. Note that we have left open a step at $(*)$.

$$\begin{aligned}
\texttt{cls}\ (\texttt{pr2}\ (\texttt{elimM}^*\ m)) &\overset{9.7}{=} \texttt{cls}\ (\texttt{pr2}\ (\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ g\ x))) \\
&\overset{9.2.4}{=} \texttt{cls}\ (\texttt{pr2}\ \langle \texttt{pr1}\ (g\ x),\ \lambda b.\texttt{corec}^*_{\texttt{M}}\ X\ g\ (\texttt{pr2}\ (g\ x)\ b)\rangle) \\
&\overset{\beta}{=} \texttt{cls}\ (\lambda b.\texttt{corec}^*_{\texttt{M}}\ X\ g\ (\texttt{pr2}\ (g\ x)\ b)) \\
&\overset{(*)}{=} \texttt{cls}\ (\lambda b.\texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ (\texttt{pr2}\ (g\ x)\ b))) \\
&\overset{9.7}{=} \texttt{cls}\ (\lambda b.\texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (\texttt{pr2}\ (g'\ (f\ x)))) \\
&\overset{\beta}{=} \texttt{cls}\ (\texttt{pr2}\ \langle \texttt{pr1}\ (g'\ (f\ x)),\ \lambda b.\texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (\texttt{pr2}\ (g'\ (f\ x))\ b)\rangle) \\
&\overset{9.2.4}{=} \texttt{cls}\ (\texttt{pr2}\ (\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ x)))) \\
&\overset{9.9}{=} \texttt{cls}\ (\texttt{pr2}\ (\texttt{elimM}^*\ n))
\end{aligned}$$

We shall now fill in the step $(*)$. Let $(b : B(a))$. It suffices to show the following:

$$\texttt{corec}^*_{\texttt{M}}\ X\ g\ (\texttt{pr2}\ (g\ x)\ b) \equiv \texttt{corec}^*_{\texttt{M}}\ Y\ g'\ (f\ (\texttt{pr2}\ (g\ x)\ b)) \tag{$*$}$$

This clearly holds since f is indeed a morphism by assumption. □

By combining 9.4.5 and 9.4.6, we can lift $\texttt{elimM}^*$.

Definition 9.4.7. We define the lifted version of $\texttt{elimM}^*$:

$$\begin{aligned}
&\texttt{elimM} : \prod(m : M_{AB}) \to M_{AB} \to \sum(a : A).B(a) \to M_{AB} \qquad \text{(elimination)} \\
&\texttt{elimM} := \overline{\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2} \rangle \circ \texttt{elimM}^*}
\end{aligned}$$

We show that this improved M-type satisfies the expected computation rule.

Proposition 9.4.8. The β-rule holds for M_{AB}. Let $(X : \mathcal{U})$, $(r : X \to \sum(a : A).B(a) \to X)$ and $(x : X)$. We have that the following equation holds:

$$\texttt{elimM}\ (\texttt{corec}_{\texttt{M}}\ X\ r\ x) \overset{\beta}{=} \langle \texttt{pr1}\ (r\ x),\ \texttt{corec}_{\texttt{M}}\ X\ r\ (\texttt{pr2}\ (r\ x))\rangle \quad \text{(computation)}$$

Proof. We derive the statement below:

$$\begin{aligned}
\texttt{elimM}\ (\texttt{corec}_{\texttt{M}}\ X\ r\ x) &:= \overline{\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*}\ (\texttt{corec}_{\texttt{M}}\ X\ r\ x) \\
&:= \overline{\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*}\ (\texttt{cls}\ (\texttt{corec}^*_{\texttt{M}}\ X\ r\ x)) \\
&\overset{\beta\text{-quot}}{=} (\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*)\ (\texttt{corec}^*_{\texttt{M}}\ X\ r\ x) \\
&= \langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle\ (\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ r\ x)) \\
&\overset{9.2.4}{=} \langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle\ \big\langle \texttt{pr1}\ (r\ x),\ (\texttt{corec}^*_{\texttt{M}}\ X\ r) \circ (\texttt{pr2}\ (r\ x))\big\rangle \\
&\overset{\beta}{=} \big\langle \texttt{pr1}\ (r\ x),\ \texttt{cls} \circ (\texttt{corec}^*_{\texttt{M}}\ X\ r) \circ (\texttt{pr2}\ (r\ x))\big\rangle \\
&=: \big\langle \texttt{pr1}\ (r\ x),\ (\texttt{corec}_{\texttt{M}}\ X\ r) \circ (\texttt{pr2}\ (r\ x))\big\rangle
\end{aligned}$$ □

In the remainder of this section, we shall prove that the η-rule holds. We first show that $(\texttt{corec}^*_{\texttt{M}}\ X\ g)$ and $\texttt{cls}$ are $\mathcal{S}$-morphisms.

Lemma 9.4.9. Let $(X : \mathcal{U})$, and $(g : X \to \sum(a : A).B(a) \to X)$. We have that $\big((\texttt{corec}^*_{\texttt{M}}\ X\ g) : X \to M^*_{AB}\big)$ forms an $\mathcal{M}$-morphism.

$$\Big(\texttt{MorphM}\ X\ g \quad M^*_{AB}\ \texttt{elimM}^* \quad (\texttt{corec}^*_{\texttt{M}}\ X\ g)\Big)$$

Proof. Let $(x : X)$. We need to show the following equations:

$$\texttt{pr1}\ \big(\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ g\ x)\big) = \texttt{pr1}\ (g\ x) \tag{9.10}$$
$$\texttt{pr2}\ \big(\texttt{elimM}^*\ (\texttt{corec}^*_{\texttt{M}}\ X\ g\ x)\big) = (\texttt{corec}^*_{\texttt{M}}\ X\ g) \circ (\texttt{pr2}\ (g\ x)) \tag{9.11}$$

Both follow immediately from the β-rules of 9.2.4. □

Lemma 9.4.10. The class function $(\texttt{cls} : M^*_{AB} \to M_{AB})$ forms an $\mathcal{M}$-morphism.

$$\Big(\texttt{MorphM}\ M^*_{AB}\ \texttt{elimM}^* \quad M_{AB}\ \texttt{elimM} \quad \texttt{cls}\Big)$$

Proof. Let $(m : M^*_{AB})$. We need to show the following equations:

$$\texttt{pr1}\ (\texttt{elimM}\ (\texttt{cls}\ m)) = \texttt{pr1}\ (\texttt{elimM}^*\ m) \tag{9.12}$$
$$\texttt{pr2}\ (\texttt{elimM}\ (\texttt{cls}\ m)) = \texttt{cls} \circ (\texttt{pr2}\ (\texttt{elimM}^*\ m)) \tag{9.13}$$

These equations follow directly from the definition of $\texttt{elimM}$ and the β-rule of quotients from 5.3.5. We first show Equation (9.12).

$$\begin{aligned}
\texttt{pr1}\ (\texttt{elimM}\ (\texttt{cls}\ m)) &:= \texttt{pr1}\ (\overline{\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*}\ (\texttt{cls}\ m)) \\
&\overset{5.3.5}{=} \texttt{pr1}\ ((\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*)\ m) \\
&\overset{\beta}{=} \texttt{pr1}\ (\texttt{elimM}^*\ m)
\end{aligned}$$

To conclude, we show that Equation (9.13) holds.

$$\begin{aligned}\texttt{pr1}\ (\texttt{elimM}\ (\texttt{cls}\ m)) &:= \texttt{pr2}\ (\overline{\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*}\ (\texttt{cls}\ m)) \\ &\overset{5.3.5}{=} \texttt{pr2}\ ((\langle \texttt{pr1}, \texttt{cls} \circ \texttt{pr2}\rangle \circ \texttt{elimM}^*)\ m) \\ &\overset{\beta}{=} \texttt{cls} \circ (\texttt{pr2}\ (\texttt{elimM}^*\ m)) \end{aligned}$$ □

Lemma 9.4.11. Let $(m : M^*_{AB})$ be an M-type. Then we have that the following are related by `CoLimM`:

$$\left(\texttt{corec}^*_{\texttt{M}}\ M^*_{AB}\ \texttt{elimM}^*\ (\texttt{cls}\ m)\right) \equiv \left(\texttt{corec}^*_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ m\right)$$

Proof. By 9.4.10 we know that `cls` forms an $\mathcal{M}$-morphism from which it follows that `CoLimM` holds. □

Lemma 9.4.12. Let $(m : M^*_{AB})$ be an M-type. We have the following identity:

$$m \equiv (\texttt{corec}^*_{\texttt{M}}\ M^*_{AB}\ \texttt{elimM}^*\ m)$$

Proof. We destruct m to obtain $(X : \mathcal{U})$, $(x : X)$ and $(g : X \to \sum(a : A).B(a) \to X)$ such that $m = \texttt{pack}\ X\ \langle x, g\rangle =: \texttt{corec}^*_{\texttt{M}}\ X\ g\ x$ by using 6.1.4. By 9.4.9 we know that $(\texttt{corec}^*_{\texttt{M}}\ X\ g)$ forms an $\mathcal{M}$-morphism. We thus have that:

$$\begin{gathered} m = \texttt{corec}^*_{\texttt{M}}\ X\ g\ x \wedge \\ (\texttt{corec}^*_{\texttt{M}}\ M^*_{AB}\ \texttt{elimM}^*\ m) = \texttt{corec}^*_{\texttt{M}}\ M^*_{AB}\ \texttt{elimM}^*\ ((\texttt{corec}^*_{\texttt{M}}\ X\ g)\ x) \end{gathered}$$

We thus satisfied the `CoLimM` predicate which relates m to $(\texttt{corec}^*_{\texttt{M}}\ M^*_{AB}\ \texttt{elimM}^*\ m)$. □

Lemma 9.4.13. We have the following identity:

$$\texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM} = \texttt{id}_{M_{AB}}$$

Proof. It is enough to show the following equation (c.f. 6.4.13):

$$(\texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM}) \circ \texttt{cls} = \texttt{cls} \tag{9.14}$$

Let $(m : M^*_{AB})$ be an M-type. By function extensionality and the definition of $\texttt{corec}_{\texttt{M}}$ we can reduce Equation (9.14) to the following:

$$\texttt{cls}\ (\texttt{corec}^*_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ (\texttt{cls}\ m)) = \texttt{cls}\ m \tag{9.15}$$

We destruct $m = \texttt{corec}^*_{\texttt{M}}\ X\ g\ x$ for some $(X : \mathcal{U})$, $(g : X \to \sum(a : A).B(a) \to X)$ and $(x : X)$. We conclude the following.

$$\begin{aligned}\texttt{cls}\ (\texttt{corec}^*_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ (\texttt{cls}\ m)) &\overset{9.4.11}{=} \texttt{cls}\ (\texttt{corec}^*_{\texttt{M}}\ M^*_{AB}\ \texttt{elimM}^*\ m) \\ &\overset{9.4.12}{=} \texttt{cls}\ m \end{aligned}$$ □

Theorem 9.4.14. For all $(X : \mathcal{U})$, $(g : X \to \sum(a : A).B(a) \to X)$ and $(f : X \to M_{AB})$ we have:

$$\big(\texttt{MorphM}\ X\ g \quad M_{AB}\ \texttt{elimM} \quad f\big) \implies (f = \texttt{corec}_{\texttt{M}}\ X\ g) \tag{9.16}$$

Proof. Let X, g, and f be as described. Assume $\big(\texttt{MorphM}\ X\ g \quad M_{AB}\ \texttt{elimM} \quad f\big)$ holds. We show the statement by function extensionality by fixing some $(x : X)$. Note that $(\texttt{corec}_{\texttt{M}}\ X\ g\ x)$ and $(\texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ (f\ x))$ are related by `CoLimM`. This yields the following conclusion.

$$\begin{aligned} f\ x &\overset{9.4.13}{=} \texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ (f\ x) \\ &= \texttt{corec}_{\texttt{M}}\ X\ g\ x \end{aligned}$$

□

9.5 Coinduction

In this final section, we shall show that the newly defined M-types have a coinduction principle. This principle states that two M-types are equal if they have the same behavior, i.e. if they are related by a bisimulation relation. We define the bisimulation relation `BiSimM` and show that two M-types are equal if and only if they are related by this relation.

Definition 9.5.1. We define the bisimulation relation between M-types. Two M-types are bisimilar if there exists a bisimulation relation. Let $(m, n : M_{AB})$ be M-types.

$$\texttt{BiSimM}\ m\ n := \sum(R : M_{AB} \to M_{AB} \to \mathcal{U}).\texttt{IsBisimM}\ R\ \wedge R\ m\ n$$

$$\begin{aligned} \texttt{IsBisimM}\ R := \prod(m, n : M_{AB}).R\ m\ n \implies\ & \texttt{pr1}\ (\texttt{elimM}\ m) = \texttt{pr1}\ (\texttt{elimM}\ n) \wedge \\ & R\ ((\texttt{pr2}\ (\texttt{elimM}\ m))\ b)\ ((\texttt{pr2}\ (\texttt{elimM}\ n))\ b) \end{aligned}$$

Notation 9.5.2. We denote the relation `BiSimM` using the infix symbol $\sim$, so instead of writing $(\texttt{BiSimM}\ m\ n)$ we write $(m \sim n)$.

Lemma 9.5.3. The propositional equality relation on M-types is a bisimulation. So for all $(m, n : M_{AB})$ we have $(m = n) \implies (m \sim n)$. In other words, we have that the `IsBisimM` predicate holds for $=$.

$$\texttt{IsBisimM}\ (\lambda m.\lambda n.m = n) \tag{9.17}$$

Proof. Define $R := (\lambda m.\lambda n.m = n)$. Let $(m, n : M_{AB})$ and suppose $(R\ m\ n)$. From $m = n$, if follows that $(\texttt{elimM}\ m) = (\texttt{elimM}\ n)$. The statement follows trivially. □

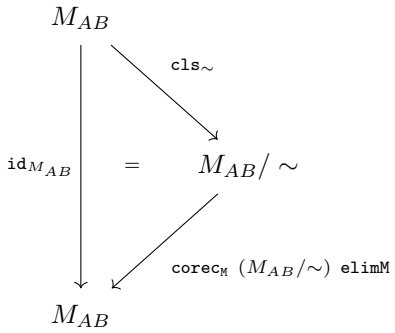
Theorem 9.5.4. We have the following coinduction proof principle `CoIndM`. Let $(m, n : M_{AB})$.

$$\texttt{CoIndM}\ m\ n := m \sim n \iff m = n$$

Proof. ($\Longleftarrow$) This follows from 9.5.3.
($\Longrightarrow$) Let $(m, n : M_{AB})$ be M-types and assume $(m \sim n)$. We define the following quotient:

$$M_{AB}/\sim := \texttt{quot}\ M_{AB}\ \texttt{BiSimM}$$

By $(m \sim n)$ there exists an bisimulation $(R : M_{AB} \to M_{AB} \to \mathcal{U})$. Let R be that bisimulation. We lift the M-destructor and define $\texttt{elimX} := \overline{\langle \texttt{pr1}, \texttt{cls}_\sim \circ \texttt{pr2} \rangle \circ \texttt{elimM}}$. This construction is well-defined by the definition of `IsBisimM` and a proof is almost identitcal to the (lengthy though not complicated) proofs of 9.4.5 and 9.4.6.
We show that the following diagram commutes:



We define $f := (\texttt{corec}_\texttt{M}\ (M_{AB}/\sim)\ \texttt{elimX}) \circ \texttt{cls}_\sim$. We have that $(f : M_{AB} \to M_{AB})$ forms an $\mathcal{M}$-morphism. To show this, we prove the following equations for $(x : X)$.

$$\texttt{pr1}\ (\texttt{elimM}\ (f\ x)) = \texttt{pr1}\ (\texttt{elimM}\ x) \tag{9.18}$$
$$\texttt{pr2}\ (\texttt{elimM}\ (f\ x)) = f \circ (\texttt{pr2}\ (\texttt{elimM}\ x)) \tag{9.19}$$

We first show Equation (9.18) using the β-rule for quotients and the β-rule for M-types.

$$\begin{aligned}
\texttt{pr1}\ (\texttt{elimM}\ (f\ x)) &:= \texttt{pr1}\ \Big(\texttt{elimM}\ \big(\texttt{corec}_\texttt{M}\ (M_{AB}/\sim)\ \texttt{elimX}\ (\texttt{cls}_\sim\ x)\big)\Big) \\
&\overset{9.4.8}{=} \texttt{pr1}\ (\texttt{elimX}\ (\texttt{cls}_\sim\ x)) \\
&\overset{5.3.5}{=} \texttt{pr1}\ \big((\langle \texttt{pr1}, \texttt{cls}_\sim \circ \texttt{pr2} \rangle \circ \texttt{elimM})\ x\big) \\
&= \texttt{pr1}\ (\texttt{elimM}\ x)
\end{aligned}$$

Next, we show Equation (9.19). Again using the β-rule for quotients and the β-rule for M-types.

$$
\begin{aligned}
\texttt{pr2}\ (\texttt{elimM}\ (f\ x)) &:= \texttt{pr2}\ \Big(\texttt{elimM}\ \big(\texttt{corec}_{\texttt{M}}\ (M_{AB}/\sim)\ \texttt{elimX}\ (\texttt{cls}_{\sim}\ x)\big)\Big) \\
&\overset{9.4.8}{=} \texttt{corec}_{\texttt{M}}\ (M_{AB}/\sim)\ \texttt{elimX}\ \big(\texttt{pr2}\ (\texttt{elimX}\ (\texttt{cls}_{\sim}\ x))\big) \\
&\overset{5.3.5}{=} \texttt{corec}_{\texttt{M}}\ (M_{AB}/\sim)\ \texttt{elimX}\ \big(\texttt{pr2}\ ((\langle \texttt{pr1}, \texttt{cls}_{\sim} \circ \texttt{pr2}\rangle \circ \texttt{elimM})\ x)\big) \\
&= \big((\texttt{corec}_{\texttt{M}}\ (M_{AB}/\sim)\ \texttt{elimX}) \circ \texttt{cls}_{\sim}\big)\ (\texttt{cls}_{\sim}\ (\texttt{pr2}\ (\texttt{elimM}\ x))) \\
&=: f \circ (\texttt{pr2}\ (\texttt{elimM}\ x))
\end{aligned}
$$

From the η-rule of M-types (9.4.14), we have that:

$$
f := (\texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimX}) \circ \texttt{cls}_{\sim} = (\texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM}) \tag{9.20}
$$

Because $(m \sim n)$ we also have that:

$$
\texttt{cls}_{\sim}\ m = \texttt{cls}_{\sim} n \tag{9.21}
$$

We derive the following:

$$
\begin{aligned}
m &\overset{9.4.13}{=} \texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ m \\
&\overset{9.20}{=} \texttt{corec}_{\texttt{M}}\ (M_{AB}/\sim)\ \texttt{elimX}\ (\texttt{cls}_{\sim}\ m) \\
&\overset{9.21}{=} \texttt{corec}_{\texttt{M}}\ (M_{AB}/\sim)\ \texttt{elimX}\ (\texttt{cls}_{\sim}\ n) \\
&\overset{9.20}{=} \texttt{corec}_{\texttt{M}}\ M_{AB}\ \texttt{elimM}\ n \\
&\overset{9.4.13}{=} n
\end{aligned}
$$

We conclude that $m = n$. □

10 Conclusion

In this thesis, we extended the technique of [AFS18] of encoding η-rules within the definition of System F style data types and applied this to lists, quotients, streams, W-types, and M-types. We showed how to define System F style impredicative data types and that each definition satisfied the expected formation, introduction, elimination, and computation rules.
For the inductive lists and W-types, we looked at the categorical definition using the initial algebra of the functors $\mathcal{L}(X) = 1 + E \times X$ and $\mathcal{W}(X) = \sum(a : A).B(a) \to X$ respectively. We distilled the uniqueness requirement of these initial objects and encoded them as a subtype using sigma types. We showed that these improved types satisfied the η-rules and proved the induction principles.
We also showed how to dualize the technique. We first created an encoding of the inductive quotient type and showed that it satisfied the η-rule and the induction principle. We then created a stream type and M-type using existential types by looking at the final coalgebra of the functors $\mathcal{S}(X) = E \times X$ and $\mathcal{M}(X) = \sum(a : A).B(a) \to X$ respectively. We distilled the uniqueness requirement of these final objects and encoded them using our quotients. We showed that these improved types satisfied the η-rules and proved the bisimulation principles.
Finally, we looked at the technique in general, closing the gap between the concrete implementations in this thesis and the abstract generic definitions in the paper [AFS18].

11 Future work

In this final chapter, we list some possible future topics one can look at.

- First, the entirety of this thesis can be formalized in a proof assistant such as Coq or Lean. Some attempts have been made to formalize parts of the paper [AFS18] by Awodey `https://github.com/awodey/Impredicative`.
- We have not looked at higher inductive types. In this case, one should drop the `UIP` axiom and use a notion of `Set`. The same goes for higher coinductive types.
- An interesting avenue is to look at the differences between the inductive definitions in Coq and the impredicative encodings. Perhaps inductive types in Coq can be translated to these impredicative encodings.
- We used the equality type from Homotopy Type Theory. Perhaps the Leibnitz equality '$a = b$' $:= \prod(P : A \to \mathcal{U}).P\ a \iff P\ b$ will also suffice. This would make the system smaller.
- It might be interesting to look at the exact difference or similarity between `Lim`, `Ind` and `Unq` mentioned in chapter 7.
- We have not looked at the exact circumstances under which the initial and final objects exist in this system. This has something to do with the "positivity" of the functor F and the existence of limits/colimits.

A System F typing rules

We define the syntax of System F using a BNF grammar. Let 'X' be a set of type variables and 'x' a set of term variables.

$$A, B ::= X \mid A \to B \mid \prod(X : \mathcal{U}).A \qquad \text{(types)}$$
$$t, u ::= x \mid \lambda(x : T).t \mid t\ u \mid \lambda(X : \mathcal{U}).t \mid t\ A \qquad \text{(terms)}$$

We write $A[X]$ to denote a type with a free type variable X and $t[x]$ to denote a term with a free term variable x. We write $A[X := B]$ for the substitution of X with the type B. Similarly, we write $t[x := u]$ for the substitution of x with the term u.
Next, we define the typing rules of System F.

$$\frac{}{\Gamma, x : A \vdash x : A}\ \text{id}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x : A).t : A \to B}\ \to\text{-intro} \qquad \frac{\Gamma \vdash t : A \to B \quad \Gamma \vdash u : A}{\Gamma \vdash t\ u : B}\ \to\text{-elim}$$

$$\frac{\Gamma, X : \mathcal{U} \vdash t : A}{\Gamma \vdash \lambda(X : \mathcal{U}).t : \prod(X : \mathcal{U}).A}\ \Pi\text{-intro}^* \qquad \frac{\Gamma \vdash t : \prod(X : \mathcal{U}).A}{\Gamma \vdash t\ A : B[X := A]}\ \Pi\text{-elim}$$

*where X is not free in Γ

Finally, we define β-reduction for System F.

$$(\lambda(x.A).t[x])\ u \overset{\beta}{=} t[x := u]$$
$$(\lambda(X : \mathcal{U}).t[X])\ A \overset{\beta}{=} t[X := A]$$

B Data types in System F

Definition B.1. We define a System F **unit** type.

$$\begin{aligned}
\mathbf{1}^* &:= \prod(X : \mathcal{U}).X \to X && \text{(formation)}\\
\mathtt{tt}^* &:= \lambda(X : \mathcal{U}).\lambda(x : X).x && \text{(introduction)}\\
\mathtt{rec}^*_{\mathbf{1}} &:= \lambda(X : \mathcal{U}).\lambda(x : X).\lambda(u : \mathbf{1}^*).u\ X\ x && \text{(elimination)}
\end{aligned}$$

Definition B.2. We define a System F **empty** type.

$$\begin{aligned}
\mathbf{0}^* &:= \prod(X : \mathcal{U}).X && \text{(formation)}\\
\mathtt{rec}^*_{\mathbf{0}} &:= \lambda(X : \mathcal{U}).\lambda(z : \mathbf{0}).z\ X && \text{(elimination)}
\end{aligned}$$

Definition B.3. We define a System F **coproduct** type.

$$\begin{aligned}
A +^* B &:= \prod(X : \mathcal{U}).(A \to X) \to (B \to X) \to X && \text{(formation)}\\
\mathtt{inl}^* &:= \lambda(a : A).\lambda(X : \mathcal{U}).\lambda(f : A \to X).\lambda(g : B \to X).f\ a && \text{(introduction)}\\
\mathtt{inr}^* &:= \lambda(b : B).\lambda(X : \mathcal{U}).\lambda(f : A \to X).\lambda(g : B \to X).g\ b && \text{(introduction)}\\
\mathtt{rec}^*_{A+^*B} &:= \lambda(X : \mathcal{U}).\lambda(f : A \to X).\lambda(g : B \to X).\lambda(t : A +^* B).t\ X\ f\ g && \text{(elimination)}
\end{aligned}$$

We denote $[f, g] := \mathtt{rec}^*_{A+^*B}\ X\ f\ g$ when X, A and B are clear from the context.

Definition B.4. We define a System F **product** type.

$$\begin{aligned}
A \times^* B &:= \prod(X : \mathcal{U}).(A \to B \to X) \to X && \text{(formation)}\\
\mathtt{pair}^* &:= \lambda(a : A).\lambda(b : B).\lambda(X : \mathcal{U}).\lambda(f : A \to B \to X).f\ a\ b && \text{(introduction)}\\
\mathtt{fst}^* &:= \lambda(p : A \times^* B).p\ A\ (\lambda(a : A).\lambda(b : B).a) && \text{(elimination)}\\
\mathtt{snd}^* &:= \lambda(p : A \times^* B).p\ B\ (\lambda(a : A).\lambda(b : B).b) && \text{(elimination)}
\end{aligned}$$

C Inference rules of the system

These rules are partly taken from Appendix A of the master's thesis of Sam Speight [Spe17] and are equal to those of HoTT [HOT13].

Inference rules for product types

$$\frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, a : A \vdash B : \mathcal{U}}{\Gamma \vdash \prod(a : A).B : \mathcal{U}}\ \Pi\text{-form}_1 \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, a : A \vdash B : \mathcal{U}_i}{\Gamma \vdash \prod(a : A).B : \mathcal{U}_i}\ \Pi\text{-form}_2$$

$$\frac{\Gamma, a : A \vdash b : B}{\Gamma \vdash \lambda(a : A).b : \prod(a : A).B}\ \Pi\text{-intro} \qquad \frac{\Gamma \vdash f : \prod(x : A).B \quad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[x := a]}\ \Pi\text{-elim}$$

$$\frac{\Gamma, x : A \vdash b : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda x.b)\ a \overset{\beta}{=} b[x := a] : B[x := a]}\ \Pi\text{-}\beta \qquad \frac{\Gamma \vdash f : \prod(x : A).B}{\Gamma \vdash f = (\lambda x.f\ x) : \prod(x : A).B}\ \Pi\text{-}\eta$$

Inference rules for sigma types

$$\frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma, a : A \vdash B : \mathcal{U}}{\Gamma \vdash \sum(a : A).B : \mathcal{U}}\ \Sigma\text{-form}_1 \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, a : A \vdash B : \mathcal{U}_i}{\Gamma \vdash \sum(a : A).B : \mathcal{U}_i}\ \Sigma\text{-form}_1$$

$$\frac{\Gamma, x : A \vdash B : \mathcal{U}_i \quad \Gamma \vdash a : A \quad \Gamma \vdash b : B[x := a]}{\Gamma \vdash \langle a, b\rangle : \sum(x : A).B}\ \Sigma\text{-intro}$$

$$\frac{\Gamma \vdash p : \sum(x : A).B}{\Gamma \vdash \texttt{pr1}\ p : A}\ \Sigma\text{-elim} \qquad \frac{\Gamma \vdash p : \sum(x : A).B}{\Gamma \vdash \texttt{pr2}\ p : B[x := \texttt{pr1}\ p]}\ \Sigma\text{-elim}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma, a : A \vdash b : B}{\Gamma \vdash \texttt{pr1}\ \langle a, b\rangle \overset{\beta}{=} a : A}\ \Sigma\text{-}\beta \qquad \frac{\Gamma \vdash a : A \quad \Gamma, x : A \vdash b : B}{\Gamma \vdash \texttt{pr2}\ \langle a, b\rangle \overset{\beta}{=} b : B[x := a]}\ \Sigma\text{-}\beta$$

$$\frac{\Gamma \vdash p : \sum(x : A).B}{\Gamma \vdash p = \langle \texttt{pr1}\ p,\ \texttt{pr2}\ p\rangle : \sum(x : A).B}\ \Sigma\text{-}\eta$$

Lemma 3.2.3. We have the following sigma injection principle.

$$(\hookrightarrow) : \prod(X :\ \mathcal{U}).\prod(P : X \to \mathcal{U}).\prod\big(y, y' : \sum(x : X).P\ x\big).y = y' \iff \mathtt{pr1}\ y = \mathtt{pr1}\ y'$$

Proof. Assume $(X :\ \mathcal{U})$, $(P : X \to \mathcal{U})$ and $\big(y, y' : \sum(x : X).P\ x\big)$.
($\implies$) Assume we know that $y = y'$. By the Σ-η rule, we know $y = \langle \mathtt{pr1}\ y,\ \mathtt{pr2}\ y\rangle$ and $y' = \langle \mathtt{pr1}\ y',\ \mathtt{pr2}\ y'\rangle$. Because $y = y'$, we know that $\langle \mathtt{pr1}\ y,\ \mathtt{pr2}\ y\rangle = \langle \mathtt{pr1}\ y',\ \mathtt{pr2}\ y'\rangle$. By taking the first projection of both sides, we see that $\mathtt{pr1}\ y = \mathtt{pr1}\ y'$.
($\impliedby$) Assume we know that $\mathtt{pr1}\ y = \mathtt{pr1}\ y'$. Because y and y' are well-formed, we know that $(\mathtt{pr2}\ y : P\ (\mathtt{pr1}\ y))$ and $(\mathtt{pr2}\ y' : P\ (\mathtt{pr1}\ y'))$. Since $\mathtt{pr1}\ y = \mathtt{pr1}\ y'$ we obtain by `UIP` that $P\ (\mathtt{pr1}\ y) = P\ (\mathtt{pr1}\ y')$. We thus we have that $\mathtt{pr1}\ y = \mathtt{pr1}\ y'$ and that $\mathtt{pr2}\ y = \mathtt{pr2}\ y'$ and conclude that $y = y'$. □

Inference rules for equality types

$$\frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash (a =_A b) : \mathcal{U}}\ =\text{-form}_1 \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash (a =_A b) : \mathcal{U}_i}\ =\text{-form}_1$$

$$\frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma \vdash a : A}{\Gamma \vdash \mathtt{refl} : a =_A a}\ =\text{-intro}$$

$$\frac{\begin{array}{c}\Gamma \vdash a : A \qquad \Gamma \vdash b : B \qquad \Gamma \vdash p : a =_A b \\ \Gamma, x : A, y : A, p : a =_A b \vdash C : \mathcal{U}_i \qquad \Gamma, z : A \vdash c : C[z, z, x := \mathtt{refl}_z, y, p]\end{array}}{\Gamma \vdash \mathtt{ind}_=(c, a, b, q) : C[a, b, x := q, y, p]}\ =\text{-elim}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma, x : A, y : A, p : a =_A b \vdash C : \mathcal{U}_i \qquad \Gamma, z : A \vdash c : C[z, z, x := \mathtt{refl}_z, y, p]}{\Gamma \vdash \mathtt{ind}_=(c, a, a, \mathtt{refl}_a) \overset{\beta}{=} c[z := a] : C[a, a, x := \mathtt{refl}_a, y, p]}\ =\text{-}\beta$$

Axioms

$$\mathtt{FunExt} \quad : \quad \Big(\prod(x : X).f\ x = g\ x\Big) \implies f = g$$

$$\mathtt{UIP} \quad : \quad \prod(X : \mathcal{U}) \prod(x, y : X) \prod(p, q : x = y).p = q$$

$$\mathtt{ExistsId} \quad : \quad \mathtt{rec}_\exists\ (\exists X.P)\ \mathtt{pack} = \mathtt{id}_{\exists X.P}$$

D CoLimStr is no equivalence relation

In this appendix, we elaborate on why the relation `CoLimStr` of 6.4.2 is not an equivalence relation. In fact, it is neither *symmetric* nor *transitive*. The relation is reflexive.

Reflexivity

Suppose $(\sigma : \texttt{Stream}^*)$. Does $(\texttt{CoLimStr}\ \sigma\ \sigma)$ hold? We use the recursor $\texttt{rec}_\exists$, we obtain some $(X : \mathcal{U})$, $(h : X \to E)$, $(t : X \to X)$ and $(x : X)$. Together with `ExistsId`, we know that $\sigma = \texttt{corec}_\texttt{s}\ X\ h\ t\ x$. It is easy to see that the identity function forms a morphism. Therefore we have $\texttt{CoLimStr}\ \sigma\ \sigma$. Note that without `ExistsId`, we are unable to prove that $\sigma = \texttt{corec}_\texttt{s}\ X\ h\ t\ x$.

Symmetry

Suppose $(\sigma, \tau : \texttt{Stream}^*)$ and $(\texttt{CoLimStr}\ \sigma\ \tau)$. Does $(\texttt{CoLimStr}\ \tau\ \sigma)$ hold? By $(\texttt{CoLimStr}\ \sigma\ \tau)$ we have $(X : \mathcal{U})$, $(h : X \to E)$, $(t : X \to X)$, $(x : X)$ and $(Y : \mathcal{U})$, $(h' : Y \to E)$, $(t' : Y \to Y)$ and $(f : X \to Y)$ such that $\sigma = \texttt{corec}_\texttt{s}\ X\ h\ t\ x$, $\tau = \texttt{corec}_\texttt{s}\ Y\ h'\ t'\ (f\ x)$ and $(\texttt{MorphStream}\ X\ h\ t \quad Y\ h'\ t \quad f)$ hold. In this type system, we cannot simply inverse f to obtain a function $(g : Y \to X)$.

Transitivity

Suppose $(\sigma, \tau, \upsilon : \texttt{Stream}^*)$, $(\texttt{CoLimStr}\ \sigma\ \tau)$ and $(\texttt{CoLimStr}\ \tau\ \upsilon)$. From $(\texttt{CoLimStr}\ \sigma\ \tau)$ we obtain the left column, from $(\texttt{CoLimStr}\ \tau\ \upsilon)$, we obtain the right column:

Let $h : X \to E$	Let $\widetilde{h} : \widetilde{Y} \to E$
$t : X \to X$	$\widetilde{t} : \widetilde{Y} \to \widetilde{Y}$
$x : X$	$\widetilde{y} : \widetilde{Y}$
$Y : \mathcal{U}$	$Z : \mathcal{U}$
$h' : Y \to E$	$h'' : Z \to E$
$t' : Y \to Y$	$t'' : Z \to Z$
$f : X \to Y$	$g : \widetilde{Y} \to Z$
$\texttt{MorphStream}\ X\ h\ t \quad Y\ h'\ t' \quad f$	$\texttt{MorphStream}\ \widetilde{Y}\ \widetilde{h}\ \widetilde{t} \quad Z\ h''\ t'' \quad g$
$\sigma = \texttt{corec}_\texttt{s}\ X\ h\ t\ x$	$\sigma = \texttt{corec}_\texttt{s}\ \widetilde{Y}\ \widetilde{h}\ \widetilde{t}\ \widetilde{y}$
$\tau = \texttt{corec}_\texttt{s}\ Y\ h'\ t'\ (f\ x)$	$\tau = \texttt{corec}_\texttt{s}\ Z\ h''\ t''\ (g\ y)$

We are unable to prove that $Y = \widetilde{Y}$, since the exact implementation of τ is hidden within the existential type. Neither can we obtain a proof of $Y = \widetilde{Y}$ because this proof lives in type universe $\mathcal{U}_1$ since $(Y : \mathcal{U})$.

References

[Gir71] Jean-Yves Girard. "Une Extension De LInterpretation De Gödel a LAnalyse, Et Son Application a LElimination Des Coupures Dans LAnalyse Et La Theorie Des Types". In: *Proceedings of the Second Scandinavian Logic Symposium*. Vol. 63. Studies in Logic and the Foundations of Mathematics. Elsevier, 1971, pp. 63–92. DOI: https://doi.org/10.1016/S0049-237X(08)70843-7. URL: https://www.sciencedirect.com/science/article/pii/S0049237X08708437.

[Rey74] John C. Reynolds. “Towards a theory of type structure”. In: *Programming Symposium, Proceedings Colloque sur la Programmation, Paris, France, April 9-11, 1974*. Vol. 19. Lecture Notes in Computer Science. Springer, 1974, pp. 408–423. DOI: 10.1007/3-540-06859-7_148. URL: https://doi.org/10.1007/3-540-06859-7%5C_148.

[GTL93] Jean-Yves Girard, Paul Taylor, and Yves Lafont. In: *Proofs and types*. Cambridge U.P., 1993.

[Geu14a] Herman Geuvers. “The Church-Scott representation of inductive and coinductive data”. In: 2014. URL: https://api.semanticscholar.org/CorpusID:13897083.

[Geu01] Herman Geuvers. “Induction is not derivable in second order dependent type theory”. In: *Lecture Notes in Computer Science* (2001), pp. 166–181. DOI: 10.1007/3-540-45413-6_16.

[Chu40] Alonzo Church. “A Formulation of the Simple Theory of Types”. In: *J. Symb. Log.* 5.2 (1940), pp. 56–68. DOI: 10.2307/2266170. URL: https://doi.org/10.2307/2266170.

[MS84] Per Martin-Löf and Giovanni Sambin. In: *Intuitionistic type theory*. Bibliopolis, 1984, pp. 79–86.

[HOT13] In: *Homotopy type theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013, pp. 154–163.

[CH88] Thierry Coquand and Gérard P. Huet. “The Calculus of Constructions”. In: *Inf. Comput.* 76.2/3 (1988), pp. 95–120. DOI: 10.1016/0890-5401(88)90005-3. URL: https://doi.org/10.1016/0890-5401(88)90005-3.

[Bar91] Henk Barendregt. “Introduction to generalized type systems”. In: *Journal of Functional Programming* 1.2 (1991), pp. 125–154. DOI: 10.1017/S0956796800020025.

[PP89] Frank Pfenning and Christine Paulin-Mohring. “Inductively Defined Types in the Calculus of Constructions”. In: *Mathematical Foundations of Programming Semantics, 5th International Conference, Tulane University, New Orleans, Louisiana, USA, March 29 - April 1, 1989, Proceedings*. Vol. 442. Lecture Notes in Computer Science. Springer, 1989, pp. 209–228. DOI: 10.1007/BFB0040259. URL: https://doi.org/10.1007/BFb0040259.

[Pau93] Christine Paulin-Mohring. “Inductive Definitions in the system Coq - Rules and Properties”. In: *Typed Lambda Calculi and Applications, International Conference on Typed Lambda Calculi and Applications, TLCA ’93, Utrecht, The Netherlands, March 16-18, 1993, Proceedings*. Vol. 664. Lecture Notes in Computer Science. Springer, 1993, pp. 328–345. DOI: 10.1007/BFB0037116. URL: https://doi.org/10.1007/BFb0037116.

[AFS18] Steve Awodey, Jonas Frey, and Sam Speight. “Impredicative Encodings of (Higher) Inductive Types”. In: *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*. ACM, 2018, pp. 76–85. DOI: 10.1145/3209108.3209130. URL: https://doi.org/10.1145/3209108.3209130.

[Spe17] Sam Speight. *Impredicative Encodings of Inductive Types in Homotopy Type Theory*. Master's thesis. Pittsburgh, USA: Carnegie Mellon University, 2017. URL: https://www.cs.ox.ac.uk/people/sam.speight/publications/sams-hott-thesis.pdf0.

[JR12] Bart Jacobs and Jan Rutten. "An introduction to (co)algebra and (co)induction". In: *Advanced Topics in Bisimulation and Coinduction*. Vol. 52. Cambridge tracts in theoretical computer science. Cambridge University Press, 2012, pp. 38–99.

[NG14] Rob Nederpelt and Herman Geuvers. *Type Theory and Formal Proof: An Introduction*. Cambridge University Press, 2014.

[Rip23] Xavier Ripoll Echeveste. *Alternative Impredicative Encodings of Inductive Types*. Master's thesis. Universiteit van Amsterdam, 2023.

[Geu14b] Herman Geuvers. "The Church-Scott representation of inductive and coinductive data". In: 2014. URL: https://api.semanticscholar.org/CorpusID:13897083.

[nLa24] nLab authors. *hom-functor preserves limits*. https://ncatlab.org/nlab/show/hom-functor+preserves+limits. Revision 11. June 2024.

[Rij22] Egbert Rijke. *Introduction to Homotopy Type Theory*. 2022. arXiv: 2212.11082 [math.LO]. URL: https://arxiv.org/abs/2212.11082.

[Ott20] Daniël Otten. "M-types and Bisimulation". Bachelor's Thesis. Leiden University, 2020. URL: https://theses.liacs.nl/pdf/2019-2020-OttenDD.pdf.